

INTRODUCTION TO VISUAL COMPUTING - VU 186.822

Summary of the Lecture Units

in Computer Graphics

Research Unit of Computer Graphics
Institute of Visual Computing and
Human-Centered Technology

SS 2025

i	Preface	4
1	Introduction to Computer Graphics	5
	Who needs Computergraphics?	5
	Computer Graphics Software Components	6
2	Graphics Pipeline and Object Representation	7
	Graphics Pipeline	7
	Intermission: Graphs and Trees	7
	Polygon Lists (B-Reps)	7
	Constructive Solid Geometry (CSG)	8
	Quadtrees and Octrees	9
	Scenegraphs	10
	Other Object Representations	10
3	Transformations	11
	Simple 2D Transformations	11
	Translation	11
	Rotation	11
	Scaling	11
	Mirroring	11
	Homogeneous Coordinates	11
	3D Transformations	13
4	Color	15
	What is Color?	15
	Colorimetry	15
	Color Vision Deficiency	16
	Color Models	16
	CIE 1931 XYZ color model	16
	RGB color model	17
	CMY color model	17
	HSV and HLS color models	17
	Color Symbolism	18
5	Rasterization	19
	Line algorithms	19
	DDA Procedure	19
	Bresenham Algorithm	19
	Attributes	21
	Attributes of (points and) lines	21
	Attributes of Text	21
	Attributes of (2D) polygons and surfaces	21
	Barycentric coordinates	21
	Rasterizing triangles	21
	calculation of the barycentric coordinates	21
	What is the inside of a polygon	22
6	Viewing	23
	Viewing and Projection Transformation	23
	Viewport transformation	23
	Projection transformation	23
	Camera transformation	24
	Orthographic viewing	25
	Textures on Perspective Distorted Triangles	25
7	Clipping and Antialiasing	27
	Line Clipping	27
	Clipping of Lines: Cohen-Sutherland Method	27

Polygon Clipping	28
Aliasing and Antialiasing	28
Sampling and Reconstruction in Spatial and Frequency Domains	30
8 Visibility	32
Backface Detection (Backface Culling)	32
Z-Buffer-Method (Depth Buffer)	32
Scanline Method	32
Depth Sorting Method	32
Area Subdivision Method	33
Octree Method	33
Ray Casting	33
Classification of Visibility Algorithms	34
9 Lighting and Shading	35
Light Sources and Surfaces	35
Light Sources	35
Object Surfaces	35
A Simple Lighting Model	35
Background Light (Ambient Light)	35
Lambert's Law	35
Specular Highlights	36
Shading of Polygons	37
Flat Shading	37
Gouraud Shading	37
Phong Shading	38
10 Ray Tracing	39
The Ray Tracing Principle	39
Correct Visibility and Shading	39
Shading	39
Mirror Images	39
Transparency	40
Recursion	40
Perspective	40
Ray-Tracing Implementation	40
Intersections between Beams and Objects	41
Beam-Ball Section	41
Beam-Polygon Intersection	42
Ray-Tracing Acceleration	42
Object Environments	42
Space Division Methods	43
11 Global Illumination and Textures	44
Radiosity	44
The Radiosity Equation	44
Calculation of Form Factors	44
Progressive Refinement	45
Aspects of Radiosity	46
Path Tracing	46
Photon Mapping	47
Environment Mapping	47
Texture Mapping	48
Creating a Texture	48
Texture-Object-Transformation	48
Viewing and Projection Transformation	49
Anti-Aliasing for Textures	49
Textures on Perspective Distorted Triangles	49
Solid Texturing	50
Bump Mapping	50
Bump-Mapping Algorithm	50
Displacement Mapping	51

Combination of several Mappings	51
12 Curves and Surfaces	52
Quadratic Surfaces	52
Curves	52
Cubic Spline Interpolation	53
Hermite Interpolation	53
Bézier Curves	53
B-Spline Curves	54
Free-Form Surfaces	55
Bézier- and B-Spline Surfaces	55
13 Computer Animation	56
Transformations	56
Translation	56
Rotation	56
Keyframing	56
Deformations	58
Physically-based Simulation	58
Numerical Integration	58
Differential Equations	59
Character Animation	59
Procedural Techniques	59
14 Machine Learning for 3D Graphics	60
Machine Learning and Neural Networks	60
Generative Models	60
Using Generative Models to Create Images	61
Training a Generative Adversarial Network (GAN)	61
Training a Diffusion Model	61
Inference in a Diffusion Model	62
Scene Representations for 3D Machine Learning	62
Neural Radiance Fields (NeRFs)	62
Gaussian Splatting	63
Generative Models for 3D Objects	63

These lecture notes were written by Werner Purgathofer and additionally edited by Annalena Ulschmid, Maath Musleh, Áron Kovács, Johannes Eschner, and Daniel Pahr. Translations by Heinrich D. Ehlers. They are intended to provide a compact summary of the lecture, but should not be used as a stand-alone resource for exam preparation. We strongly recommend that you also read the lecture slides and consult additional resources (e.g. Marschner et al. (2022)).

If you notice any errors or inconsistencies or have suggestions for improvement please contact us by e-mail at dpahr@cg.tuwien.ac.at.

1. Introduction to Computer Graphics

Computer graphics refers to the branch of computer science that deals with artificially generated images and their manipulation. This also includes the digital representation, generation and manipulation of data.

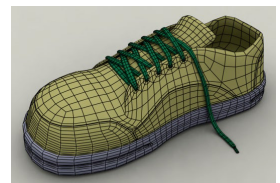
Who needs Computergraphics?

With the computerization of more and more areas of our lives, the number of applications that require artificial images is also increasing. The following list of typical applications is therefore incomplete.

Entertainment The entertainment industry uses artificial images and animations in a variety of ways. The development of computer games is the application of computer graphics with the largest market worldwide, and the funds invested in this industry have a significant share in the relevant research. The film industry also frequently requires computer graphics, on the one hand, to depict scenes that could not be filmed or could only be filmed with immense effort, and, on the other hand, to rework filmed scenes and add content. The methods of classic film production and classic animation are merging more and more in this way.



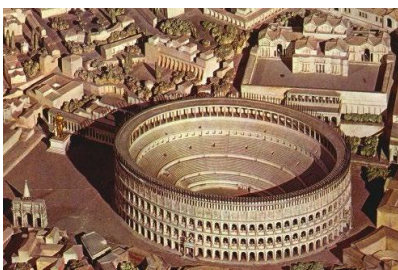
Computer Aided Design (CAD) Today, industrial products are generally developed and modeled on computers and, of course, visually inspected, i.e. represented with computer graphics. This applies to appliances, containers, sporting goods, and jewelry as well as cars, airplanes, and windows. In architecture, buildings are inspected virtually before they are built, and in road and landscape planning, the results are visually anticipated and taken into account in the planning process.



Advertising Commercials have been using computer graphics support for a long time. Nor should the financially strong advertising industry be overlooked, which uses all kinds of manipulations of film material in addition to short animations. Companies that use modern interactive visual methods for marketing have a demonstrable image advantage.



Simulators Learning to use some technologies can be either very expensive or dangerous. The training of airplane pilots and astronauts, even in extreme situations, has led to the development of very realistic-looking simulators whose technology is also used for simpler applications such as car simulation in driving schools. The simulation of other dangerous or catastrophic situations is also used to prepare and train people for them. Perception-based rendering, which takes into account the eye's ability to perceive under given circumstances, is also useful for this purpose.



Cultural Heritage The preservation or restoration of things and buildings that have fallen victim to the ravages of time is possible virtually with computer graphics. In addition to museums, such programs also support normal education, for example history lessons.

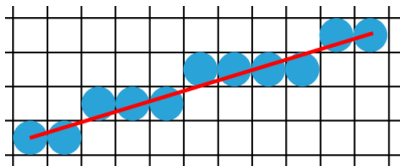
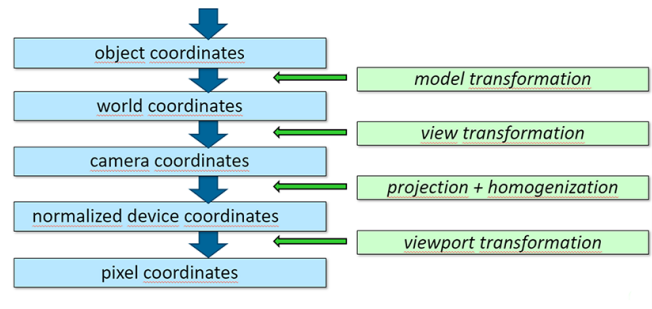
Science Visualization methods in particular make it possible to find structures and useful information in confusing or huge amounts of data. This is used exploratively in many sciences, but also routinely in medicine, for example (visualization of CT data, etc.).



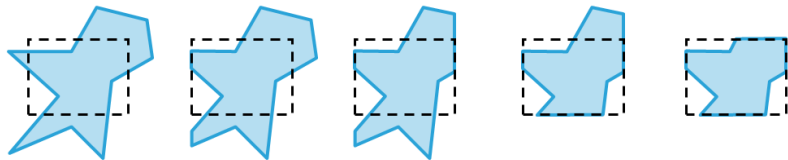
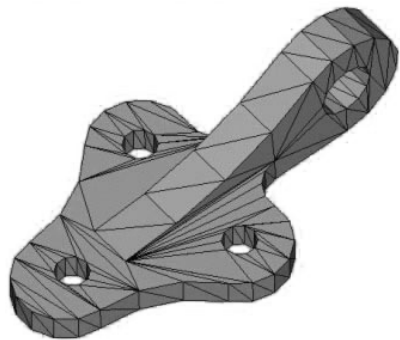
Computer Graphics Software Components

A graphics system comprises many components, from the modeling of the data to the efficient and correct display of the finished images. The resulting chain of operations and data is referred to as a *graphics pipeline*.

In order to be able to display images without having to take into account the respective technology of the output device, so-called *graphic primitives* are used, which are simple geometric shapes such as straight lines, circles or rectangles. These are converted into pixel information using *rasterization* operations. Objects are *modeled* by combining geometric primitives and storing them in suitable geometric data structures (this also includes *freeform surfaces*), of which there are several different ones.



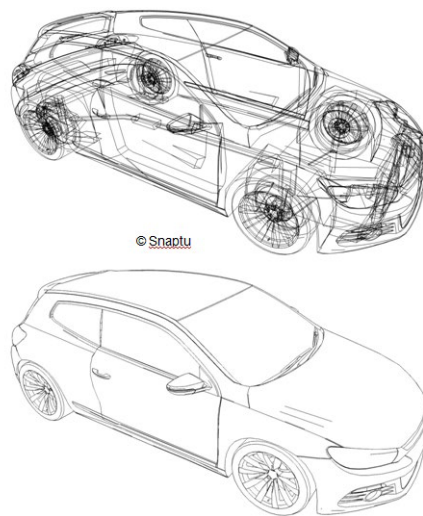
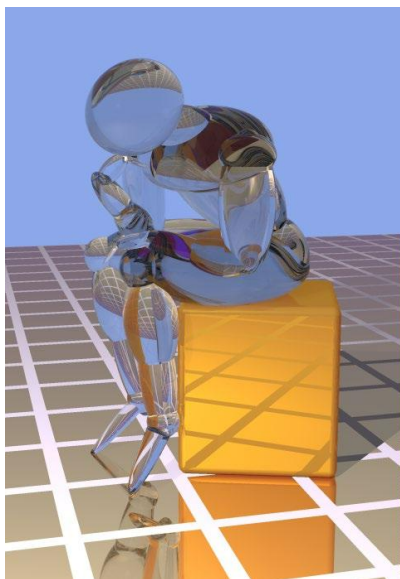
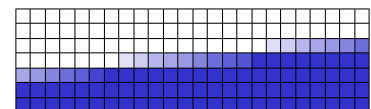
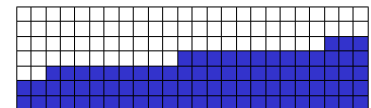
After placing the objects in a *world coordinate system*, the *projection* is determined by defining the corresponding *camera* parameters. Both operations require geometric *transformations*, which are usually represented as homogeneous matrices. Parts of the image that lie outside the viewing window must be omitted (*clipping*), as must image information that is obscured by other objects (*visibility calculation*).



For simple representations, simple *illumination models* are sufficient, which lead to shading of the object surfaces.

If realistic-looking images are required, then *Ray-Tracing* and/or global illumination calculation is used.

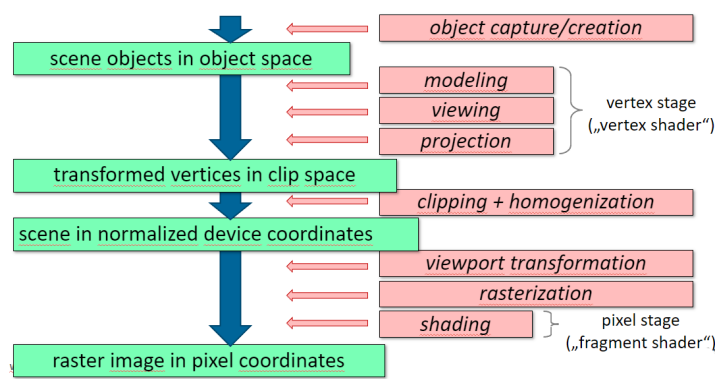
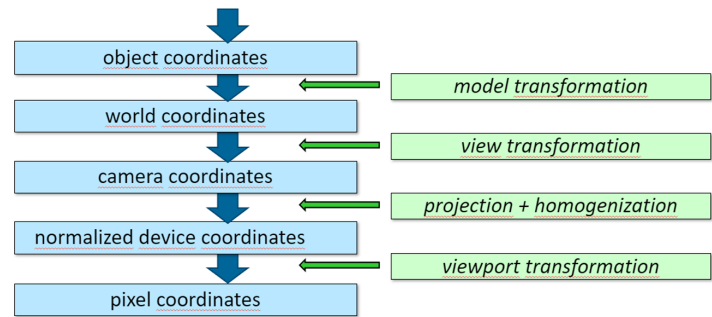
Furthermore, additional *textures* are often applied to surfaces, which can also be combined with local geometric structures. Finally, image output processes are used to reduce the raster impression of the output medium, so-called *anti-aliasing*.



Graphics Pipeline

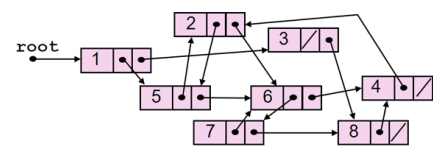
Information is transformed into an image in successive steps: first, the objects and the scene are described in some way, then the viewing direction is determined, then the objects can be projected accordingly, and finally, the result is converted into raster points. This sequence is called the graphics pipeline or, depending on the way it is viewed, the viewing pipeline (top image), transformation pipeline or rendering pipeline (bottom image).

Objects can be created by modeling or by scanning. The terms “vertex shader” and “fragment shader” are used for programmable parts of graphics cards. Note that there are many other similar illustrations that all describe the same basic principle of the graphics pipeline. Most of the other graphics chapters in this script fit directly into this scheme.

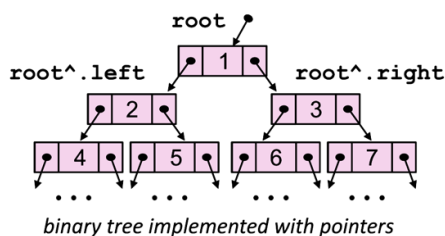


Intermission: Graphs and Trees

Similar to linked lists, graphs in a general sense, and trees in a more specific sense, can be represented with pointer-linked structures. The individual nodes must contain a sufficient number of pointer components so that the desired structure can be achieved, e.g. two pointers per node for a binary tree. Operations for inserting and removing nodes can be easily considered in the same way as for linked lists. Furthermore, the processing sequence of such a data structure is controlled by calling the appropriate successors.



arbitrary graph implemented with pointers

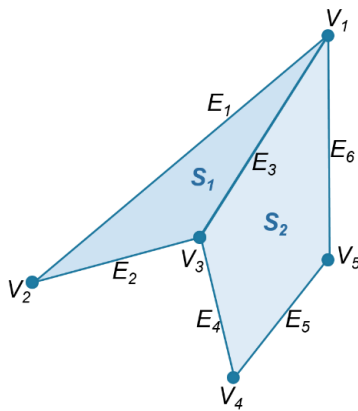


The recursive processing of a binary tree can be controlled in this way, for example:

- “pre-order”: 1st root, 2nd left successor, 3rd right successor (→ 1245367)
- “in-order”: 1st left successor, 2nd root, 3rd right successor (→ 4251637)
- “post-order”: 1. left successor, 2. right successor, 3. root (→ 4526731)

Polygon Lists (B-Reps)

Three-dimensional objects are usually represented by polygon lists (often triangles). A set of polygons that describes the surface of an object is called a *Boundary-Representation* (“B-Rep”). In addition to geometric information, data structures for B-Reps also contain attributes (properties). The geometry consists of point lists, edge lists, and surface lists and must be checked for consistency and completeness.



VERTEX TABLE	
V_1 :	x_1, y_1, z_1
V_2 :	x_2, y_2, z_2
V_3 :	x_3, y_3, z_3
V_4 :	x_4, y_4, z_4
V_5 :	x_5, y_5, z_5

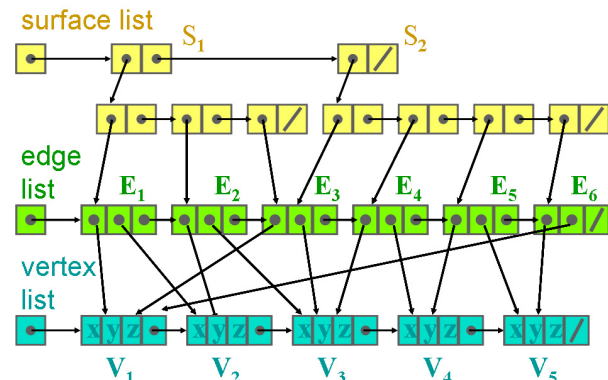
EDGE TABLE	
E_1 :	V_1, V_2
E_2 :	V_2, V_3
E_3 :	V_3, V_1
E_4 :	V_3, V_4
E_5 :	V_4, V_5
E_6 :	V_5, V_1

POLYGON TABLE	
S_1 :	E_1, E_2, E_3
S_2 :	E_3, E_4, E_5, E_6

The example on the left shows a very simple situation with 2 polygons and how the vertex table, edge table, and surface table reference each other. Only the vertex table contains the actual geometric information, the other tables only describe the topology.

The same structure can also be represented with pointer lists (see illustration on the right).

The entire coordinate information is located in the point nodes (V stands for vertex). If a point is transformed to a new location, it is sufficient to change the coordinates of this point. The topology remains unchanged. The linear linking of edges and points makes editing easier (e.g. draw all edges once or move all points).



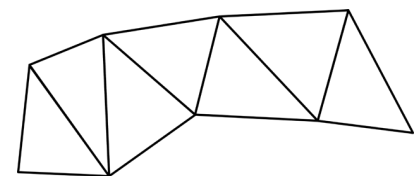
Some Terms: The representation of each individual polygon surface includes the support plane $Ax + By + Cz + D = 0$ and the corner points V_1 to V_n . From the plane parameters A, B, C, D you immediately get the normal vector on the plane (A, B, C) . The *Backface* is the back of the polygon, which looks into the object, and the *Frontface* is the front, which shapes the outside of the object. “Behind the polygon” means all points that are visible from the back face, “in front of the polygon” means that you can see the front face from there.

In a right-handed coordinate system, assuming the corner points of each polygon (viewed from the front) are arranged in the mathematically positive sense (i.e. counterclockwise), then the following applies to a point (x, y, z) :

- if $Ax + By + Cz + D = 0$ then the point lies **on** the plane
- if $Ax + By + Cz + D < 0$ then the point is **behind** the plane
- if $Ax + By + Cz + D > 0$ then the point is **in front** of the plane

Similarly, an outwardly directed normal vector N can be calculated from three consecutive vertices V_1, V_2, V_3 by $N = (V_2 - V_1) \times (V_3 - V_1)$. We will need this knowledge later.

Very often, only triangles are used as polygons because they have many properties that simplify the algorithms (e.g. triangles are always flat). A data structure consisting exclusively of triangles is also called *triangles-mesh*; a linear sequence of triangles (each additional triangle is indicated by only one point!) is called *triangle-strip* (see figure).

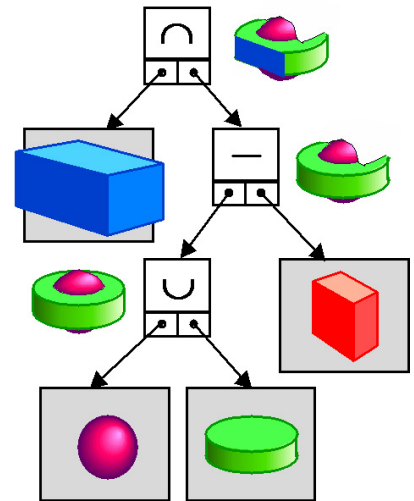


Constructive Solid Geometry (CSG)

In CSG, objects are constructed from three-dimensional primitives using set operations. They are arranged in such a way that they can be arranged in a hierarchical data structure, which - although it is actually only a circle-free graph - is normally called a CSG tree. The primitives are simply geometric shapes such as spheres, tetrahedrons, cubes, and cylinders, which are linked with the operators union, average, and difference. Since all primitives are trivially consistent and the operators only generate consistent objects from consistent parts, all objects in CSG are always consistent (no holes in the surface, well-defined interior).

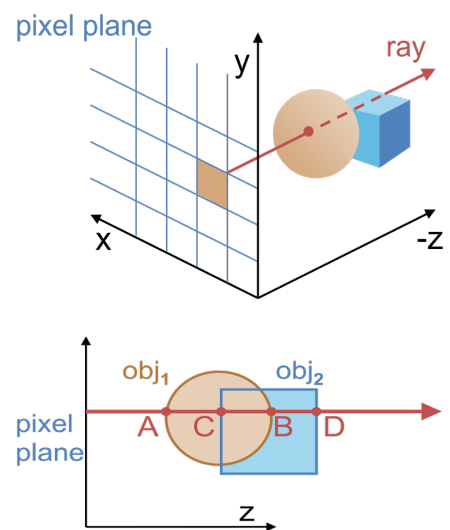
In addition, each node of a CSG tree contains transformations (in the form of matrices) that specify which transformations are applied to the subtree below. This gives primitives (e.g. axis-parallel unit cubes) a wide range of shapes (e.g. cuboids positioned anywhere in space), and even more complex objects can be moved, scaled, rotated, etc.

The advantages of CSG objects are the exact representation (a sphere is really a sphere!), the low memory requirements, and the simplicity of transformations. The main disadvantage is the much more complex calculation of images, i.e. the more complicated rendering. This requires either converting the entire data structure into a B-rep representation and rendering it in the conventional way, or using ray casting or ray tracing to create images directly.



Ray-Casting of CSG-Objects The most common method for mapping CSG objects is ray casting, in which the image is calculated pixel by pixel. For each pixel, a ray is cast in the viewing direction and intersected with all objects in the scene. The foremost of these intersection points indicates which object can be seen in this pixel, and the pixel is given its color. In a CSG tree, this calculation is performed recursively:

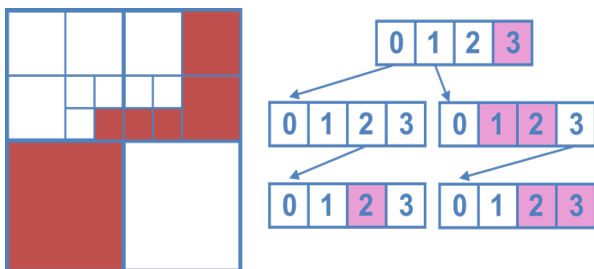
- For **end node** the calculation of all intersection points is simple.
- For **intermediate node**, the intersection point lists of the two successors are linked according to the operator: the lists (A,B) and (C,D) in the example on the right result in
 - at union the list (A,D),
 - for average the list (C,B),
 - for difference the list (A,C).
- For **tree root**, the first point of the linked intersection list is selected.



We will cover the topic of ray tracing, a superset of ray casting, in much more detail later.

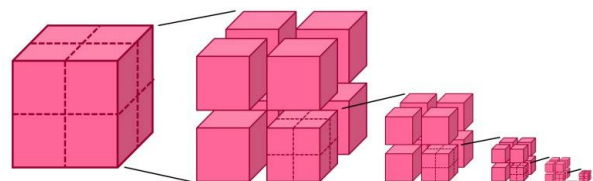
Quadtrees and Octrees

A *Quadtree* is a data structure that is suitable for representing any two-dimensional structure. The relevant area is divided into four quarters wherever the information is still too complicated to be stored simply, otherwise the simple information is stored. Each image area corresponds to a node of a tree in which each node has (at most) four successors (“quadtree”).



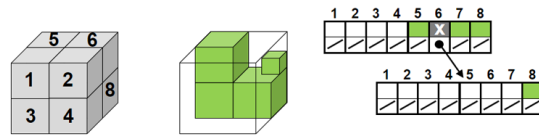
The example opposite shows a simple quadtree that represents a two-color simple graph. The root node corresponds to the entire image, the nodes in the second row correspond to the top two quarters of the image and the last two nodes correspond to the two areas with the finest resolution.

An *Octree* is the extension of this concept to three dimensions. An arbitrarily shaped object (or even an entire scene) within a cube is represented by “simple” subcubes (empty or entirely within an object) being described by end nodes, and more complicated subcubes (all others!) being divided into eight smaller subcubes



(octants), to which the same rules are applied again (recursively). This creates a tree in which each node has up to 8 successors (“octree”). The subdivision also stops when the subcubes fall below a certain minimum size (e.g. one thousandth of the total size); in this case, the node of the tree receives the best possible simple information. This happens at least with all slanted surfaces at some point, and then these edge cubes must be declared as either inside or outside. The example below with only two hierarchical levels also shows the problem of limited spatial resolution. The character string is an example of a linearization of the octree information, e.g. for saving in a file.

Octrees have the advantage that you can represent any shape and that you can quickly examine what is located at a certain spatial position. The disadvantages are: imprecise representation, high memory requirements, complicated transformations. Octrees are processed recursively in the same way as quadtrees. Set operations are therefore very simple, but geometric transformations (apart from exceptions) are complex because the octree has to be completely regenerated. The rendering of octrees, on the other hand, is simple when using an overwritable memory:



linearization: X(EFFESX(EFFEEESS)SS)
E ... Empty, S ... Solid, X ... Mixed

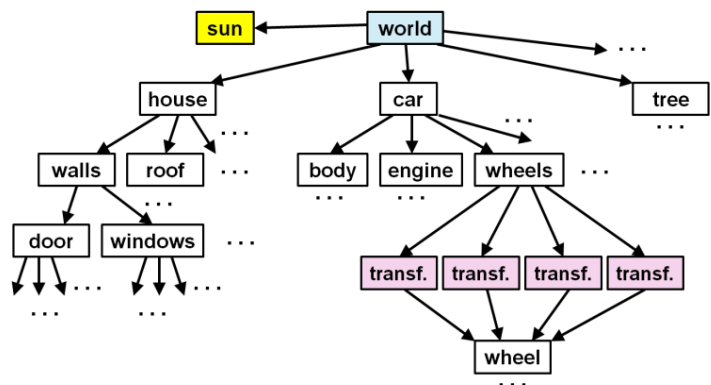
```
1 if node is simple
2 then draw node {i.e. do nothing if node is empty}
3 else recursively call the 8 octants from back to front
```

Scenegraphs

A scene graph is an object-oriented data structure with which the logical and/or spatial arrangement of individual elements of a (two- or) three-dimensional scene is described hierarchically. This term is not precisely defined, it is rather a generic term for all possible hierarchical forms of description for objects that are to be represented graphically. In terms of graph theory, it is a tree-like directed circle-free graph whose root node represents the entire scene, each intermediate node describes a sub-scene (after all, it is the root of a partial “tree”), and the end nodes represent the simplest objects in the scene (these can also be very different representations). In OpenSceneGraph, VRML and X3D, scene graphs are central concepts.

As an example, let’s think of a city scene where the entire scene graph represents the city. Each house corresponds to an intermediate node, each window also corresponds to an intermediate node, but further down in the “tree”, and simple objects such as window panes or screws could be the simplest objects. If a part occurs more than once, then you can use the existing information several times in the scene graph, but then you need information on where and in what position and size these parts occur.

Intermediate nodes contain information that affects the entire subgraph, such as the material or color, the position and location in space, the size, and a possible distortion. All geometric transformations can be stored very elegantly in matrices - we will see how this works in the next chapter.



Other Object Representations

There is a wealth of other object representations and associated data structures, some of which have been developed for very specific objects and applications. These include BSP trees, fractals, graphical grammars and procedural models, particle systems, physically based models, three-dimensional volume data, and some others. We will take a closer look at curved surfaces and freeform surfaces later. The other data structures will be covered in more advanced courses.

3. Transformations

Geometric transformations are operations like moving, enlarging and reducing, rotating, mirroring, etc. objects within a coordinate system or between coordinate systems. We will always describe the necessary rules for points so that all other objects can also be easily transformed by transforming their defining points.

Simple 2D Transformations

Translation

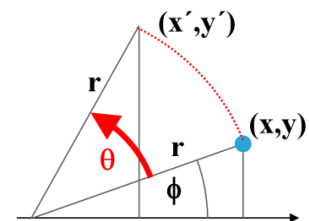
Moving a point (x, y) by the vector (t_x, t_y) provides the transformed point:

$$(x', y') = (x + t_x, y + t_y)$$

Rotation

By rotating an object with the angle θ around the coordinate origin, the point (x, y) due to $x = r \cdot \cos\theta$ and $y = r \cdot \sin\theta \Rightarrow x' = r \cdot \cos(\phi + \theta) = r \cdot \cos\phi \cdot \cos\theta - r \cdot \sin\phi \cdot \sin\theta = x \cdot \cos\theta - y \cdot \sin\theta$ (y' analog) is moved to

$$(x', y') = (x \cdot \cos\theta - y \cdot \sin\theta, x \cdot \sin\theta + y \cdot \cos\theta).$$



Scaling

When scaling an object by the factor s around the origin $(0, 0)$, a point (x, y) is mapped to

$$(x', y') = (s \cdot x, s \cdot y).$$

If different scaling factors s_x and s_y are used in the x and y directions, the result is

$$(x', y') = (s_x \cdot x, s_y \cdot y).$$

Mirroring

Mirroring on a coordinate axis is a special case of scaling with $s_x = -1$ or $s_y = -1$.

All other transformations can be achieved by carrying out the simple transformations described one after the other. These mappings (with the exception of translation) can also be represented by transformation matrices. The points are represented as vectors so that the matrix operations can be carried out with them:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} \quad \begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} \quad \begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} \quad (x', y') = (x + t_x, y + t_y) \quad \dots?$$

(a) Scaling

(b) Rotation (counterclockwise)

(c) Mirroring around x-axis

(d) Translation

Homogeneous Coordinates

So that the translation can also be specified in matrix notation, *homogeneous coordinates* are used. Each point is assigned an additional coordinate h , whereby the conversion to 2D coordinates is carried out by dividing the x and y components by h . Therefore, $h = 1$ is usually used. For the point (x, y) we therefore write $(x, y, 1)$. The transformation matrices are extended by a row and column with unit values:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

(a) 2D-Scaling

(b) 2D-Rotation

(c) 2D-Translation

Why is it advantageous to formulate all transformations in uniform matrix notation? In most cases, larger parts (objects, images) are transformed as a whole, i.e. the same sequence of transformations is applied to each point of these entities. This corresponds to a sequential multiplication of a point P with matrices $M_1, M_2, M_3, \dots$: $P' = M_1 \cdot P, P'' = M_2 \cdot P', P''' = M_3 \cdot P'', \dots$ Now you can make use of the associativity of matrix multiplication [i.e. $(M_1 \cdot M_2) \cdot M_3 = M_1 \cdot (M_2 \cdot M_3)$] and thus massively reduce the computational effort:

$$\text{Instead of } \mathbf{P}^{(n)} = \mathbf{M}_n \cdot (\mathbf{M}_{n-1} \cdot \dots (\mathbf{M}_3 \cdot (\mathbf{M}_2 \cdot (\mathbf{M}_1 \cdot \mathbf{P}))) \dots) \\ \text{we can write } \mathbf{P}^{(n)} = (\mathbf{M}_n \cdot \mathbf{M}_{n-1} \cdot \dots \cdot \mathbf{M}_3 \cdot \mathbf{M}_2 \cdot \mathbf{M}_1) \cdot \mathbf{P}.$$

Now you can calculate $M = (M_n \cdot M_{n-1} \cdot \dots \cdot M_3 \cdot M_2 \cdot M_1)$ beforehand and then apply this one total matrix to all points.

We want to give the simple transformation matrices names for a clearer presentation:

- $\mathbf{T}(t_x, t_y)$ = translation around the vector (t_x, t_y)
- $\mathbf{R}(\theta)$ = rotation around the angle θ [pos. angle = counterclockwise rotation].
- $\mathbf{S}(s_x, s_y)$ = scaling by the factors s_x and s_y .

The inverse transformations of these simple transformations are:

$$T^{-1}(t_x, t_y) = T(-t_x, -t_y) \quad R^{-1}(\theta) = R(-\theta) \quad S^{-1}(s_x, s_y) = S(1/s_x, 1/s_y)$$

More complex transformations can now be derived from these simple transformations.

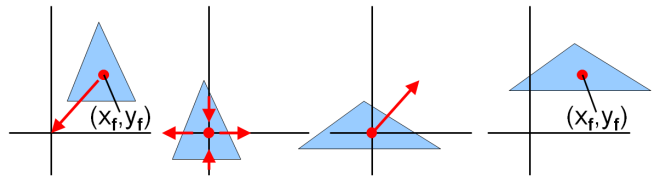
As an example, we consider the

Scaling around a point other than the coordinate origin:

1. step = Move the scaling center to the origin of the coordinates: $T(-x_f, -y_f)$
2. step = Scale the object in the coordinate origin: $S(s_x, s_y)$
3. step = Move the object back to the original position: $T^{-1}(-x_f, -y_f) = T(x_f, y_f)$

The general matrix for the scaling with (x_f, y_f) as the center is now obtained as follows:

$$S(x_f, y_f, s_x, s_y) = T(x_f, y_f) \cdot S(s_x, s_y) \cdot T(-x_f, -y_f)$$



As a further example, we consider the

Reflection on any axis $y = mx + b$:

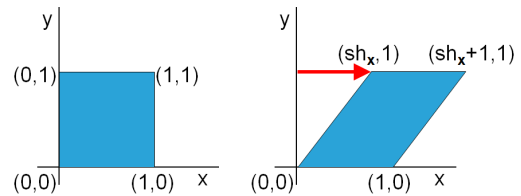
1. step = move so that the axis passes through the origin of the coordinates: $T(0, -b)$
2. step = Rotate so that the axis coincides with the x-axis, for example: $R(-\theta)$ [$m = \tan\theta$]
3. step = mirroring on the x-axis: $S(1, -1)$
4. step = turn back so that the axis has the original angle: $R^{-1}(-\theta) = R(\theta)$
5. step = move back so that the axis is at the original position: $T^{-1}(0, -b) = T(0, b)$

The general matrix for the reflection on the axis $y = mx + b$ is now obtained as follows:

$$X(m, b) = T(0, b) \cdot R(\theta) \cdot S(1, -1) \cdot R(-\theta) \cdot T(0, -b)$$

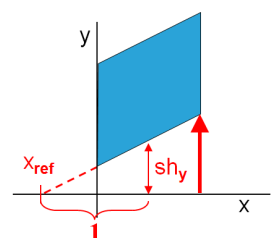
Another important transformation is the shear, which in the simplest case in the x-direction with a fixed x-axis has the form:

$$\begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



More generally, the shearing can also take place parallel to an axis, which looks like this in the y-direction:

$$\begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & -sh_y \cdot x_{ref} \\ 0 & 0 & 1 \end{bmatrix}$$



Of course, you can now easily derive the general matrix for a shear that is not parallel to one of the coordinate axes: first rotate to a position parallel to the axis, then shear, and then rotate back again.

The general viewing transformation can also be easily described using transformation matrices. The operations used are shifting one origin to the other, rotating the viewport in the axis directions of the window, and scaling the axes. In contrast to the composite transformations discussed above, however, there is no backward rotation, backward shifting, etc. here.

Affine Transformations All transformations discussed here are *affine transformations*, i.e. the coordinates can be transformed into each other using linear functions plus a translation. Affine mappings maintain collinearity, i.e. (each of) 3 points on a straight line are also on a straight line after the mapping, and the proportionality of distances along a straight line, i.e. the ratio of lengths on a straight line, is maintained. Furthermore, parallel lines always remain parallel, and finite points remain finite. All affine transformations can be composed of scaling, rotation, and translation (including shearing!). Affine transformations in which only rotation, translation, and mirroring are used are also length- and angle-preserving.

3D Transformations

All 2D concepts can be easily extended to 3D. Again, you need a homogeneous component so that 4x4 matrices operate on 4-dimensional vectors. Later we will see that projections can also be formulated in this way.

Here are the most important 3D transformations:

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad \begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad \begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

(a) 3D-Scaling

(b) 3D-Translation

(c) Mirroring around yz/xz/xy plane

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad \begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad \begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

(d) 3D Rotation around x-axis

(e) 3D Rotation around y-axis

(f) 3D Rotation around z-axis

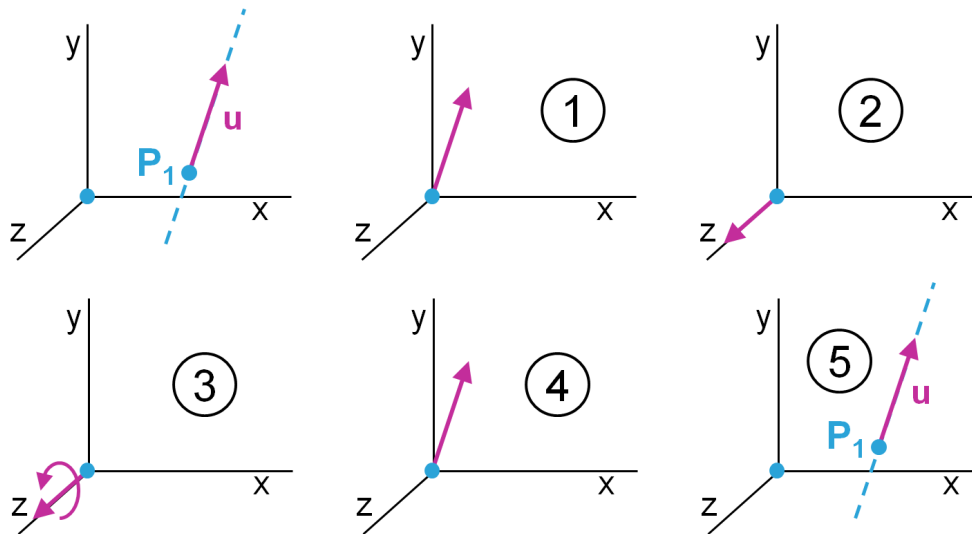
The names for these simple transformation matrices are

- $\mathbf{T}(t_x, t_y, t_z)$ = translation around the vector (t_x, t_y, t_z)
- $\mathbf{R}_x(\theta)$ = rotation by the angle θ around the x-axis; y- and z-axis analogous
- $\mathbf{S}(s_x, s_y, s_z)$ = Scaling by the factors s_x, s_y and s_z .

As an example of a more complex transformation, let's use a

rotation by the angle θ around any axis in space

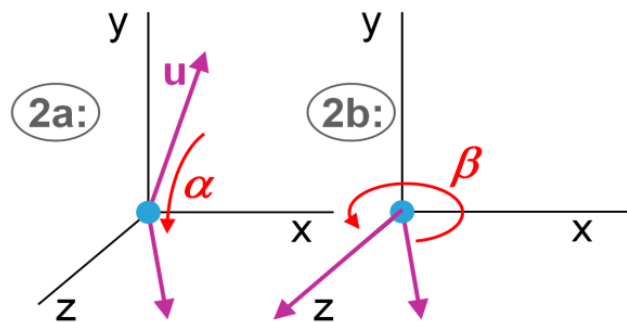
can be derived. Let the axis be given by a point $P_1(x_1, y_1, z_1)$ and a direction vector u .



1. step = move point P_1 to the origin: $T(-x_1, -y_1, -z_1)$
2. step = rotate vector u in the z-axis
 - (a) Rotate vector u around the x-axis in the xz-plane: $R_x(\alpha)$
 If $u = (a, b, c)$, then $u' = (0, b, c)$ is the projection of u onto the yz-plane. The angle of rotation α around the x-axis results from $\cos\alpha = c/d$ with $d = \sqrt{b^2 + c^2}$
 - (b) Rotate vector u around the y-axis into the z-axis: $R_y(\beta)$
 The angle of rotation β around the y-axis results from $\cos\beta = d$ (or $\sin\beta = -a$)
3. step = perform rotation by θ around the z-axis: $R_z(\theta)$
4. step = rotate vector u back in the original direction: first $R_y(-\beta)$, then $R_x(-\alpha)$
5. step = move point P_1 back to the original position: $T(x_1, y_1, z_1)$

The resulting matrix is calculated as follows:

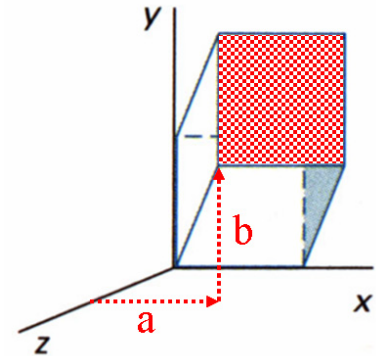
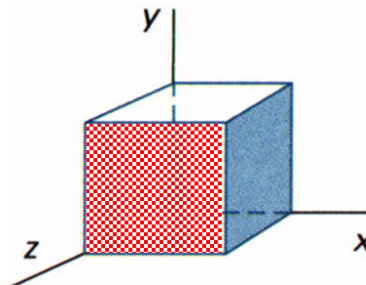
$$\begin{aligned}
 R(\theta) &= T^{-1}(-x_1, -y_1, -z_1) \cdot R_x(-\alpha) \cdot R_y(-\beta) \cdot R_z(\theta) \cdot R_y(\beta) \cdot R_x(\alpha) \cdot T(-x_1, -y_1, -z_1) = \\
 &= T(x_1, y_1, z_1) \cdot R_x(-\alpha) \cdot R_y(-\beta) \cdot R_z(\theta) \cdot R_y(\beta) \cdot R_x(\alpha) \cdot T(-x_1, -y_1, -z_1)
 \end{aligned}$$



A **shear in 3D** is also easy to display:

A shear parallel to the xy-plane by the value a in the x-direction and the value b in the y-direction can be achieved using

$$\begin{bmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

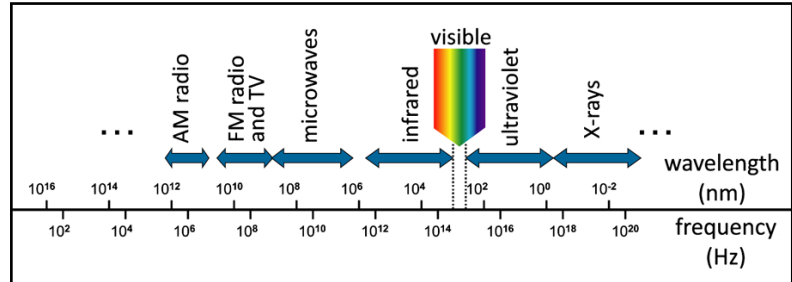


A shear with a fixed plane other than one of the coordinate principal planes can be easily derived.

Color is one of the basic essences of computer graphics. Understanding and handling color correctly is a basic tool for computer graphic artists. However, the commonly used RGB color model is not capable of representing all colors and is otherwise very approximate. Many color calculations are usually only approximate (which is often sufficient), and exact color theory is very complex.

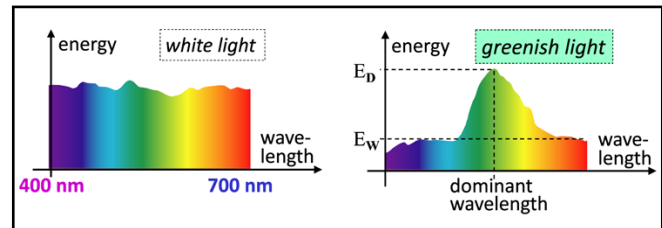
What is Color?

Our eye can detect electromagnetic radiation in the rather narrow frequency range between about $3.8 \cdot 10^{14} \text{ Hz}$ ($\approx 780 \text{ nm}$) and $7.8 \cdot 10^{14} \text{ Hz}$ ($\approx 380 \text{ nm}$). We perceive this radiation as light. This visible range varies slightly from person to person, and many animals have different limits. Other frequency ranges serve other purposes (see diagram). Our eyes can even distinguish the frequency of the radiation within the visible range, which we then perceive as different *colors*. We perceive long-wave light (i.e. lower frequency) as red, and short-wave light (i.e. higher frequency) as blue to violet. All rainbow colors lie in between.



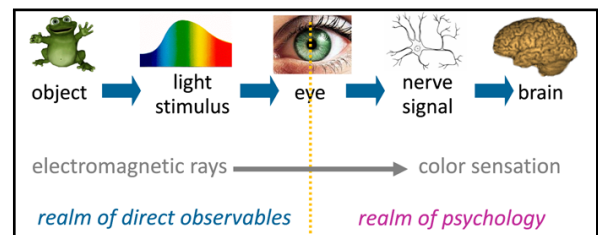
[reminder : $c = \lambda \cdot f$, where $c \dots$ lightspeed, $\lambda \dots$ wavelength, $f \dots$ frequency]

In fact, spectrally pure light (which only has exactly one wavelength) rarely occurs in nature, but we usually see a mixture of many colors (*spectrum*). Frequencies with more energy then determine which color we perceive, one speaks of *dominant wavelength*. If all components are (approximately) the same size, we see a colorless light (i.e. white or grey). If E_D is the energy of the dominant wavelength and E_W is the average energy of the other wavelengths, then $(E_D - E_W)/E_D$ is called the *purity* of a color. The *brightness* results as the area (integral) under the spectral curve.



Colorimetry

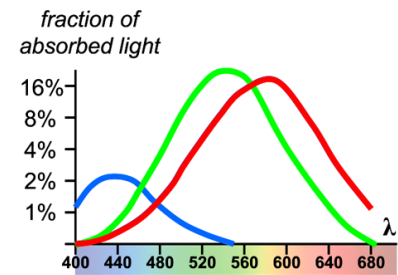
Colorimetry is the science of the technical description of colors. The aim is to describe a color by numbers, by exact specifications. However, since a color is a perceived sensory impression and not a physically directly measurable quantity, only the visual stimulus can be defined numerically (i.e. what a person sees) in such a way that



1. Stimuli with the same specifications look the same under the same conditions,
2. Stimuli that look the same have the same specifications,
3. The numbers used are continuous functions of the physical parameters (i.e. small changes in the numbers cause small changes in the colors and vice versa)

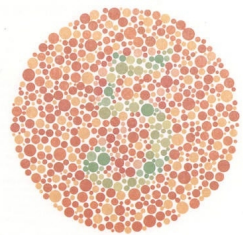
Colorimetry therefore only takes into account the *visual distinguishability* of electromagnetic radiation. All spectra that produce the same color impression are indistinguishable in this sense and form an equivalence class in the color space.

The *retina* of the eye, i.e. the light-sensitive layer at the back of the eyeball, contains around 120 million rods and cones. Rods cannot distinguish colors, but they are very sensitive to light. Cones are much less easily activated, but there are three different types, each of which is sensitive to a different wavelength range (the sensitivity curves are shown in the diagram on the right). Our color perception is therefore made up of the combination of three separate “non-colored” scalar signals, which is why human color perception is called *Tristimulus*. The sensitivity curves of the three cone types have their maxima at red, green, and blue, so it is quite appropriate to speak of red, green, and blue cones. The brain mixes these three values together to form a color. This is also the basis for the fact that the eye can be fooled into seeing “all” colors by composing a color from 3 basic colors. If you place small points of light in red, green, and blue close enough to each other, we perceive this as a point in the color mixed in this way *additive*.



Color Vision Deficiency

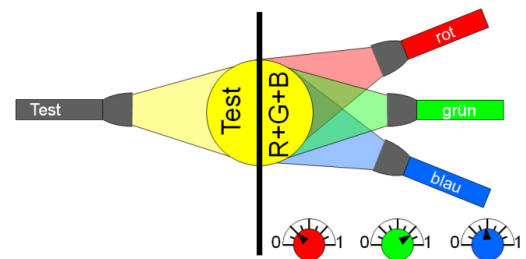
In some people, one type of cone (or even two) is missing due to hereditary factors or the sensitivity curves of the cones are not sufficiently different, in which case the ability to distinguish as many different colors as most people is missing. This is called *color weakness* or *color blindness*. The most common type is red-green blindness, where the red and green cones react to wavelengths that are too similar. About 8% of all men are at least slightly color blind! Test images, in which the information can only be recognized if certain (e.g. reddish and greenish) tones of the same brightness can be distinguished (see image), are used to diagnose color vision deficiency. As you can cope very well in life with reduced color vision, many people are unaware of their impairment.



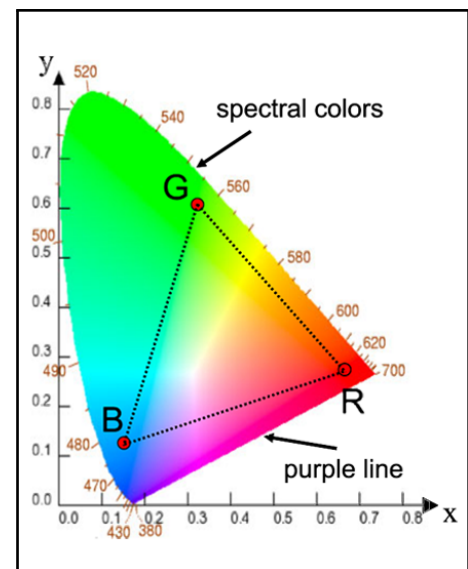
Color Models

CIE 1931 XYZ color model

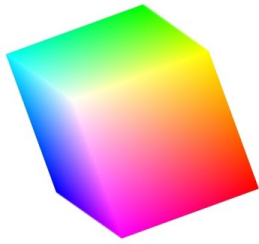
The XYZ color system is obtained directly from the tristimulus theory. To do this, *color comparison experiments* are used to determine which combination of the 3 primary colors produces which test color. For each spectrally pure test color, test subjects must dose red, green and blue light with three controls so that the same color is produced. Some colors cannot be produced from such a combination, which is why it is sometimes necessary to use negative components, which is done by mixing in some light of a basic color on the test color side. After a transformation into positive numbers, we obtain “imaginary primary colors” X, Y, and Z. If the colors generated in this way are standardized to brightness 1 and the result is projected onto the XY plane, we obtain the *CIE diagram*, which was standardized by the Commission Internationale d’Eclairage (CIE) in 1931. The colors are described by the coordinates (x, y) , from $x + y + z = 1$ follows z . In addition, the brightness can be specified, this is denoted by Y . Thus, a complete color definition is given by (x, y, Y) .



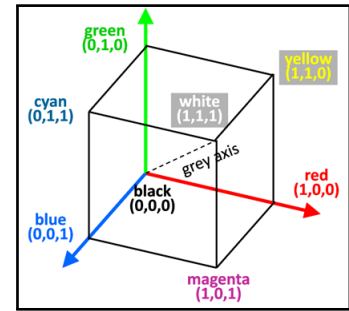
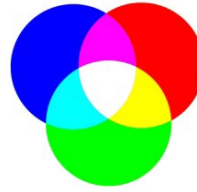
In the CIE 1931 diagram, all spectrally pure colors are located at the U-shaped outer edge. Between the end points of this line runs the so-called purple line, which contains complementary colors of spectrally pure colors, but which themselves do not have only one wavelength. Each point in the diagram corresponds to a different color. A linear combination of two colors is located on the straight line between these two colors. The color white lies approximately in the middle. Complementary colors lie on opposite ends of straight lines that pass through the white point. The colors that can be displayed with an RGB monitor, i.e. the linear combinations of the colors red, green and blue that the monitor can produce, all lie within the triangle spanned by these three points (see sketch). Since there are no three colors that contain the entire diagram, no monitor can display all colors.



RGB color model



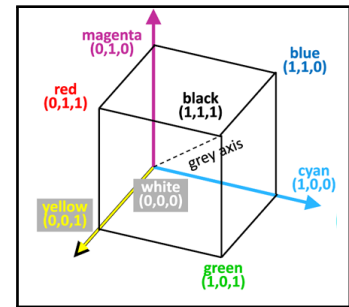
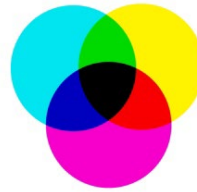
In addition to color spaces (actually color space descriptions) such as the CIE model, which are capable of describing all colors, there are color spaces for describing the colors of a device. For screens, we almost always use the



the RGB model. A pixel is composed of three small color dots, whose light sum (*additive* color mixing! - see sketch with the circles) creates a color impression. Depending on the technology used and the specific materials, each monitor has slightly different basic colors, from which different subsets of all colors can be generated. The space of colors that a device can generate is called its *Gamut*.

CMY color model

The mixing of colored ink on a sheet of paper is subject to completely different rules than the additive color mixing of light. The more ink you use, the darker the result will be because you are actually applying a filter in front of the passively reflecting paper, which is why it is called *subtractive* color mixing (see sketch with the circles). The CMY model is the complement of the RGB space. For simple applications, the following therefore applies



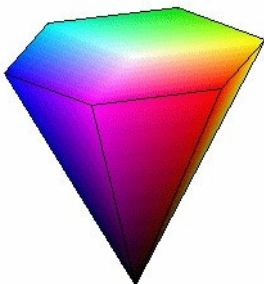
$$[C, M, Y] = [1, 1, 1] - [R, G, B]$$

The *CMYK* model is also often used. K stands for key, which corresponds to the color black. When printing, all gray components are printed separately with black ink instead of producing them as a mixture of equal proportions of cyan, magenta and yellow, which is more expensive and worse.

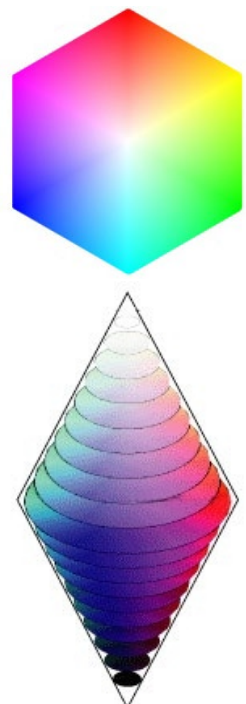
HSV and HLS color models

In addition to the color spaces that make sense for devices, there are also descriptions of the colors in a way that accommodates the *human user*. We can only describe a target color from the components R, G, B, or C, M, Y with great difficulty and a lot of practice. Our usual descriptions of colors are made up of qualities such as a name, brightness, and color purity. Therefore, color systems that function in these three dimensions are used for the user interface to define colors. These include HLS, HSV, Munsell, RAL, NCS, Coloroid, and some others.

HSV stands for **H**ue, **S**aturation and **V**alue. Hue refers to the color along a color wheel that goes from red to orange, yellow, green, cyan, blue, violet, magenta and back to red. If you look at the RGB cube exactly in the direction of its gray axis, you will see this color circle as the boundary of the resulting hexagon (illustration). Saturation means saturation and indicates how pure a color is, i.e. how strongly it differs from grey. Value means value and indicates something like the *brightness* of the color.



The darker a color is, the fewer gradations of saturation there are. As a result, all colors can be represented in a pyramid whose apex is black and whose base is the color hexagon (fig. left). The color is specified in degrees along the base edge (red=0°, green=120°, blue=240°), the saturation as a percentage of the distance from the pyramid axis and the brightness as a percentage of the distance of the base from the tip. A medium-bright saturated yellow thus has the HSV value (60, 1, 0.5).



The **HLS** system (also HSL) works in a very similar way, where H=Hue, L=Lightness or Luminance, S=Saturation. This time, however, the shape of the model is a double cone, which is white at the top and black at the bottom (fig. right). The background is the assumption that white is much brighter than any pure color.

If you now look at the typical color dialogs of a desktop program, you will recognize the use of these user-oriented color systems. Usually you have several color models to choose from, often you can also enter the values by numbers. It is an interesting experience to try to achieve a certain color tone in a certain brightness by specifying the RGB numbers.

Color Symbolism

Colors are companions in everyday life. The use and meaning of colors can diverge between different cultures. Some attributions of meaning apply across cultures, but only within a particular field. The following lines provide some examples.

Language usage Every language has a basic vocabulary for colors. Depending on the language, there are between 2 and 20 basic terms for colors and further terms for nuances. In German, it is assumed that there are 6 to 11 basic terms and around 150 to 200 additional terms (e.g. olive). In other languages this is often different, for example in Italian there are two terms for blue: *azzurro* (sky) and *blu* (dark blue captain's uniform). In Hungarian, there are two reds: *piros* and *vörös*.

Color in religion In spirituality, colors are often attributed symbolic power and cultural or religious content. This is why there is a sacred color in all regions of the world (apart from Christianity!). In Islam, for example, green is the favorite color of the religious founder Mohammed. Many Islamic state flags have the basic color green, such as the Saudi Arabian flag (pictured right).

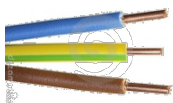


Colors in politics In politics, colors are assigned to both political movements and parties. Colors serve as a uniform distinguishing feature. For example, red is the color of Marxism-Leninism, socialism and the labor movement. Green is often the color of environmental organizations and parties.

Labeling Colors are also used as a stand-alone identifier. The color itself stands for a certain purpose and is often used without further explanation. Examples include red and blue markings on taps.

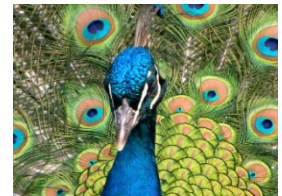


Colors in traffic Colors are deliberately used as information in traffic signs, lights and traffic lights. Prohibition and danger signs are generally red/white, while mandatory and information signs are blue/white. The traffic lights are red/yellow/green. Pass red marker lights on the left and white marker lights on the right.



Colors in technology In technical applications, colors are also often used to describe certain parts. For example, the phase, neutral conductor and earthing of electrical cables can be identified by the color of the wire.

Colors in nature Nature also makes use of color. Many bird species wear colorful mating plumage or have colorful beaks to attract females. In nature, colors also serve as camouflage or as a warning to predators.



Associations with colors Depending on the culture, people associate certain feelings and associations with many colors. *blue*, the color of the sky, stands for the vastness, the distance and for concepts that are easy to associate with it (longing, fantasy). *Red* is the color of blood. However, it not only stands for war and death, but also for vitality (passion, love, anger). *Green* stands for lush meadows, forests and nature. This is why people like to talk about green fingers or city dwellers like to go out into the countryside. Green is also associated with hope and confidence. Summer, sun and joie de vivre are associated with *yellow*. Yellow is the color of light and gold. Yellow is radiant. However, yellow is ambivalently associated with envy, avarice, jealousy, selfishness and mendacity. *Black* is associated with death, endings and emptiness. Black is also the color of mourning,



for people with "white" skin color. In contrast, *white* is seen as a perfect color (many people with dark skin use white to express their sadness and black as a color of joy)."

In order to generate graphical output on a device, the programming language used must provide commands for this. The simple graphic building blocks that can be generated with these commands are also called *graphic primitives*. In addition to simple drawings, these building blocks also include formatting instructions and meta information. The most important such commands are

- In 2D:
 - Points, Lines
 - Polygons, circles, ellipses and other curves (all also filled)
 - Bitmap operations
 - Letters and characters
- In 3D:
 - Triangles and other polygons
 - Free-form surfaces

You also need commands to define the properties of the primitives, e.g. color, fill pattern, texture, material property, and transparency. These commands usually cause all primitives created afterward to take on the properties defined last.

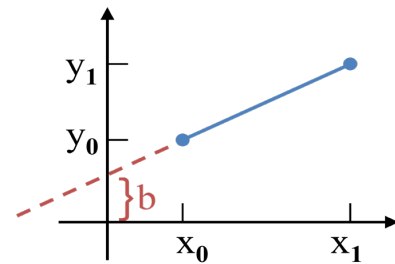
Line algorithms

Drawing straight lines on raster devices is a particularly important operation. The basic DDA (Digital Differential Analyzer) method was cleverly redesigned by Bresenham so that it only requires integer operations, making it *faster* and *easier to implement in hardware*.

Notation: A line is given in the form $y = mx + b$, where m describes the slope of the line and $(0, b)$ is the point of intersection with the y-axis.

Aus den Endpunkten (x_0, y_0) und (x_1, y_1) der Linie lassen sich m und b berechnen:

$$m = (y_1 - y_0) / (x_1 - x_0) \quad b = y_0 - mx_0$$



DDA Procedure

The simple *DDA algorithm* for $|m| < 1$ adds the value m to y_0 for each step to the right ($x += 1$) and then rounds the result to whole numbers. This creates a line in which exactly one pixel is generated for the line for each x value.

```
1 dx = x1 - x0; dy = y1 - y0;
2 m = dy / dx;
3
4 x = x0; y = y0;
5 setPixel (round(x), round(y));
6
7 for (k = 0; k < dx; k++) {
8     x += 1; y += m;
9     setPixel (round(x), round(y))
10 }
```

For $|m| > 1$, x and y are swapped and the process is carried out in a vertical direction. The following Bresenham algorithm is also only shown for $0 < m < 1$, the other directions are obtained by mirroring and rotating by 90° .

Bresenham Algorithm

The *Bresenham algorithm* produces exactly the same result as the simple DDA, but uses only integer arithmetic. It is therefore faster, easier to implement in firmware or hardware, and can also be easily adapted for other curves, e.g. circles, ellipses, spline curves, etc.

For $0 < |m| < 1$, the exact y-value is not calculated for x_{k+1} based on the known position of the pixel in the x_k column, but a decision is merely made as to whether y_k or y_{k+1} is closer to the exact y-value.

From $y = mx + b$ the exact y-value follows for the column to the right of x_k

$$y = m \cdot (x_k + 1) + b$$

The distance to y_k is $d_{\text{lower}} = y - y_k = m(x_k + 1) + b - y_k$ the distance to y_{k+1} is $d_{\text{upper}} = (y_k + 1) - y = y_k + 1 - m(x_k + 1) - b$

If the difference $d_{\text{lower}} - d_{\text{upper}} = 2m \cdot (x_k + 1) - 2y_k + 2b - 1$ is negative, then the lower point (x_{k+1}, y_k) is selected; if it is positive, (x_{k+1}, y_{k+1}) is selected.

If you set $m = \Delta y / \Delta x$ to $(\Delta x = x_1 - x_0, \Delta y = y_1 - y_0)$ and multiply this difference by Δx , you get a decision variable:

$$p_k = \Delta x \cdot (d_{\text{lower}} - d_{\text{upper}}) = 2\Delta y \cdot x_k - 2\Delta x \cdot y_k + c,$$

which has the *same sign* as $d_{\text{lower}} - d_{\text{upper}}$, but does not require division.

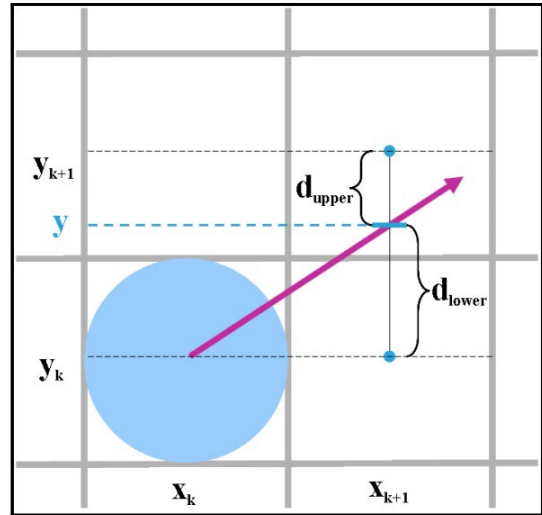
Now you can easily use the decision variable for x_k : $p_k = 2\Delta y \cdot x_k - 2\Delta x \cdot y_k + c$ to calculate the decision variable for x_{k+1} :

$$p_{k+1} = 2\Delta y \cdot x_{k+1} - 2\Delta x \cdot y_{k+1} + c + p_k - 2\Delta y \cdot x_k + 2\Delta x \cdot y_k - c = p_k + 2\Delta y - 2\Delta x \cdot (y_{k+1} - y_k)$$

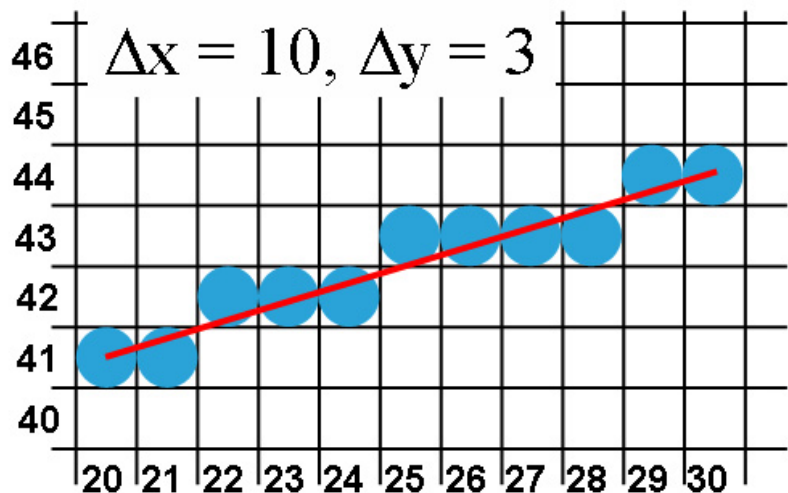
simply by adding a number that remains constant for all points on the line. The initial value is of course $p_0 = 2\Delta y - \Delta x$.

The Bresenham algorithm thus looks something like this:

1. store left line endpoint in (x_0, y_0)
2. plot pixel (x_0, y_0)
3. calculate constants $\Delta x, \Delta y, 2\Delta y, 2\Delta y - 2\Delta x$, and obtain $p_0 = 2\Delta y - \Delta x$
4. At each x_k along the line, perform test:
 - if $p_k < 0$
 - then plot pixel $(x_{k+1}, y_k); p_{k+1} = p_k + 2\Delta y$
 - else plot pixel $(x_{k+1}, y_{k+1}); p_{k+1} = p_k + 2\Delta y - 2\Delta x$
5. perform step 4 $(\Delta x - 1)$ times.



k	p_k	(x_{k+1}, y_{k+1})
		(20,41)
0	-4	(21,41)
1	2	(22,42)
2	-12	(23,42)
3	-6	(24,42)
4	0	(25,43)
5	-14	(26,43)
6	-8	(27,43)
7	-2	(28,44)
8	4	(29,44)
9	-10	(30,44)

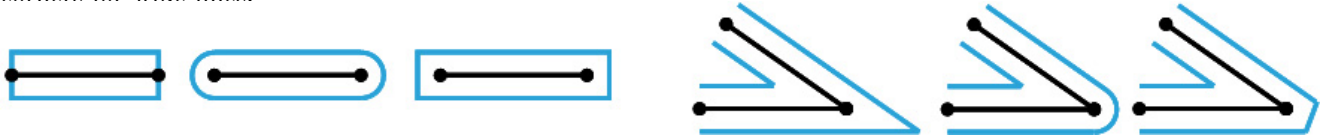


Attributes

Graphic primitives can be created with many different properties, so-called *attributes*.

Attributes of (points and) lines

In addition to generally known properties of lines, such as stroke thickness, stroke pattern, color, or brush type, there are a few attributes that are often less obvious. These include the line ends for wider lines and the shape of corners for wide lines:



Antialiasing is also an issue for lines, details of which are provided below.

Attributes of Text

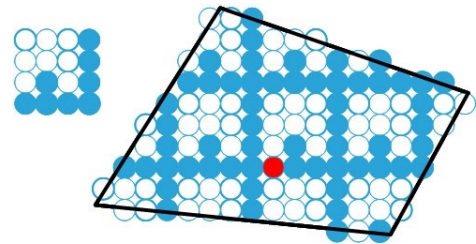
The properties that text can take on are now largely common knowledge: Font (e.g. Courier, **Helvetica**, Times, *Fractal*, ...), style (normal, **bold**, *italic*, underlined,...), size, direction, color, alignment (left, right, center, justified) and so on.

Fonts with serifs (above) are better suited for continuous text, and fonts without serifs for bold text. Fonts are normally represented by defining the outline curves of the letters and and, for some applications also by pixel grids.

Sfzrn
Sfzrn

Attributes of (2D) polygons and surfaces

It is clear that the attributes of the edge of areas are the same as those of lines. In addition, there is now the surface itself, which can be provided with a fill. Patterns are usually created by repetitively stringing together a basic pattern starting from a reference point (also known as a seed point).



In many applications, it is also necessary to create a combination of the newly drawn pattern with the background. There are many variants here, which are often based on logical links: AND, OR, XOR. The mixing of colors is usually done by a linear combination of the existing background color B with the foreground color F to be drawn: $P = t \cdot F + (1 - t) \cdot B$

Barycentric coordinates

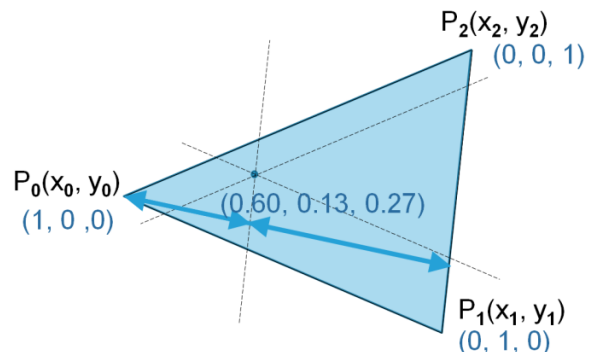
Barycentric coordinates are a basis for the interpolation of pixels in triangles.

Rasterizing triangles

To fill triangles, you often use *barycentric coordinates*. Each point of the plane is represented as the weighted average of the three vertices of the triangle:

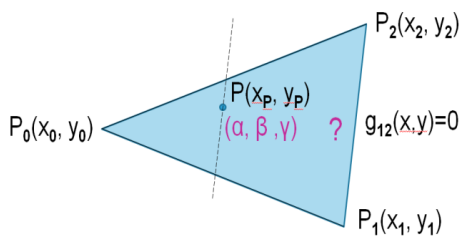
$$P = \alpha P_0 + \beta P_1 + \gamma P_2$$

(α, β, γ) are then called the barycentric coordinates of the point P , whereby the following always applies: $\alpha + \beta + \gamma = 1$. All points with $(0 < \alpha < 1, 0 < \beta < 1, 0 < \gamma < 1)$ lie within the triangle; as soon as one of these values is negative or greater than 1, the point lies outside the triangle.



To fill a triangle, calculate the barycentric coordinates for each pixel of a (preferably narrow) environment and draw all pixels for whose center $(0 < \alpha < 1, 0 < \beta < 1, 0 < \gamma < 1)$ applies. It is very easy to calculate any corner point attributes (e.g. color) in each pixel weighted with (α, β, γ) , this corresponds to a linear interpolation of these values.

calculation of the barycentric coordinates



If $g_{12}(x, y) = a_{12}x + b_{12}y + c_{12} = 0$ is the carrier line through the points P_1 and P_2 , then α of the point $P(x_p, y_p)$ is calculated as

$$\alpha = g_{12}(x_p, y_p) / g_{12}(x_0, y_0).$$

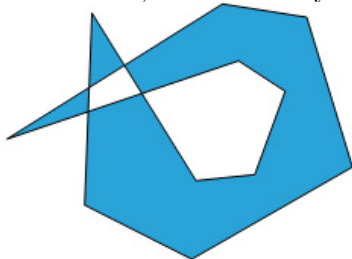
β and γ are calculated analogously.

In order to avoid drawing the edges of adjacent triangles twice, only pixels are drawn whose

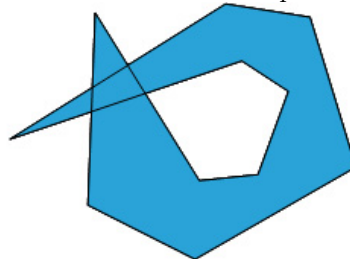
center point must lie within an (exact) triangle. Pixels exactly on an edge must be treated specially, e.g. by rules such as “edges at the bottom and right are rendered, edges at the top and left are not”. This ensures that each edge pixel is only treated once.

What is the inside of a polygon

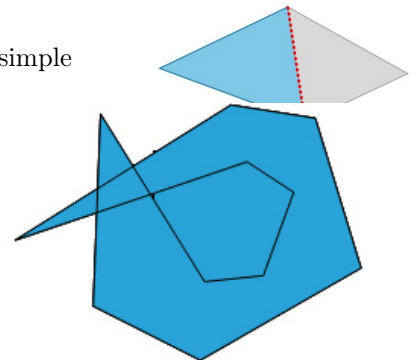
Before you start filling surfaces, you have to ask yourself what is to be filled. For a simple closed curve, “inside” is easy to define, but what about more complicated curves?



like this?

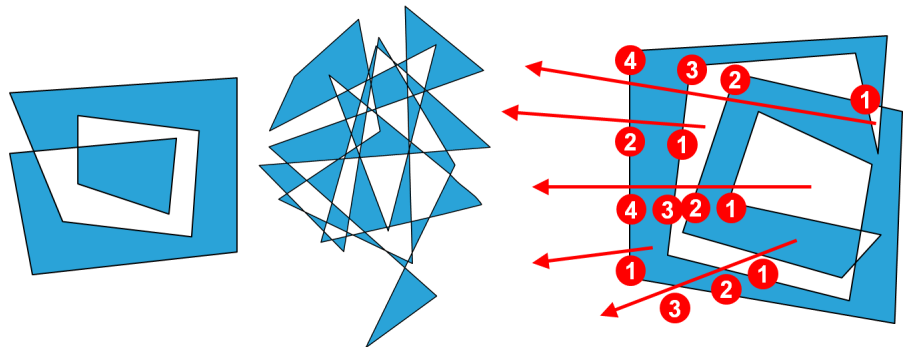


or this?

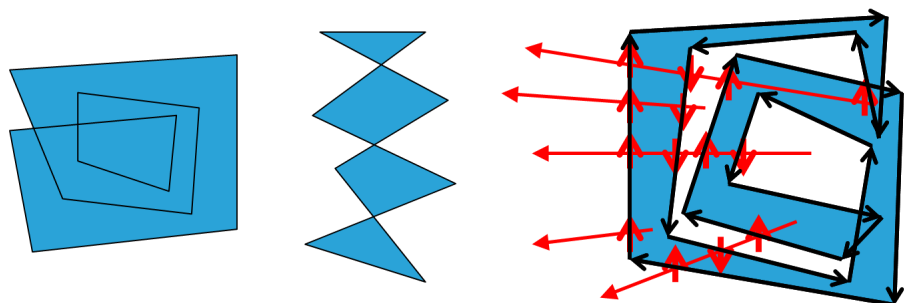


or this?

Odd-Even-Rule If you draw any half ray from a point, the point is inside if the number of intersections with the curve is odd, otherwise, the point is outside (top left in fig. and all images on the right). Each edge therefore has one side inside and the other outside.

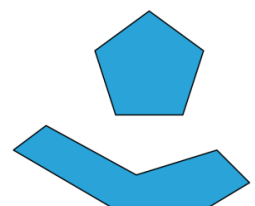


Nonzero-Winding-Number-Rule Points are outside if there are the same number of clockwise and counterclockwise curve edges on any half-beam, otherwise, they are inside (in the top center of the figure, as well as all images on the right).



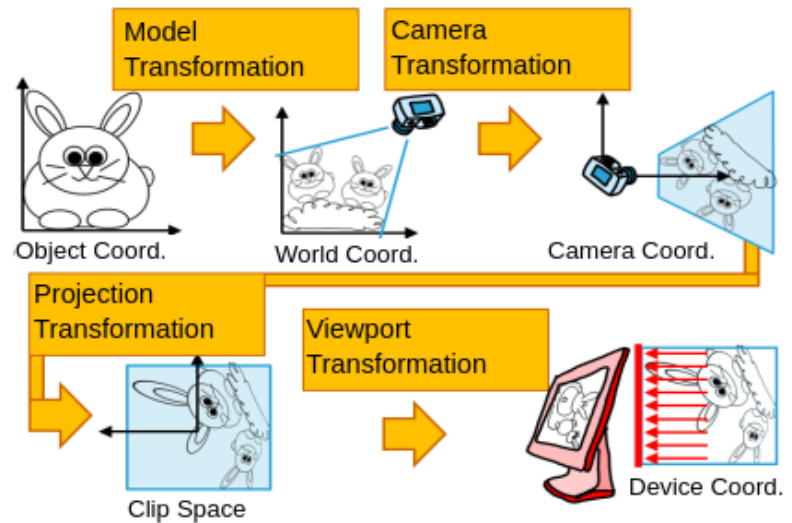
All-In-Rule Everything that is somehow enclosed is inside. Rarely used, mostly in poker ☺ (top right in illus.).

A polygon is called *convex* if all internal angles are less than 180° (top image), otherwise *concave* (bottom image). Since convex polygons generate far fewer special cases, many algorithms are designed for convex polygons (often even only for triangles). Therefore, methods are also needed to divide concave polygons into several convex polygons (often into triangles).



Viewing in the graphics pipeline

In principle, the scene to be displayed is modeled in world coordinates, whereby individual parts of their local coordinate systems are converted into world coordinates using geometric transformations (matrices!). After defining the camera parameters, these coordinates are converted into camera coordinates to which the projection is then applied. The result is stored in a normalized cube (often with a side length of 2), from where it is transformed into the device coordinates of the output medium used by the so-called viewport transformation.

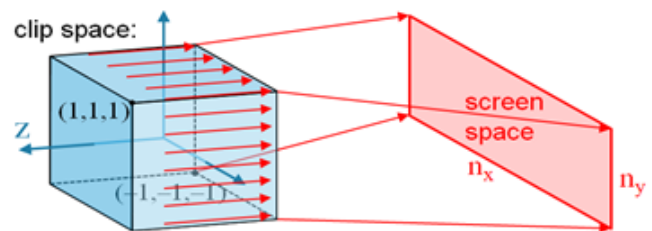


Several different projections are known in geometry, of which only the parallel projection and the perspective are of major importance in computer graphics. We will now first assume that the camera performs a parallel projection and then consider how the perspective projection can also be integrated into the pipeline.

We'll start from the back because it's easier to think about this way.

Viewport transformation

We therefore assume that the scene already exists in clip space, i.e. all relevant coordinates are located in an axis-parallel cube with side length 2 and center (0,0,0). We want to perform an orthographic mapping (parallel normal projection) with viewing direction -z onto a screen with dimensions $n_x \times n_y$ (pixels). All points $(-1, -1, z)$ must therefore be mapped to $(0, 0)$ and all points $(1, 1, z)$ to $(n_x \times n_y)$. This linear mapping is achieved by the matrix Mvp:



Their correctness can be proven immediately by inserting the corner points. However, the matrix has a special feature displayed in red: the z-values are retained! This

is not important at the moment but will be of great value in later steps (especially when calculating the visibility).

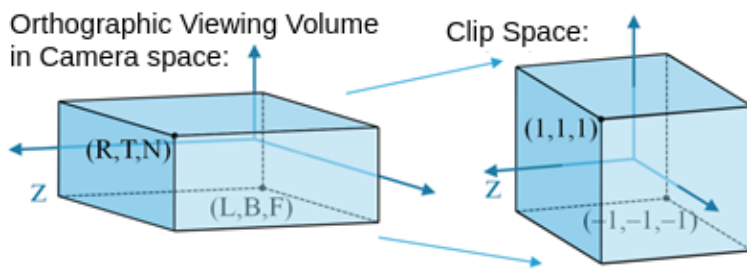
$$\begin{bmatrix} x_{screen} \\ y_{screen} \\ \textcolor{red}{z} \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{n_x}{2} & 0 & 0 & \frac{n_x}{2} \\ 0 & \frac{n_y}{2} & 0 & \frac{n_y}{2} \\ \textcolor{red}{0} & \textcolor{red}{0} & \textcolor{red}{1} & \textcolor{red}{0} \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ \textcolor{red}{z} \\ 1 \end{bmatrix}$$

Projection transformation

As already mentioned above, we assume for the moment that an orthographic projection is to be carried out. This simplifies the transformation enormously because now we only have to move and distort an axis-parallel cuboid with the boundaries L(ef), R(ight), B(ottom),

T(op), N(ear), F(ar) so that the cube $[-1, 1]^3$ is created (see figure). (L,B,F) must therefore become $(-1, -1, -1)$ and (R,T,N) must become $(1, 1, 1)$. Again, this can be easily expressed with a transformation matrix:

Orthographic Viewing Volume in Camera space:



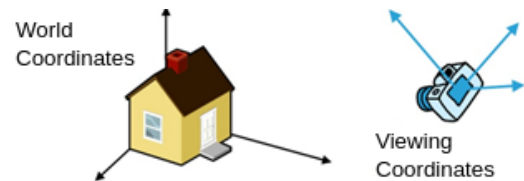
$$M_{orth} = \begin{bmatrix} \frac{2}{R-L} & 0 & 0 & -\frac{R+L}{R-L} \\ 0 & \frac{2}{T-B} & 0 & -\frac{T+B}{T-B} \\ 0 & 0 & \frac{2}{N-F} & -\frac{N+F}{N-F} \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Note: A parallel projection can also be made at an angle to an image plane (for example when casting a shadow); this variant is not considered here.

Camera transformation

As with photography, you have several degrees of freedom when setting the camera values:

1. Position of the camera in space
2. Viewing direction from this position
3. Orientation of the camera (where is the top?)
4. Size of the image section (corresponds to the focal length or zoom factor of a camera)



The camera coordinate system u, v, w (viewing coordinates) is calculated from the first three values. Normally, the uv plane of this viewing coordinate system is normal to the main viewing direction and you look in the direction of the negative w axis.

Starting from the camera position, the viewing coordinate system is defined as follows:

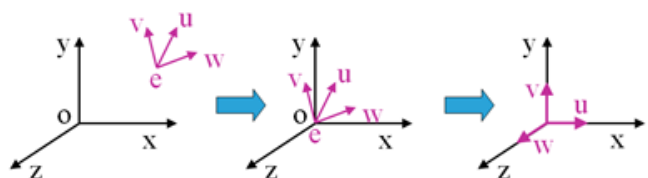
1. Selection of a camera position (also called eye point or viewing point).
2. Selection of a viewing direction, the negative viewing direction results in the w -axis.
3. Selection of a direction "upwards"; the u and v axes can then be calculated from this.
4. Since the mapping plane is normal to the viewing direction, the vector product $t \times w$ gives the direction of the u -axis.
5. Calculation of the v -axis as the vector product of the w - and u -axes: $v = w \times u$.

In animations, the camera definition is often calculated automatically from certain conditions, e.g. when the camera moves around an object or in a flight simulation, so that the desired effects can be achieved easily.

To convert the world coordinates into viewing coordinates, you need a chain of simple transformations: for example, a translation of the coordinate origins on top of each other and then 3 rotations so that the coordinate axes are also on top of each other (two rotations for the first axis, one for the second axis, and the third is then automatically correct).

These transformations can of course be summarized again by multiplication to form a matrix, which looks something like this: $M_{WC,VC} = R_z * R_y * R_x * T$

In order to fully describe the subsequent projection, the limits of the area to be displayed are required. The complete definition of the camera therefore also includes



6. The selection of minimum and maximum u , v , and w values to limit the section of the scene that is displayed: L(ef), R(ight), B(ottom), T(op), N(ear), F(ar).

In the parallel projection, L, R, B, T refer to the respective values of the boundary planes ($u=L, u=R, v=B, v=T$), in the perspective projection to the image boundaries in the near plane.

Orthographic viewing

For the orthographic projection (camera images in parallel), we have now described all the steps using matrices. As with the geometric transformations, we can combine (multiply) these into a single matrix, which then performs the entire viewing transformation:

$$\begin{bmatrix} x_{screen} \\ y_{screen} \\ z \\ 1 \end{bmatrix} = (M_{vp} * M_{orth} * M_{cam}) * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Note: the rightmost matrix is first multiplied by the point (x,y,z). If you apply the associative law, you can also multiply the 3 matrices with each other first, and can thus transform all points directly from world coordinates to device coordinates using only this one result matrix!

Perspective

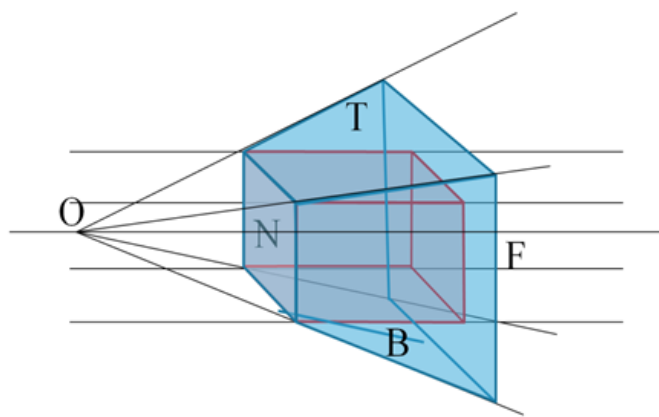
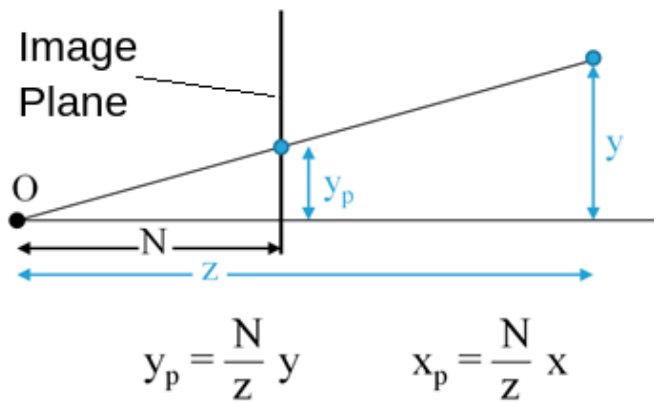
In perspective, several of the laws for affine transformations no longer apply (e.g. parallel lines are generally no longer parallel after a perspective projection), so it is not an affine transformation and cannot be generated by a 3x3 matrix. Fortunately, the homogeneous coordinates help here again, but this is the only case in which the homogeneous component h does not retain the value 1, and therefore a subsequent division step by this value is necessary.

First, let's consider a formula for the perspective transformation: We denote the projection center of the image by O, and the viewing direction is the negative z-direction. Then the mapping plane is normal to the z-axis at a distance N(ear). If a point (x,y,z) is mapped onto this plane, it has the coordinates (x*N/z, y*N/z, N). This can be achieved using a matrix P:

$$P = \begin{bmatrix} N & 0 & 0 & 0 \\ 0 & N & 0 & 0 \\ 0 & 1 & N+F & -F*N \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

If you multiply a point (x,y,z,1) by P, you first get (x*N, y*N, z*(N+F) - F*N, z). By homogenizing (also normalizing = dividing by the last component, i.e. z) we obtain

$$(x * N/z, y * N/z, (N+F) - F * N/z, 1)$$



This operation corresponds to a distortion of the scene area to be depicted (this truncated pyramid is called the "view frustum") into an axis-parallel cuboid in which the orthographic projection provides exactly the same image as the perspective projection in the view frustum. We can then apply the parallel projection we have already worked out and use it to calculate the perspective matrix M_{per}. Alternatively, you can simply insert P at the correct point in the overall view calculation and thus create an overall matrix that performs everything from model coordinates (x,y,z) to device coordinates (pixel positions (xscreen, yscreen)) in one step:

$$\begin{bmatrix} x_{screen} \\ y_{screen} \\ z \\ 1 \end{bmatrix} = M_{vp} * \dagger \left(\overbrace{M_{orth} * P}^{M_{per}} * M_{cam} * M_{mod} \right) * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

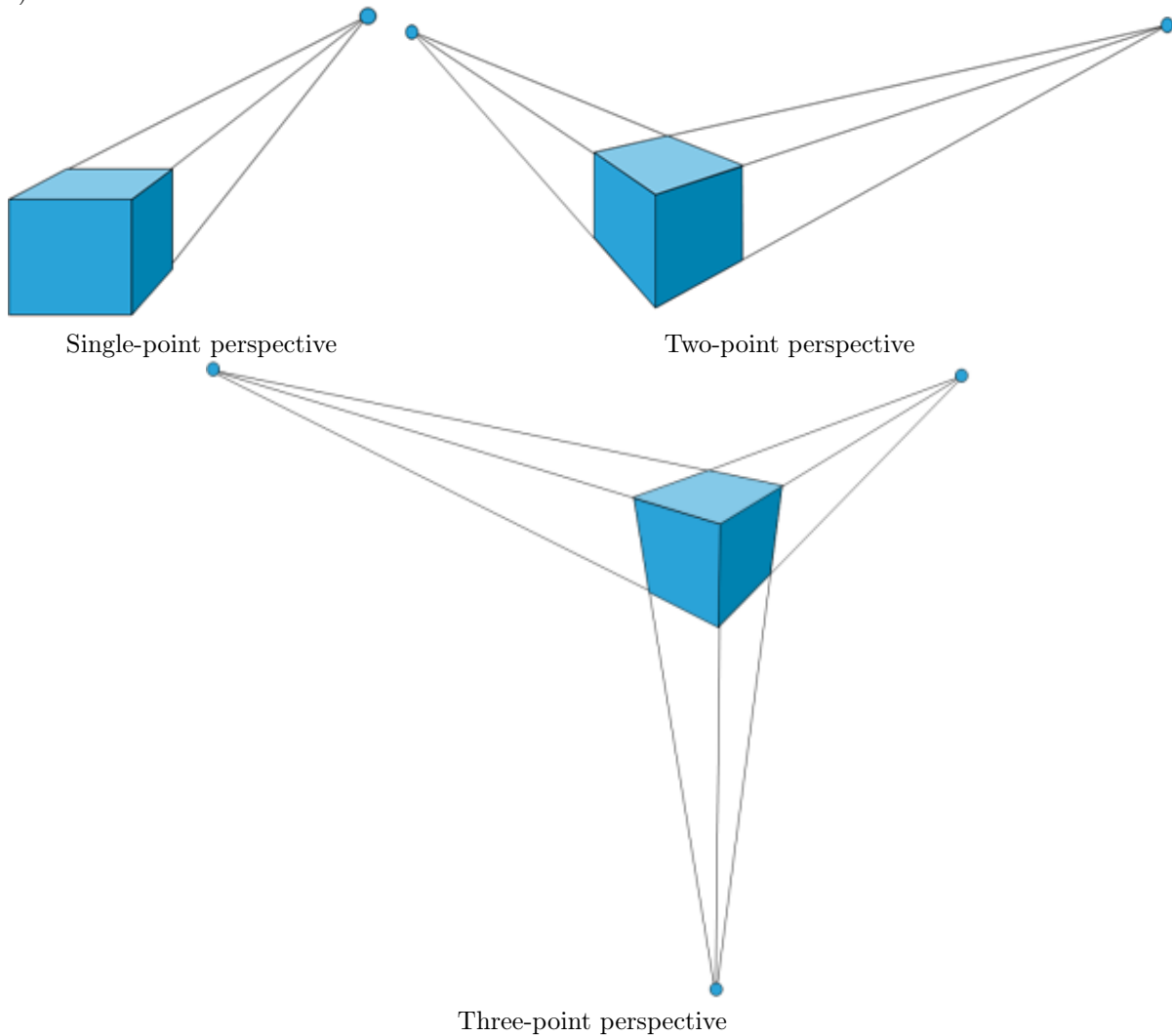
If a perspective mapping is involved, the result must be divided by the homogeneous component (in this case z') at the end. In practice, at the point marked with †, i.e. in the clip space, not only the clipping is carried out, but also the homogenization. The viewport matrix is then applied separately as the last step.

Other important properties of the projection transformation are

1. Straight lines remain straight lines. To map such a line (e.g. the side of a polygon), it is sufficient to transform the two endpoints.
2. the relative order of the z-values (i.e. distance from the camera) is retained (but not the distance values themselves), which is important for the visibility calculation:

$$\begin{aligned}
 & z_1, z_2, N, F < 0 \\
 & z_1 < z_2 \\
 & \frac{1}{z_1} > \frac{1}{z_2} & | * (-F * N) (< 0) \\
 & -F * \frac{N}{z_1} < -F * \frac{N}{z_2} & | + (N + F) \\
 & (N + F) - F * \frac{N}{z_1} < (N + F) - F * \frac{N}{z_2}
 \end{aligned}$$

Finally, it should be mentioned that the number of main alignment points depends on the position of the image plane in relation to the coordinate system. If two axes are parallel to the image plane, it is called one-point perspective, if only one axis is parallel to the image plane, it is called two-point perspective, and if all three axes are not parallel to the image plane, it is called three-point perspective (because there are then three main vanishing points).



Line Clipping

Clipping is the cutting off of parts of the image that lie outside the display window (see also illustration on the right). The earlier the clipping operation is carried out in the viewing pipeline, the more unnecessary subsequent reshaping of parts that are not visible anyway can be avoided:

- **Clipping in world coordinates** = analytical calculation at the earliest possible point in time,
- **Clipping in clip coordinates** = analytical calculation at axis-parallel boundaries (simple!),
- **Clipping in raster conversion** = within the algorithm that converts a graphic primitive into points.

Clipping is a very common operation, so it must be simple and fast.

Clipping of Lines: Cohen-Sutherland Method

Algorithms for clipping lines generally exploit the fact that each line in a rectangular window has at most one visible part. Furthermore, basic principles of efficiency must be utilized, such as eliminating frequent simple cases at an early stage and avoiding unnecessary expensive operations (intersection calculations). The simplest line clipping could look something like this:

```

1 for endpoints (x0,y0), (xend,yend)
2 intersect parametric representation
3   x = x0 + u * (xend - x0)
4   y = y0 + u * (yend - y0)
5 with window borders:
6   intersection ⇔ 0 < u < 1
    
```

The Cohen-Sutherland algorithm first classifies the endpoints of a line according to their position in relation to the clipping window: top, bottom, left, right, and encodes this information in 4 bits. Now you can quickly check:

1. OR of the two codes = 0000 → Line completely visible
2. AND of the two codes ≠ 0000 → line completely invisible
3. otherwise intersect with a relevant window edge and replace the cut point with the intersection point. GOTO 1.

Intersection point calculations:

with vertical window edges:

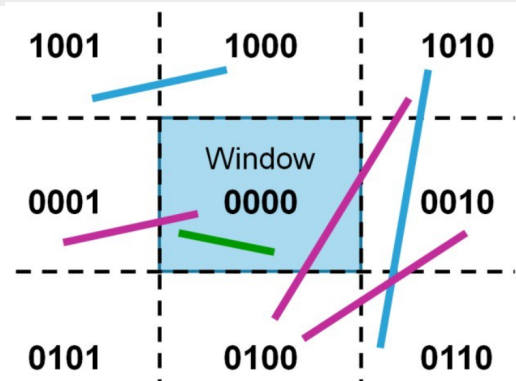
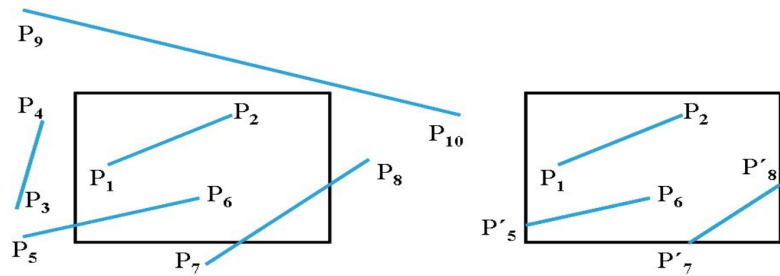
$$y = y0 + m(xw_{min} - x0), y = y0 + m(xw_{max} - x0)$$

with horizontal window edges:

$$x = x0 + (yw_{min} - y0)/m, x = x0 + (yw_{max} - y0)/m$$

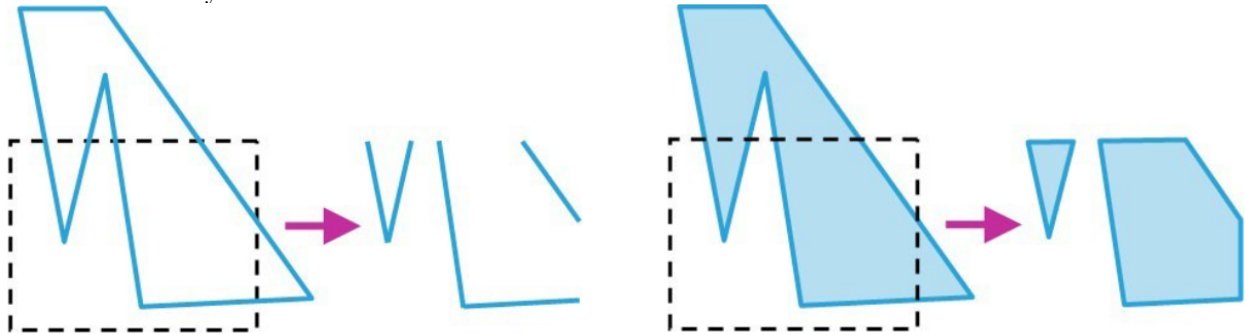
Points exactly on the edges of the window must of course be considered to be inside, in which case a maximum of 4 loop passes can be made. As you can also see, intersection calculations are only carried out when it is really necessary.

There are similar procedures for clipping circles. It must of course be taken into account that circles can break up into several parts when clipping.



Polygon Clipping

The clipping of polygons must take into account that a polygon is created again after clipping, even if several parts are created by the clipping process. The illustration on the left shows a polygon that has been clipped using a line-clipping algorithm. It is no longer possible to tell what is inside and what is outside. The image on the right shows the result of a correct polygon clipping process. The polygon breaks down into several parts, all of which can be filled correctly.



Clipping of Triangles

sec:triangle:clip

In practice, geometric data very often consists only of triangles, and the rendering process no longer has any knowledge of their relationship. This is referred to as "triangle soup". In this case, it is important that there are no other primitives apart from triangles. Therefore, triangles must always be created when clipping. When clipping a triangle against an edge, a quadrilateral can also be created in one of four possible cases (see figure on the right). This must be immediately divided into two triangles to ensure further processing. It can happen that more triangles are created at the corners of the clip window than would be necessary (e.g. Fig. left), but this is more than compensated for by the simplicity of the algorithm.

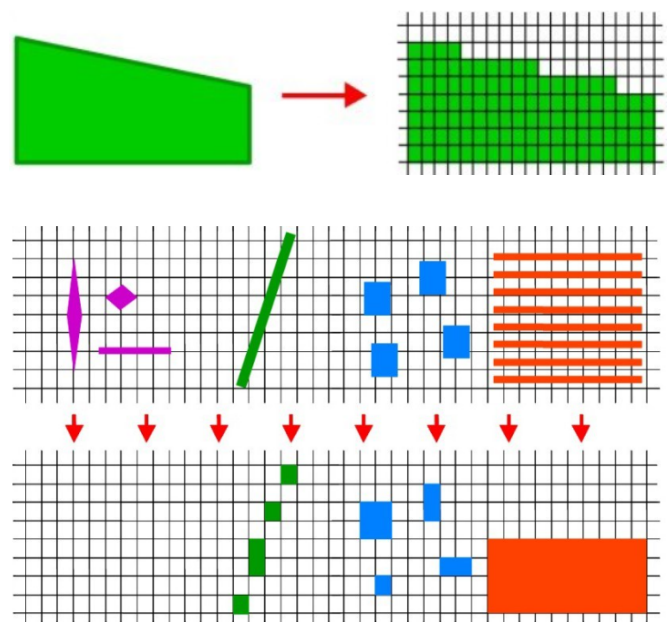
Clipping in Clip Coordinates

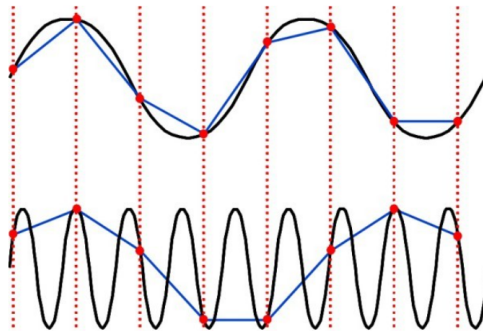
sec:clip:clip In the clip space, the boundary surfaces of the view frustum (i.e. the area to be clipped) are all axis-parallel ($x = \pm 1, y = \pm 1, z = \pm 1$), which reduces the determination of whether a point lies inside or outside such a surface to a single comparison of two numbers. By performing this step before homogenizing the point coordinates by clipping at the planes $x = \pm h, y = \pm h, z = \pm h$ (which is just as easy), you ensure that points that lie behind the camera point are not projected (that would be completely wrong!), and it also saves the homogenization division for all points that were outside the clip area.

Aliasing and Antialiasing

Aliasing effects ['eiliæsiŋ] are errors that occur during the conversion (discretization) of analog to digital information. Among other things, aliasing refers to all imperfections in raster images that occur because a pixel can only have one value but actually represents a small area. Visible aliasing effects have the following causes, for example: too low resolution, too few available colors, too few images/sec, geometric errors, and numerical errors.

Antialiasing is the name given to methods to reduce unwanted aliasing artifacts. As improving the hardware is usually unrealistic, software methods are mainly used. In the following, only anti-aliasing for dealing with the resolution problem will be discussed. Some known effects besides the staircase effect are the disappearance of small objects, interrupted narrow objects, different sizes of the same objects, complete destruction of fine textures (see figure).

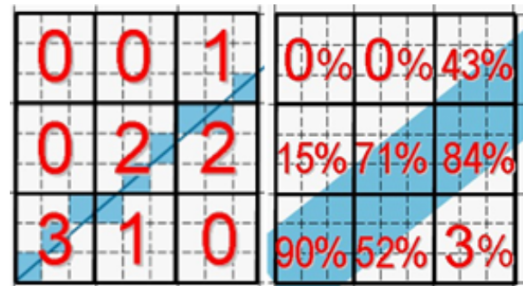




Antialiasing of lines

sec:alias:lines The cause of aliasing is an insufficiently fine sampling of the true image. The theoretical basis for this is the *Nyquist-Shannon sampling theorem*. According to this theorem, information can only be correctly reconstructed if a sampling rate is used that is greater than twice as high as the highest information frequency to be transmitted. This limit is called *Nyquist-Limit*. The figure shows how a too-coarse sampling rate of a signal (curve) can lead to a completely incorrect reconstruction (polygon course). Such errors can be reduced either by pre-filtering the signal or by post-processing the finished image. However, post-processing is always inferior to pre-filtering. The central strategy for pre-filtering is *Supersampling* (also *Oversampling*).

Pixels that are crossed further by a line should be given more line color than pixels that are only lightly touched by a line. To do this, divide each pixel into sub-pixels, count how many sub-pixels lie on the line, and select an intensity that is proportional to this number. For wider lines, you calculate the percentage of overlap of the pixel by the line and then select the intensity of the line color. Based on the insight that the center of a pixel is more important than its edge, the subpixels in the center are sometimes weighted more heavily than those at the edge ("weighted oversampling").



Antialiasing of Polygon Edges

sec:alias:edge

There are the same alternatives for polygon edges as for lines: either you work with supersampling or you calculate the degree of coverage of the pixel by the polygon analytically. The degree of coverage is calculated at the same time as the raster conversion, i.e. the creation of the edge and the filling of a polygon. When calculating the endpoints of the spans in the scan line filling method, you have enough information available that the degree of coverage drops off almost free of charge.



We remember the decision variable p_k in the Bresenham line algorithm, the sign of which indicated which pixel was to be drawn next. This variable can be transformed so that its value corresponds to the degree of coverage of the last pixel. $p' = y - y_{mid}$, where $y_{mid} = (y_k + y_{k+1})/2$, has the same sign property as p_k . If $p = p' + (1 - m)$ is used, the comparison is not made with 0 but with $(1 - m)$, but $0 \leq p \leq 1$ applies, and p corresponds to the degree of coverage at the point x_k . In this way, the anti-aliasing can be calculated incrementally very quickly. For other angles, you work with rotations of 90° and/or reflections of this method.



Sampling and Reconstruction in Spatial and Frequency Domains

A sound understanding of sampling and reconstruction requires the consideration of a signal in spatial and frequency space. The Fourier transform describes a signal in spatial space (e.g. a scan line in an image) as the sum of sinusoidal oscillations in frequency space. A sinusoidal oscillation is described by frequency, phase, and amplitude. In frequency space, a signal is specified by a spectrum, with phase and amplitude depending on the frequencies ω . With the help of the Euler identity $e^{ix} = \cos x + i \sin x$, phase and amplitude can be efficiently described by an imaginary number. The inverse Fourier transform converts a spectrum in frequency space into a signal in spatial space. A convolution combines two functions, resulting in the integrally weighted sum product of the two functions:

$$f_1 * f_2(x) = \int_{\mathbb{R}} f_1(\tau) f_2(x - \tau) d\tau$$

The convolution theorem states that the multiplication of two functions in space corresponds to a convolution (*) of the corresponding spectra in frequency space ($f_1 f_2 = F_1 * F_2$) and vice versa ($f_1 * f_2 = F_1 F_2$). Sampling (i.e. discretization) in space is now a multiplication of the signal with a comb function $comb_T$. In frequency space, this corresponds to a convolution with the corresponding comb function $comb_{1/T}$. The tooth spacing in $comb_T$ and $comb_{1/T}$ are inversely proportional to each other. Convolution with the comb function results in a replication of the spectrum in frequency space (shadow spectra). If the sampling frequency is too low (below or at the Nyquist limit, T too large), the combs of $comb_T$ are too far apart in the spatial domain and $comb_{1/T}$ are too close together in the frequency domain. After sampling, the shadow spectra overlap, which causes aliasing. An error-free reconstruction is no longer possible. In the exact reconstruction, the shadow spectra created by the discretization are removed again. In frequency space, the spectrum of the discretized function is multiplied by a rectangular function, leaving only the original spectrum. This corresponds to the convolution with the $Sinc(x) = \sin(x)/x$ function in local space. Since the sinc function is different from zero over an infinite range, the rectangular function (nearest-neighbor interpolation) or triangular function (linear interpolation) is used as a practicable reconstruction method.

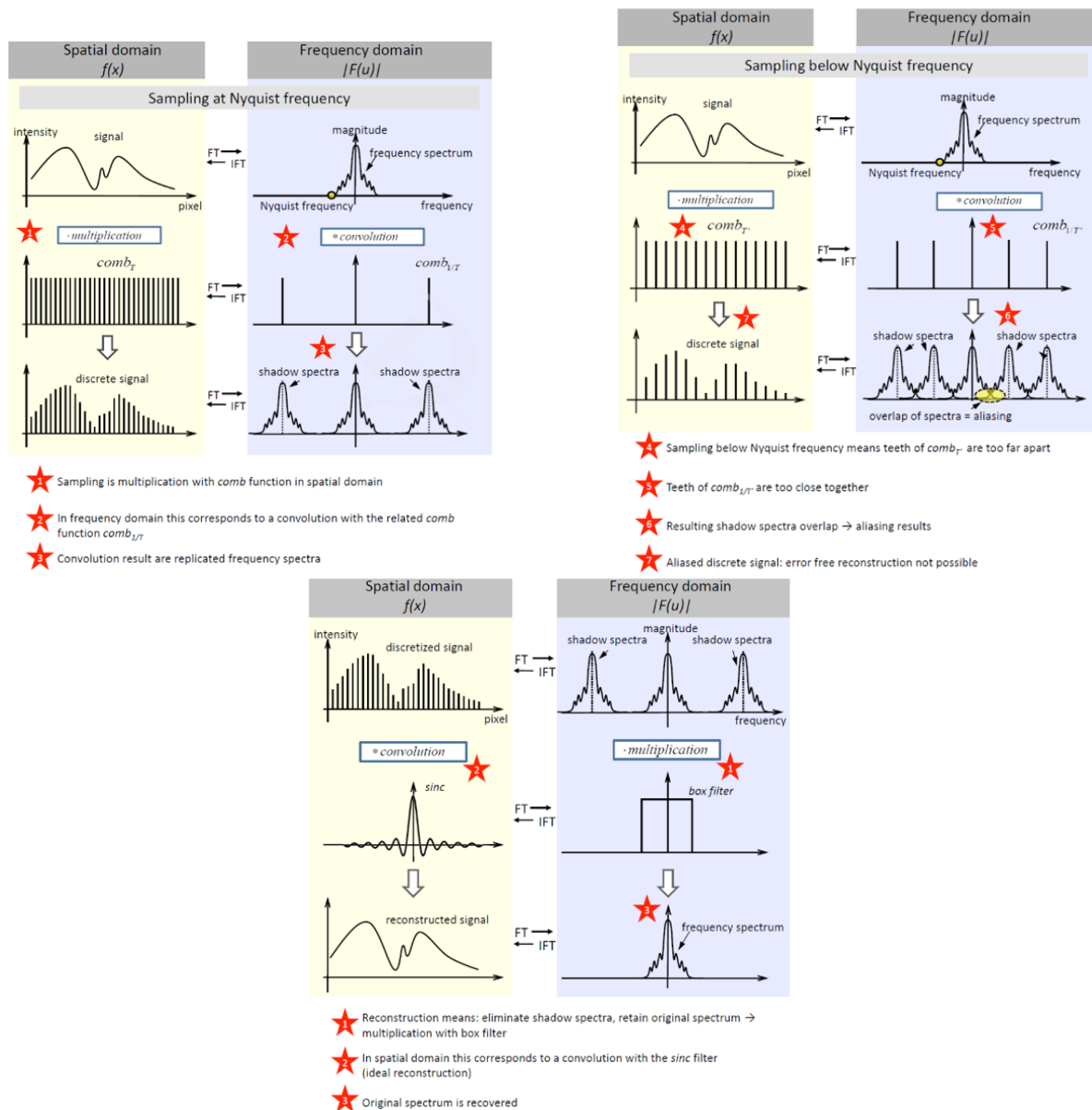


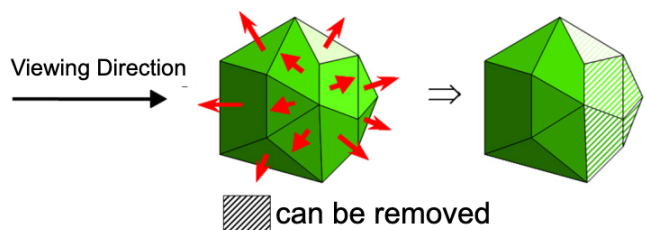
Abbildung 10: Note: In the top left image, it would be more accurate to say 'Sampling at greater than Nyquist frequency,' since sampling at exactly the Nyquist frequency can still lead to errors (e.g., sampling a sine function at exactly the Nyquist frequency can result in a constant line if the zero crossings are always sampled).

In order to depict scenes credibly and correctly, all parts that are not visible from the viewing direction must be omitted. These are in particular the backs of objects and parts of objects that are covered by other objects. This is referred to as hidden line or hidden surface elimination.

Depending on the scene complexity, the object types and data structures, the available hardware and application requirements, different visibility algorithms are used. With the object space methods, the position of the objects is compared with each other and only the front (visible) parts are drawn; with the image space methods, the visible parts are calculated separately for each part of the image. The following explanations do not take transparent objects into account.

Backface Detection (Backface Culling)

Backface Detection (Backface Culling) Backface culling is not a complete visibility determination algorithm. It merely eliminates all polygons whose surface normal points away from the viewer and which therefore definitely cannot be visible, in order to reduce the effort required for subsequent work steps. This removes on average 50% of the polygons. With orthographic projection, this is calculated using the scalar product of the viewing direction vector with the surface normal ($V_{view} \cdot N > 0 \rightarrow invisible$) or in the case of perspective projection by inserting the viewpoint (x, y, z) into the plane equation ($Ax + By + Cz + D < 0 \rightarrow invisible$). [Assumptions as for "polygon lists"].



Z-Buffer-Method (Depth Buffer)

The z-buffer algorithm solves the visibility problem for a specific image resolution as follows: For each pixel, the position of the displayed object is remembered in separate memory in addition to its color information. Since the z direction is normally used as the viewing direction, the x and y values of this position correspond to those of the image plane and only the z value needs to be stored. Therefore, in addition to the frame buffer, additional memory is required to store a coordinate value (z-value) for each pixel; this memory is called the z-buffer or depth buffer. All objects can now be drawn in any order. The z-values of the next object to be drawn (usually a polygon) are calculated and compared with the z-values of the pixels into which the object is to be drawn. If the new z-value is closer to the viewer (i.e. normally larger), the object is drawn at this point over the old image value and the z-value in the z-buffer is also replaced. Otherwise, the new object is hidden and is not drawn at this point:

```

7  for all (x,y)                                Initializing the background
8      depthBuff(x,y) = - 1                      greatest possible distance
9      frameBuff(x,y) = backgroundColor
10 for each polygon P                            Loop over all polygons
11     for each position (x,y) on polygon P
12         calculate depth z
13         if z > depthBuff(x,y) then
14             depthBuff(x,y) = z
15             frameBuff(x,y) = surfColor(x,y)    else nothing !

```

For flat polygons, the z-values can of course be calculated incrementally more efficiently. The great advantage of the z-buffer method is that the objects (polygons) do not need to be sorted.

(Note: In the viewport transformation, z is often multiplied by -1 before the z-buffer method is applied).

Scanline Method

With the scanline method, the correct visibility is calculated line by line (in the example from top to bottom, i.e. y decreasing). This takes advantage of the fact that two rows of pixels (scanlines) lying on top of each other often differ only insignificantly in their visibility behavior.

Depth Sorting Method

The basic principle of the depth sorting method is to sort all polygons from back to front and then draw them in this order. As all hidden parts are further back than the parts that hide them, an image with correct visibility remains at the end (painter's algorithm). Here, the main effort lies in sorting, which must be done in such a way that no polygon (partially) obscures another polygon that comes later in the list (i.e. is further in front"). To

do this, a rough sort is first carried out quickly and then checked to see if everything is correct and re-sorted if necessary.

Area Subdivision Method

Similar to the quadtree representation of images, simple cases are solved at a coarser resolution and more complicated cases are simplified by dividing the area into four quarters. Recursive application up to the maximum image resolution guarantees a pixel-precise solution to the visibility problem.

Octree Method

If the scene is represented as an octree instead of polygons, the data structure already knows what is in front and what is behind for each viewing direction. Recursively, you can always render the most distant subcube in each cube first, then the 3 closest ones, then the next three, and finally the foremost one. A possible sequence for a frontal viewing direction can be seen in the following example:

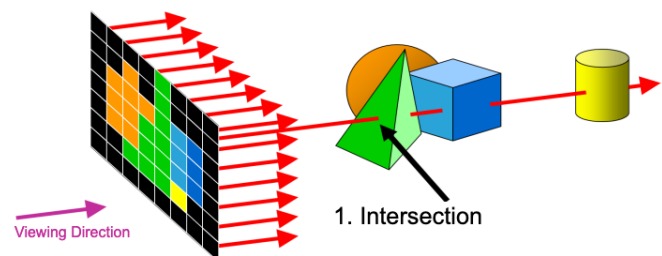


Alternatively, you can also draw from front to back. Then you have to remember all the areas in which something has already been drawn and only draw things that actually remain visible. The advantage over other data structures remains the implicit knowledge of what is further forward or further back.

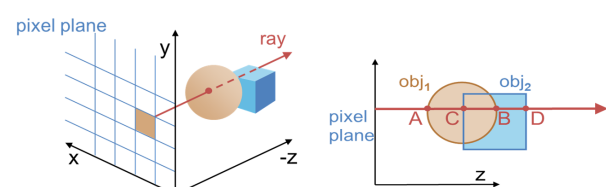
Ray Casting

Ray casting is a visibility algorithm that calculates what is visible for each pixel separately. To do so, a ray is cast from a particular pixel in the direction of view, i.e. a straight line through the pixel into the scene. In the opposite direction, the light from the scene travels along this line to the image plane, i.e. to the viewer. If you intersect this ray of light with all the objects/polygons in the scene, you will obtain a number of intersection points from which you can select the one closest to the viewer. The color of the surface at this point determines the color of the pixel through which you have placed the ray. If you do this for all pixels, you get the color of the foremost object, i.e. the visible object, for each point.

In addition to polygons, ray-casting can also be used to easily render all other surfaces (e.g. free-form surfaces) for which the intersection with a straight line can be calculated. As we will see later, you normally also need the surface normal at the point of impact in order to obtain usable shading. However, this is very time-consuming, since for each pixel (that's several million for a screen) you have to perform an intersection with each object (and that can be thousands to millions of objects). Efficient implementation of the intersection test and further optimizations are therefore necessary.



Ray Casting of CSG Objects The most common method for mapping CSG objects is ray casting, in which the image is calculated pixel by pixel. For each pixel, a ray is cast in the viewing direction and intersected with all objects in the scene. The foremost of these intersection points indicates which object can be seen in this pixel, and the pixel receives its color. In a CSG tree, this calculation is performed recursively:



- for end nodes, the calculation of all intersection points is simple,
- for intermediate nodes, the intersection point lists of the two successors are linked according to the operator:
 - for union the list (A,D)
 - for average the list (C,B)
 - for difference the list (A,C)
- for the tree root, the first point of the linked intersection list is selected

Ray casting is a simplified version of ray tracing that can be used to simulate many more optical effects. This will be discussed in a subsequent chapter.

Ray-Casting = for each pixel of the display area:

- create a straight line through a pixel in the viewing direction ("ray of vision")
- intersect the line of sight with all objects
- select the point closest to the viewer from the intersection point list
- color the pixel with the color of the surface of this point

Classification of Visibility Algorithms

Which methods work in object space and which in image space? This cannot always be clearly classified, but by and large the following applies:

Object space processes Backface Detection, Depth Sorting, Octree Method

Image space processes: Z-buffer, Scanline Method, Area Subdivision, Ray Casting

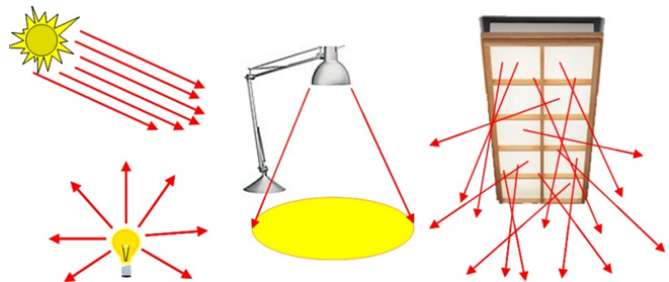
An illumination model (lighting model or shading model) is used to calculate which color/brightness the viewer perceives based on the description of the lighting conditions and the surface properties of an object, i.e. which color the corresponding pixel should receive. In addition to perspective projection, the illumination model is the most important contribution to the realistic appearance of computer graphics images.

For the sake of simplicity, all the following considerations and formulas refer only to the brightness of the lighting. To handle colors, these calculations must be performed in several wavelengths, i.e., in the simplest case, red, green and blue.

Light Sources and Surfaces

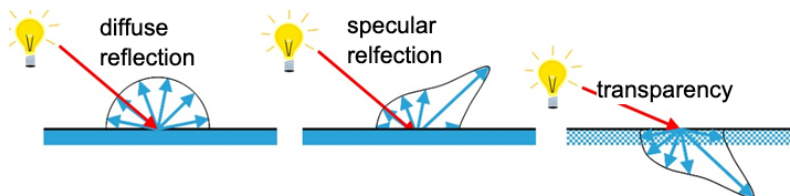
Light Sources

To be able to calculate the influence of lighting, you first need light sources. Characteristics of light sources can be Shape: Light direction, point light sources, directional point light sources, planar light sources, ... Properties: Brightness, color, distance, ...



Object Surfaces

Surfaces can either reflect the incident light diffusely, i.e. the same amount of light is reflected in each direction (e.g. paper, chalk), reflect it specularly, i.e. more light is reflected in the direction of reflection than in the other directions (e.g. paint, metal), or be transparent, i.e. the light passes through the surface and comes out on the other side (e.g. glass, water). Real surfaces usually have a mixture of these properties. Furthermore, it should be noted that light does not only hit surfaces from the light sources directly. Light reflected from other surfaces also plays a role.



A Simple Lighting Model

The physically exact simulation of light and its interaction with object surfaces is very complex. Simplified, empirical lighting models are therefore used in practice. These are structured something like this.

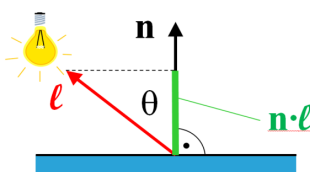
Background Light (Ambient Light)

As every object also emits some of the light that hits it, it is not completely dark even in places where no light source shines directly. This basic, omnipresent light is called *ambient* light, also known as background light. Simple lighting models include a constant value I_a in every lighting calculation.



Lambert's Law

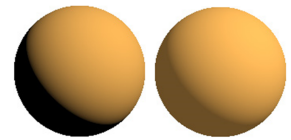
This law states that the flatter the light falls on a surface, the darker this surface appears. It is this effect that gives us the impression of a spatial form.



Let I be the brightness of the relevant light source, and let k_d be the diffuse reflection coefficient of the illuminated surface, i.e. the percentage of incident light that is re-radiated uniformly in all directions. Of course, $0 \leq k_d \leq 1$ applies. Furthermore, let θ be the angle between the surface normal and the direction to the light source, i.e. the direction of light incidence. Then the resulting pixel color L at this surface location applies:

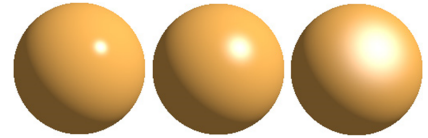
$$L = k_d * I * \cos \theta = k_d * I_l * n \cdot l \quad [n \cdot l \text{ ist skalares Produkt}]$$

If you now also add ambient light, you get a really nicely illuminated sphere (left sphere = diffuse lighting only, right sphere = diffuse + ambient).



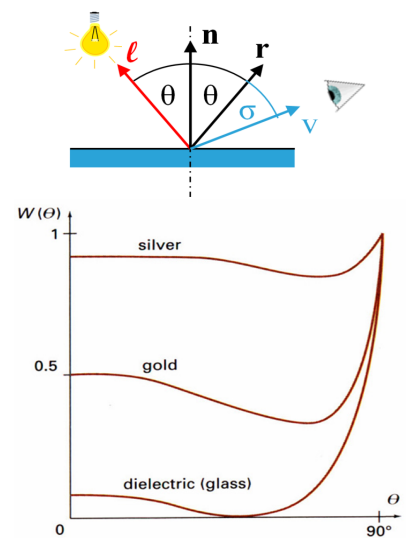
Specular Highlights

Almost every surface is also somewhat reflective. If this aspect is not modeled, then all materials appear equally dull. As the exact specular reflection is extremely complicated to calculate, a simple function is used that has a similar curve to the highlight: $\cos^p$. The free parameter p can be used to control the "polish" of the surface: the larger p is, the smaller the highlight and the smoother the surface appears (left sphere), the smaller the p is, the duller the surface appears (right sphere). In order to be able to add this effect to the lighting to the correct extent, a further factor is introduced, the specular reflection coefficient k_s .



The surface's gloss is then calculated according to this so-called Phong illumination model: $I_{spec} = k_s * I * \cos^p \sigma = k_s * I * (r \cdot v)^p$. The angle σ is the difference between the exact reflection ray r and the direction to the eye v .

To describe lighting more accurately, Fresnel's laws of reflection models the degree of reflection also on the angle of incidence of the light, i.e. coefficient k_s is actually a function $W(\Theta)$ of the direction of incidence of the light. For most materials, however, this value is almost constant. Therefore, this tends to be ignored unless you do want to represent a material where the effect is noticeable. The image on the right shows the dependence of this function $W(\Theta)$ on the angle between the incidence of light and the normal to the surface for three different materials.



When calculating r , it must be remembered that we are dealing with vectors in 3D space, where l , n and r must lie in a plane and all have length one. r results in $r = (2n \cdot l)n - l$. Since the specular highlight function is only a rough approximation anyway, a simpler formula is often used in which $r \cdot v$ is replaced by $n \cdot h$. The angle between n and the bisector h between l and v is very similar to σ . The resulting model is called the Blinn-Phong illumination model.

If we put all the previous components together, we get

$$\text{a simple complete lighting model: } L = k_a * I_a + \sum_{i=1, \dots, N} (k_d * I_i * (n \cdot l) + k_s * I_i * (n \cdot h_i)^p)$$

There are many other aspects that need to be considered in order to get closer to reality, but these are not described in detail here: Color shifts depending on the viewing direction, influence of the distance of the light source, anisotropic surfaces and light sources, transparency, atmospheric effects, shadows, and so on.

Shading of Polygons

Flat Shading

When shading a polygon, each point clearly has the same surface properties, especially the same normal vector. When simply filling each polygon with a color, the boundaries between the polygons become clearly visible. The so-called Mach band effect, which is an edge-enhancing mechanism of the eye, makes the problem even worse than it is. This effect makes us perceive the darker side of edges as darker than it is, and the lighter side as lighter than it is. The simplest solution to this problem is to interpolate the shading between the polygons. Two methods are commonly used for this: Gouraud shading and Phong shading.



Gouraud Shading

Gouraud shading interpolates the calculated brightness values over the polygon areas. To do this, brightness values are calculated at the corner points of the polygons and each polygon is filled from these using linear interpolation. Specifically, it works like this:

1 A normal is calculated at each corner point as the mean value of the normals of all adjacent polygons. Of course, this is only an approximation of the normal of the real underlying surface.

2 A brightness value ("shading") is calculated for each corner point from the properties of the surface, the normal, and the direction of light incidence. Note that adjacent polygons at these corner points all receive the same values.

3 The brightness values are interpolated linearly along the polygon edges, i.e. a value is determined for each intersection with a scanline. Note that this results in the same values for adjacent polygons along the common edge.

4 Linear interpolation is performed again along each scanline from the left to the right polygon boundary. As a result, adjacent pixels always have a very similar brightness and there are no visible edges.

$$L = t \cdot L_1 + (1-t) \cdot L_2$$

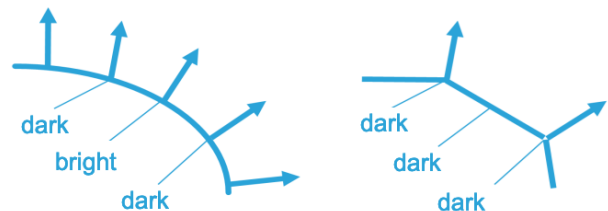
$$L = u \cdot L_2 + (1-u) \cdot L_3$$



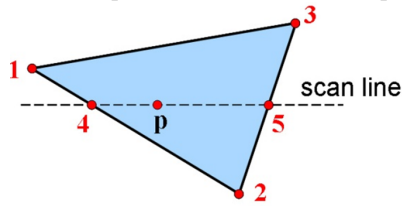
Nevertheless, sources of error remain. For example, the silhouette is of course not changed, which means that bothersome polygon edges remain visible (see image below).



Furthermore, random interpolations occur in the area of specular highlights, depending on whether there happens to be a normal that generates exactly one highlight or not. This is particularly annoying with moving objects.



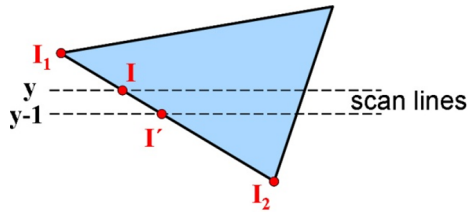
Simple linear interpolation to calculate a pixel value L_p :



$$L_4 = \frac{y_4 - y_2}{y_1 - y_2} L_1 + \frac{y_1 - y_4}{y_1 - y_2} L_2$$

$$L_p = \frac{x_5 - x_p}{x_5 - x_4} L_4 + \frac{x_p - x_4}{x_5 - x_4} L_5$$

The *linear (!) interpolation of the intensities* can of course be done incrementally again, e.g:

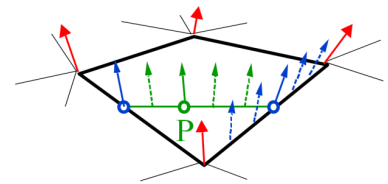


$$L = \frac{y - y_2}{y_1 - y_2} L_1 + \frac{y_1 - y}{y_1 - y_2} L_2$$

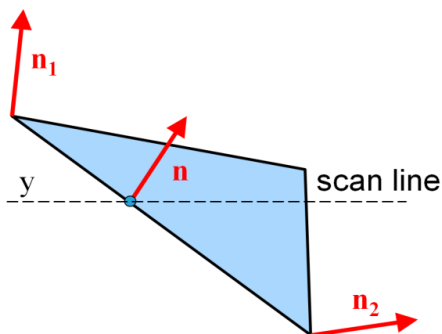
$$L' = L + \frac{L_2 - L_1}{y_1 - y_2}$$

Phong Shading

As an alternative to Gouraud shading, Phong shading (not to be confused with the Phong lighting model!) produces much more consistent gloss effects. As with Gouraud shading, the normals are calculated at the polygon corner points, but now these normals are interpolated along the polygon edges (no brightnesses have been calculated yet!), and then along the scanlines. The brightness is then calculated separately for each pixel according to an illumination model. Although this requires more effort, it also leads to better results.



Normal vector interpolation:



$$n = \frac{y - y_2}{y_1 - y_2} n_1 + \frac{y_1 - y}{y_1 - y_2} n_2$$

PLEASE NOTE: The phong illumination model (also phong shading model) and phong shading (also phong interpolation) are two completely independent things!

Ray tracing is the "tracing of ray tracks", whereby light rays are traversed in reverse. It is a very powerful method that can be used to simulate a number of important optical effects in addition to correct visibility: Shading, shadows, mirror images, and refraction. The simplicity of the method allows even very complex objects to be represented in this way, such as free-form surfaces, fractal surfaces, mathematical functions of all kinds, etc.

The Ray Tracing Principle

The basic principle is to follow the light that hits a pixel in reverse to examine where it comes from in order to draw conclusions about the appearance of the pixel.

Correct Visibility and Shading

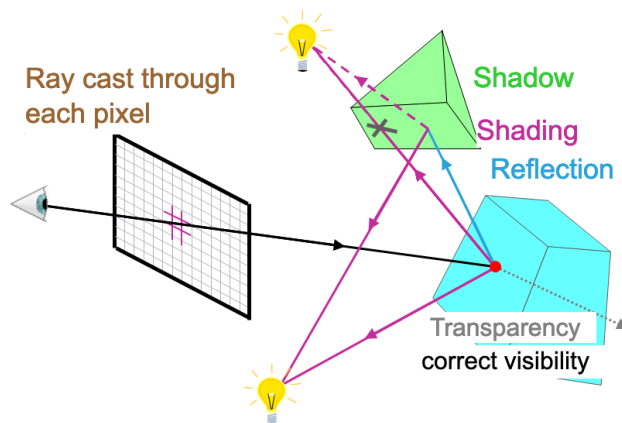
As with ray casting, a *view ray*, the so-called "primary ray", is placed through each pixel and intersected with *all* surfaces in the scene. From the resulting intersection points, select the one that is closest to the image and select the shading of this object point (as viewed from the viewing direction) as the value for said image point. When done for all image points (in the simplest case for all pixels), you will obtain an image of the scene with the correct visibility. Note that any shading model can be used, e.g. the Phong model.



Shading

To calculate the shading of a point, you need the directions to all light sources in addition to the surface normal. However, a light source only has a direct influence on the shading of a point if the incidence of light is not obscured by other objects, i.e. if there are no objects between the point and the light source. To determine this, place a *shadow sensor* (the so-called "secondary ray") from the point to be shaded to the position of the light source. Intersect this straight line with *all* objects in the scene

and do not take the light source into account when obtaining an intersection between the object and the light source. This way, all object parts that lie in the shadow of an obscuring object (in relation to the light source) receive less light than object parts that are visible from the light source without obstruction. A shadow is therefore automatically cast by the objects lying in between.

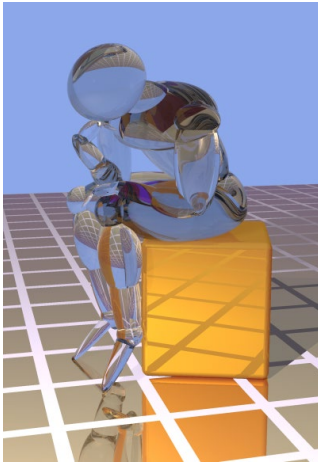


Mirror Images

If the ray of vision hits an object that is a mirror, you do not see this object itself, but the object that is visible from the point of impact in the direction of reflection. Since the law of reflection (angle of incidence is equal to the angle of reflection) is symmetrical, this mirrored object can be found by reflecting the gaze ray on the surface and following a *reflection ray* (also referred to a "secondary ray") in the direction of reflection, intersecting with *all* objects again and selecting the first point of intersection. The shading of this further point of intersection (viewed from the direction of incidence of the reflection ray) is then what the original ray of vision sees. Note that the reflection behavior is calculated locally, meaning that curved mirrors can be created without additional effort.

Transparency

It is now also no problem to handle transparent objects. If the first point of impact of a ray of light is on a transparent object, you can see what the *transparency ray* (also a secondary ray") passing through the transparent object hits. Using the law of refraction, it is easy to set the direction of the transparency beam such that the transparent material refracts the light. The transparency beam is also intersected with *all* objects and the initial



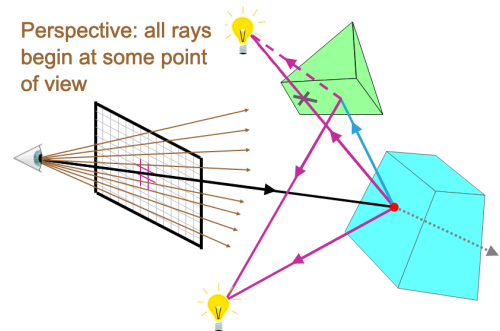
point of intersection is selected. The shading of this point of intersection (viewed from the direction of incidence of the transparency beam) is then what the original ray of vision sees.

Recursion

Every ray except a shadow sensor, i.e. every straight line that represents an inverted beam of light is basically equivalent. This means that if such a beam hits a surface, it is irrelevant for the action at this point whether it is a primary or secondary beam. whether it is a primary or secondary beam. In this way, even multiple reflections and reflections behind transparent material, etc. are just as easily possible.

Perspective

The way in which the primary rays of vision are generated determines how the scene is mapped onto the image plane. If you use parallel rays that are normal to the image plane, you get an orthonormal parallel projection of the objects. If all rays of vision originate from a fictitious eye point, the image is generated from that perspective. Note that the perspective is created naturally without any additional effort (if you disregard optimization methods that exploit the parallelism of the rays).



Ray-Tracing Implementation

Writing a ray tracer is quite simple. You need a function that intersects a straight line with all objects and returns the foremost intersection point.

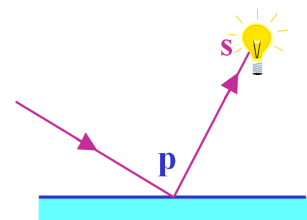
Ray-Tracing Pseudocode:

```

FOR alle Pixel  $p_0$  DO
  1. project ray of vision from eye  $e$  through  $p_0$ ,
     intersect said ray with all objects and select the closest point of intersection  $p$ 
  2. FOR all light sources  $s$  DO
     intersect shadow sensor  $p \rightarrow s$  with all objects
     IF no intersection between  $p, s$  THEN shading += influence of  $s$ 
  3. IF surface of  $p$  is reflective
     THEN track secondary beam; shading += influence of reflection
  4. IF surface of  $p$  is transparent
     THEN track secondary ray; shading += influence of transparency

```

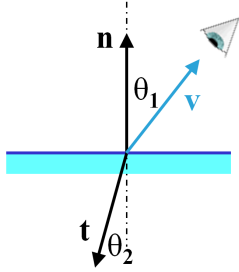
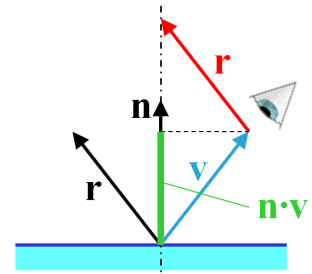
The viewing coordinate system is normally set up so that the xy plane is the image plane and the main viewing direction coincides with the negative z axis. Rays are used in a parameterized form: Starting point p_0 plus parameter t times direction vector d : $p(t) = p_0 + t \cdot d$.



Primary rays are therefore **eyepoint** $e + t \cdot (\text{pixel } p_0 - \text{eyepoint } e)$,

Shadow sensors are **surfacepoints** $p + t \cdot (\text{lightsourceposition } s - p)$,

Reflection rays are $p + t \cdot r$, where $r = (2n \cdot v)n - v$ is the direction of reflection of the ray v , which results from the law of reflection (angle of incidence = angle of reflection). This calculation also guarantees that r has a length of 1. (see sketch on the right)

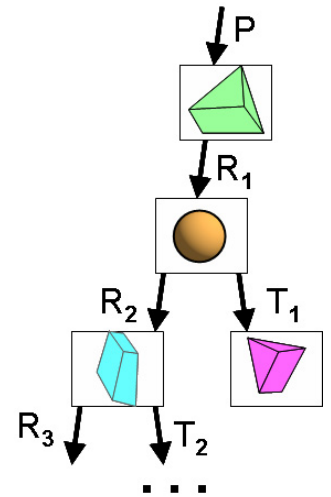
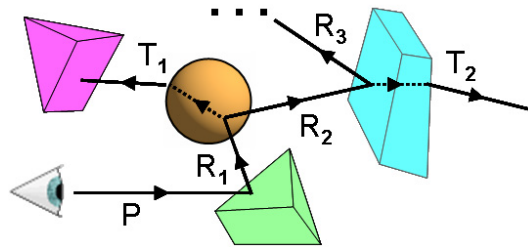


Transparency rays are $p + t \cdot t$, where t results from Snellius' law of refraction $\sin \theta_1 : \sin \theta_2 = \eta_2 : \eta_1$ (η_i is the refractive index of the material i):

$$t = -\frac{\eta_1}{\eta_2} v - (\cos \theta_2 - \frac{\eta_1}{\eta_2} \cos \theta_1) n$$

The vector t also has the length 1 (see left sketch).

If the light rays are now "tracked in reverse in this way, a recursive call sequence is created (see sketch below), which corresponds to a ray tree (right-hand sketch). Normally, however, this tree is not saved in this form but is only a symbolic representation of the recursion call sequence.



Intersections between Beams and Objects

Objects that are to be displayed with ray tracing must fulfill a few conditions:

- it must be possible to calculate the intersection point with a straight line,
- the surface normal must be known at this point
- material properties must be available (at this point).

This is of course easy for BReps, but also for CSG trees that are evaluated recursively. However, many other data formats also meet these conditions (e.g. free-form surfaces). Functions that calculate the intersection with a ray must therefore be provided for each primitive type. As an example, this is described in more detail for a sphere and a polygon.

Beam-Ball Section

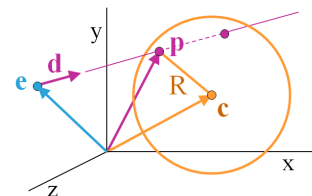
Spherical equation: $|p - c|^2 - R^2 = 0$

The ray is inserted into this: $|(e + td) - c|^2 - R^2 = 0$

Then Δp is introduced for better readability: $\Delta p = c - e$

And you get a quadratic equation in t : $t^2 - 2(d \cdot \Delta p)t + (|\Delta p|^2 - R^2) = 0 \quad d^2 = 1$

The 2 solutions correspond to the two intersection points with the sphere: $t = d \cdot \Delta p \pm \sqrt{(d \cdot \Delta p)^2 - |\Delta p|^2 + R^2}$.



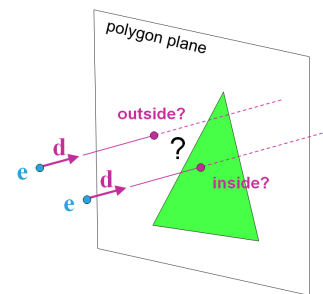
In cases where $R^2 \ll |\Delta p|^2$ (which is quite common), rounding errors occur in this formula. To avoid these, you can use $d^2 = 1$ and transform the formula so that rounding errors become less likely:

$$t = d \cdot \Delta p \pm \sqrt{R^2 - |\Delta p - (d \cdot \Delta p)d|^2}$$

Beam-Polygon Intersection

When intersecting a ray with a polygon, one first tests whether the polygon is even looking in the right direction (see Backface Detection). The ray $p = p_0 + t \cdot d$ is then inserted into the plane equation $Ax + By + Cz + D = 0$ of the polygon, which, since $n = (A, B, C)$, can also be written in the form $n \cdot p = -D$: $n \cdot (p_0 + t \cdot d) = -D$. This results in $t = -(D + n \cdot p_0)/n \cdot d$, which, when inserted into the beam equation, gives the intersection point with the plane.

Now you need to check whether the intersection point is inside the polygon edges or next to the polygon (see illustration on the right). This can be done after projection onto a main plane in 2D.

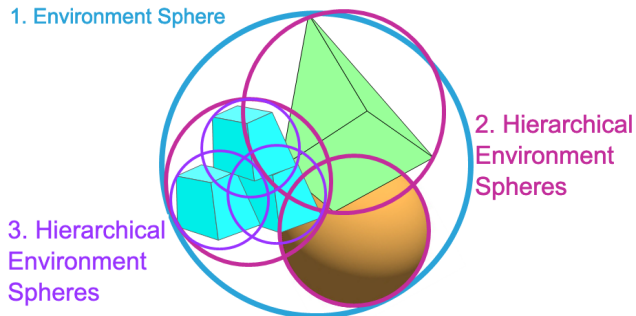


Ray-Tracing Acceleration

Ray tracing is an extremely complex process. Mapping a scene with only 1000 polygons or objects onto an area of 1000×1000 pixels requires 10^9 intersection calculations for the primary rays alone without optimizations. It is therefore necessary to speed up the process significantly. The most important method for this is to reduce the number of necessary intersection calculations by utilizing coherence.

Object Environments

1. Environment Sphere



Before cutting all the individual parts of a more complex object (or part of a scene) with a beam, you can check whether the beam even comes close to this object. For this purpose, *environmental spheres* (bounding spheres) are inserted in the data structure for such complex objects, which completely enclose the object. Rays that do not hit this bounding sphere will certainly not hit the object, and many unnecessary intersection calculations are saved. The concept can be applied hierarchically, i.e. complex partial objects all receive surrounding spheres, their parts again, etc., until simple objects are reached. In this way, the actual intersection

attempts are reduced from $O(n)$ to about $O(\log n)$. Instead of environment spheres, you can use any other object environment, e.g. environment cuboids. It is important to weigh whether the additional effort due to their more complicated shape justifies the gain due to their narrower concern.

Space Division Methods

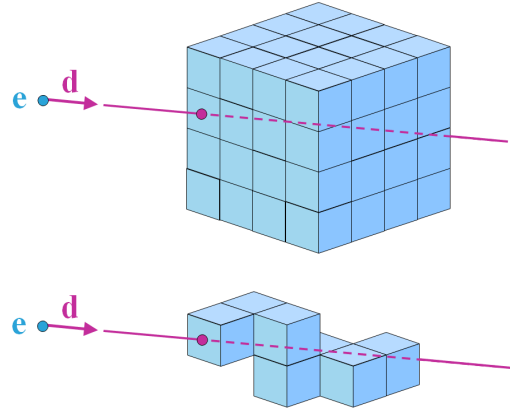
Alternatively, you can also divide the entire room in which the scene is located into a regular grid, regardless of the objects. It does not matter whether you save the subcubes in an array or use an octree. You then only need to intersect the objects with the beam that lie in the subspaces through which the beam passes. You therefore need to quickly calculate which is the next subcube on the beam path. Once you have found an intersection point in a subcube, you can stop! Algorithms similar to a 3D Bresenham method can be used for this.

You can also proceed as follows for individual subcubes:

The ray $p(t) = p_0 + t \cdot d$ enters the subcube at p_{in} . The normal vectors of the cube faces are

$$(1, 0, 0), (-1, 0, 0), (0, 1, 0), (0, -1, 0), (0, 0, 1), (0, 0, -1)$$

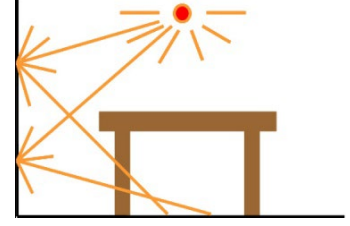
. For the three surfaces with $d \cdot n > 0$ (the others are out of the question!), determine the point of intersection with the ray and select the foremost point (smallest t). This method also works if the cubes are of different sizes (e.g. in an octree).



$$\begin{aligned} p_{out,k} &= p_{in} + t_k d \\ n_k \cdot p_{out,k} &= -D_k \\ t_k &= \frac{-D_k - n_k \cdot p_{in}}{n_k \cdot d} \end{aligned}$$

Radiosity

The *Radiosity* method originated in thermodynamics and models the propagation of light in a closed system, taking into account the energy equilibrium. The method describes the physical process of the propagation of light in a diffusely reflective environment, i.e. the calculation of the brightness of all surfaces in a scene, taking into account their mutual influence. This means that even surfaces that are not directly illuminated are given a certain brightness. Each illuminated object acts as a secondary light source. For image generation on B_i , the light propagation in space is first calculated without knowing the camera position, whereby it is assumed for simplicity's sake that the observer does not influence the propagation of the light. The objects can then be displayed from different directions without having to recalculate the light propagation each time.



The Radiosity Equation

The scene consists of n plane polygons P_i , which are referred to as *patches* in the radiosity method. To simplify matters, it is assumed that each patch has a homogeneous, perfectly diffuse surface. Light sources are also patches that emit their generated light evenly in all directions. The **radiosity** B_i of P_i is the total radiated energy, i.e. the sum of self-radiation and reflection as power per unit area. This light energy density is proportional to the perceived brightness. As a next simplification, it is assumed that the radiosity has the same value for all positions on a patch. Under these conditions, the formula for the radiosity of a patch looks like this:

$$B_i = E_i + \rho_i \cdot \sum_{j=1}^n B_j \cdot F_{ij}$$

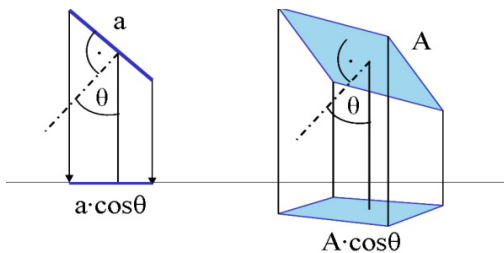
Here E_i is the intrinsic emission of P_i , ρ_i is the diffuse reflection coefficient of the surface (that indicates how many % of the incident light is diffusely reflected, also called *albedo*), n is the number of patches of the scene, B_j are the radiosities of all other patches, and F_{ij} are the so-called *form factors*, which indicate what proportion of the radiosity acting on P_i comes from P_j (this is the same as the proportion of radiosity from P_i that hits P_j). F_{ij} are purely geometric quantities and independent of light sources or radiosity values, and can therefore be calculated before the radiosity is determined. The radiosity formulas for n patches result in a linear system of n equations in n unknowns B_i :

$$B_i - \rho_i \sum_{j \neq i} B_j F_{ij} = E_i$$

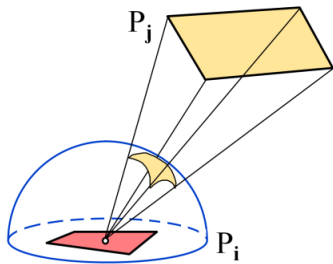
$$\begin{bmatrix} 1 & -\rho_1 F_{12} & \cdots & -\rho_1 F_{1n} \\ -\rho_2 F_{21} & 1 & \cdots & -\rho_2 F_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ -\rho_n F_{n1} & -\rho_n F_{n2} & \cdots & 1 \end{bmatrix} \cdot \begin{pmatrix} B_1 \\ B_2 \\ \vdots \\ B_n \end{pmatrix} = \begin{pmatrix} E_1 \\ E_2 \\ \vdots \\ E_n \end{pmatrix}$$

This system of equations can only be solved iteratively.

Calculation of Form Factors

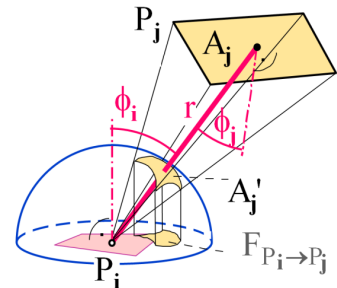


In order to derive a formula for the form factors F_{ij} , we need a simple geometric relationship: The area of the normal projection of a surface A onto another surface is reduced by the cosine of the angle between the surfaces, i.e. $A \cdot \cos \theta$.



The *shape factor* F_{ij} is now defined as the proportion of the radiant energy emitted by the patch P_i that hits the patch P_j . In other words, what percentage of the energy from patch P_i hits patch P_j . It is easy to imagine that this value also tells you what percentage of the incoming energy on patch P_i comes from patch P_j .

F_{ij} can now be calculated as follows: Let us assume that the size of the patches is small in relation to their distance r . Let A_j be the area of P_j . Above the patch P_i we imagine a hemisphere with a radius of 1, onto which the patch P_j is projected.



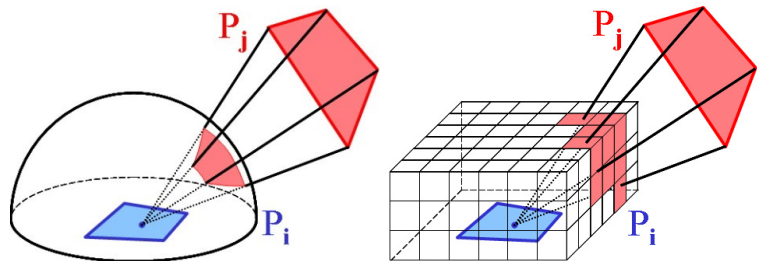
The resulting area A'_j then has approximately the size $A_j \cos \phi_j$, where ϕ_j is the angle between the patch normal and the connection between the two patches. If you now consider that energy incident on patch P_i at an angle is proportional to the cosine of the angle of incidence, you must multiply this value by $\cos \phi_i$ (this corresponds to a projection onto the bottom surface of the hemisphere) in order to obtain the correct proportion of the influence of patch P_j . Since the sum of all shape factors F_{ij} for a patch P_i must, of course, be 1 (100%), we normalize the result by the size of the base area of the hemisphere, i.e. by $1^2\pi = \pi$, and obtain the form factor:

$$F_{ij} = \frac{\cos \phi_i \cos \phi_j A_j}{\pi r^2}$$

This formula applies under the condition that there are no obstacles between the two patches, i.e. that the light can pass unhindered from P_i to P_j . Correct form factors must therefore also take mutual visibility into account.

The *reciprocity principle* also applies to the form factors, which relates the dependence of the form factors between two patches to each other: $\mathbf{A}_i \cdot \mathbf{F}_{ij} = \mathbf{A}_j \cdot \mathbf{F}_{ji}$.

In practice, a half cube ("hemicube") is used to estimate the form factors instead of a hemisphere over P_i , and the entire scene is mapped onto this. This can be done using *z-buffer* technology, i.e. the surface of the hemicube is regularly divided into pixels and all other patches are mapped onto these with the center of the cube as the projection center. For each pixel, its shape factor is determined in advance and then these proportions are added together for each patch to form its shape factor. Alternatively, shape factors can also be calculated using ray tracing methods.

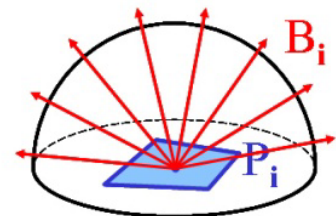


Progressive Refinement

To solve the system of equations iteratively, select the brightest patch and distribute its energy to all the others ("kicking"). This makes all patches slightly better in one step. And because the energy of the brightest patch is always distributed next, the method converges very quickly.

Let $B_{(i \text{ from } B_j)}$ be the radiosity component of P_i caused by B_j . The influence of P_i on B_j is symmetrical to the influence of P_j on B_i :

$$B_{(i \text{ from } B_j)} = \rho_i B_j F_{ij}, \quad \text{thus} \quad B_{(j \text{ from } B_i)} = \rho_j B_i F_{ji}.$$



From this and from $A_i \cdot F_{ij} = A_j \cdot F_{ji}$ follows $\mathbf{B}_{(j \text{ from } B_i)} = \rho_j \mathbf{B}_i \mathbf{F}_{ij} (\mathbf{A}_i / \mathbf{A}_j)$, so the form factor F_{ij} can be used. For each patch, in addition to the radiosity B_j collected so far (i.e. the best estimate so far), the "not yet shot radiosity" ΔB_j is also saved, which is the basis for selecting the next "brightest" patch. At the beginning, the B_j and the ΔB_j are of course initialized with E_j : $B_j = \Delta B_j = E_j$ for all j .

Simplified, an iteration step therefore looks like this:

```

select patch  $i$  with highest  $A_i * \Delta B_i$ 
FOR selected patch  $i$  { set up hemicube
                        calculate form factors  $F_{ij}$  }
FOR each patch  $j$  {  $\Delta rad := \rho_j * \Delta B_i * F_{ij} * A_i / A_j$ 
                   $\Delta B_j := \Delta B_j + \Delta rad$ 
                   $B_j := B_j + \Delta rad$  }

 $\Delta B_i := 0$ 

```

This process is also known as progressive refinement.



Abbildung 21: Three Examples of Radiosity-Images (©Lischinski; Hrbek; Feldman & Wallace)

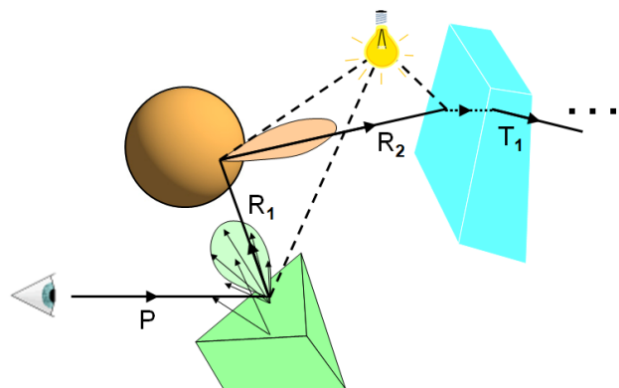
Aspects of Radiosity

Radiosity is a *viewpoint-independent* method for calculating the brightness of the individual (diffuse) patches, after which a rendering step is still necessary. A simple polygon method with Gouraud shading is usually used. However, in order to achieve the effects possible with ray tracing in the finished image, the diffuse shading values obtained in this way can also be used as base values for ray tracing. This also allows reflections and shadows etc. to be displayed beautifully.

The basic principle of radiosity presented here can be extended in many ways. To reduce the number of patches, they can be structured hierarchically so that more distant patches do not have to be treated individually. In order not to generate false smearing of the lighting due to too coarse patches in places where the lighting changes abruptly (e.g. shadow edges), and conversely not to make the patches too small in these places, *Discontinuity-Meshing* is used.

Path Tracing

Path tracing is actually an extension of ray tracing (also called "Monte Carlo ray tracing"), in which secondary rays are not placed in all relevant directions at each point of impact, but only one direction is randomly selected according to the distribution function valid there. This makes it possible to include situations where many different directions of light are relevant, such as diffuse reflection and extended light sources. Of course, many rays must now be calculated and averaged per pixel in order to avoid excessive noise in the resulting image. In principle, this procedure corresponds to the Monte Carlo integration of a multidimensional integral that describes the propagation of light in space ("rendering equation"). The use of quasi-random numbers instead of pseudo-random numbers significantly reduces the variance.



Photon Mapping

With the *photon mapping method*, light rays are traced from the light sources, i.e. similar to ray tracing, but in the forward direction. The effect of the light is stored at points of impact and the appearance of the object is later interpolated between these values. This makes it possible to correctly calculate the effect of light sources even in complicated situations (e.g. reflections of the light source(s) or caustics). A combination of path tracing and photon mapping can integrate almost all light effects into an image.

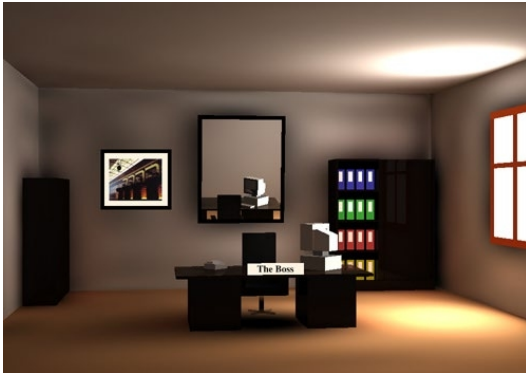
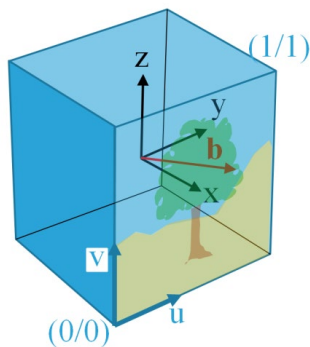


Abbildung 22: Examples for Global Illumination Results: left: Radiosity + Ray-Tracing (see Reflections) right: Path-Tracing + Photon-Mapping (©Fedat; VRVis)

An essential optical property of natural surfaces is that they exhibit various irregularities. This can be caused either by the environment, by a variable coloration of the surface, or by the shading caused by surface irregularities. For these three causes, the effects can be simulated using the three methods *Environment Mapping*, *Texture Mapping* and *Bump Mapping*. Mapping means *Mapping*.

Environment Mapping



Environment refers to the surroundings, the environment of an object. Depending on the surface properties of the object, the environment will have a different effect on the appearance of the object. For perfectly reflective surfaces, ray tracing can be used to create the exact mirror image of the environment. For imperfectly diffuse surfaces, the Phong model can be used to approximate gloss effects in the surroundings of the reflection of light sources. However, a more or less reflective surface reflects its entire surroundings more or less sharply, whereby the accuracy is greatly reduced. In order to efficiently render objects in a complex environment without having to completely model the entire environment and without claiming exactness, *Environment Mapping* is used. In a pre-processing step, the environment is produced as an image from

a central point (e.g. the center of the object or the scene to be displayed).

Whether this is a calculated or a photographed image is irrelevant. In any case, it is assumed that the environment (e.g. sphere or cube) is large in relation to the objects that are to be displayed. Now, when representing an object, it is approximately assumed for each surface point that it lies in the center of this environment. This allows you to quickly determine *only from the direction* of the reflection ray which point of the environment is hit (e.g. with polar coordinates) and saves time-consuming ray tracing.

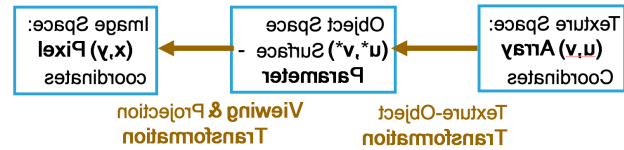
Axis-parallel cubes are often used as environment maps, these are called *Cube-Map*. If each side contains an image of size $u \times v = [0, 1] \times [0, 1]$, the image value can be determined efficiently for the direction vector of the reflection ray $\mathbf{r} = (x_r, y_r, z_r)$, e.g. for the side $x > 0$ by $u = (y_r + x_r)/2x_r$ and $v = (z_r + x_r)/2x_r$. The reflection direction $\mathbf{r}$ is again obtained according to the same principle as already derived for ray tracing and shading methods: $\mathbf{r} = (2\mathbf{n} \cdot \mathbf{v})\mathbf{n} - \mathbf{v}$, where $\mathbf{v}$ is the vector to the observer and $\mathbf{n}$ is the surface normal.



Texture Mapping

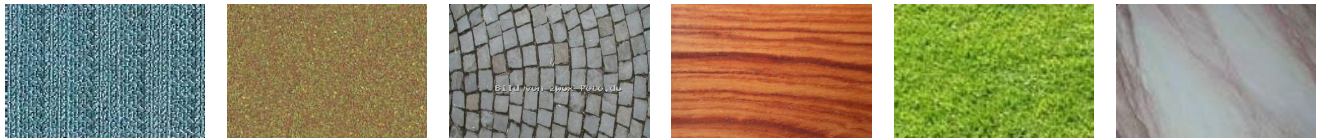
Many surfaces are not monochrome, but have a pattern, e.g. wood grain, pictures on the wall, writing on paper, dirt, clothes, or marble. Even with coarse modeling, details can be interpreted as patterns, e.g. windows on a house wall, clouds in the sky, faces, buttons, zippers, or cobblestones. Such patterns are called textures. The application of textures is called *Texture Mapping*.

Once the textures have been created, texture mapping takes place in two steps. First, it must be defined which texture is to be applied to which surface of the objects and how it is to be oriented, scaled, etc. This is actually part of the modeling, where the surfaces are given an appearance. And in the second step, the texture must then be correctly rendered onto the image of the objects, i.e. transformed into the image.



Creating a Texture

Basically, it doesn't matter where a texture comes from, it just has to be defined and accessible at all points. A texture is usually created as a pixel array in a pre-processing step and then only accessed. You can therefore use a photograph as well as a scan, but also a texture generated by a program or even random values. You can also create a database for frequently used textures such as wood grain, grass, sand, marble, cobblestones, or fabric structures. If you use textures that are obtained from a mathematical function, this is also known as "procedural texturing".

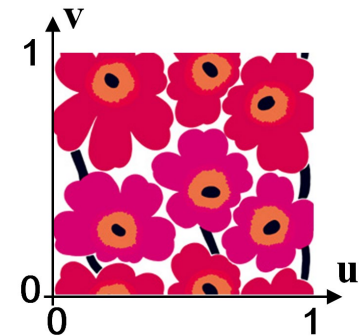


Texture-Object-Transformation

Normally, the texture will be in a 2D coordinate system, which we want to address with (u, v) . Furthermore, we want to assume that a surface to which the texture is to be applied also has a parametric representation, which we denote by (u^*, v^*) . A bilinear function for applying the texture then looks like this:

$$u^* = u^*(u, v) = a_u u + b_u v + c_u, \quad v^* = v^*(u, v) = a_v u + b_v v + c_v$$

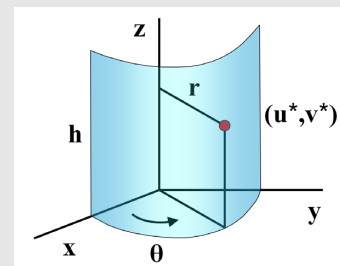
i.e. you can determine the corresponding color for each point of the object surface. This function is called texture-object transformation and is denoted by M_T .



Example:

A texture $t(u, v)$, $0 \leq u, v \leq 1$, is to be applied to a quarter cylinder with height h , whose surface is parameterized with v^* in the z -direction and with $u^* (= \theta)$ along the curvature. In order to calculate the position of a texture pixel $t(u, v)$ [also called texel] in the cylinder, the mapping M_T must be defined, which could be:

$u^* = u \cdot \pi/2$, $v^* = v \cdot h$ (so the texture fits exactly on the cylinder quarter).



Viewing and Projection Transformation

In itself, the mapping of the 3D model onto an image plane is a simple projection M_{VP} . In order to color each pixel exactly once when raster scanning the surfaces (i.e. to avoid overpainting and leaving holes), we now work in the opposite direction. For each pixel (x, y) you determine which surface point is drawn there (i.e. the (u, v) coordinates of the surface) and then from this which texel is valid for this pixel. This requires the inverse operators M_{VP}^{-1} and M_T^{-1} .

Example (continued):

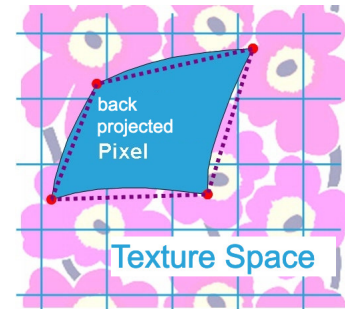
For any projection, it is sufficient if we know the (x, y, z) coordinates of each point. In the case of the cylinder above, this results in $x = r \cdot \cos u^*$, $y = r \cdot \sin u^*$, $z = v^*$.

Now the problem is approached in reverse:

- For an image point P , first determine the position (x, y, z) on the cylinder that is represented there (e.g. by ray casting).
- For this point you have to find the parameters of the surface: $u^* = \cos^{-1}(x/r)$, $v^* = z$. Up to this point, this is the inverse transformation M_{VP}^{-1} .
- Now the texture must be found for the parameter pair (u^*, v^*) by inverting M_T : $u = 2u^* / \pi$, $v = v^* / h$ (this is M_T^{-1})

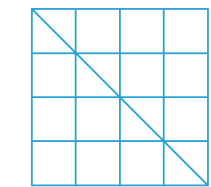
Anti-Aliasing for Textures

Textures are particularly susceptible to aliasing effects, especially when the patterns are enlarged or reduced in size. The correct method would be to calculate the average texture value of the area generated by a back projection of the pixel. As an approximation, the rectangle created by connecting the back-projected corner points is also sufficient. As this would be very slow, one of two optimizations is often used: *Mip-Mapping*, where the texture is precalculated in different sizes and then linearly interpolated, and the *Summed-Area-Table-Method*, where the average values of rectangular areas are easily determined from a summed texture by taking differences.



Textures on Perspective Distorted Triangles

The use of barycentric coordinates results in a linear interpolation of the corner point values over the triangular surface. However, this linearity of the surface parameters is lost during the perspective transformation. Therefore, the interpolation must take place before the homogenization, as it is still linear there. Instead of calculating the color of a point $\mathbf{p}(\alpha, \beta, \gamma) = \alpha \cdot \mathbf{a} + \beta \cdot \mathbf{b} + \gamma \cdot \mathbf{c}$ with $color(x, y) = \alpha \cdot t_0 + \beta \cdot t_1 + \gamma \cdot t_2$, barycentric coordinates $(\alpha_w, \beta_w, \gamma_w)$ are determined for the texture parameters u and v before the perspective and used to calculate the texture value at the point $\mathbf{p}(\alpha, \beta, \gamma) = \mathbf{p}(x, y)$:



correct
interpolation

linear
interpolation

Texture on 2
Triangles

$$d = h_1 h_2 + h_2 \beta (h_0 - h_1) + h_1 \gamma (h_0 - h_2) \quad \beta_w = h_0 h_2 \beta / d \quad \gamma_w = h_0 h_1 \gamma / d \quad \alpha_w = 1 - \beta_w - \gamma_w$$

$$u = \alpha_w u_0 + \beta_w u_1 + \gamma_w u_2 \quad v = \alpha_w v_0 + \beta_w v_1 + \gamma_w v_2 \quad color(x, y) = t(u, v)$$

Solid Texturing

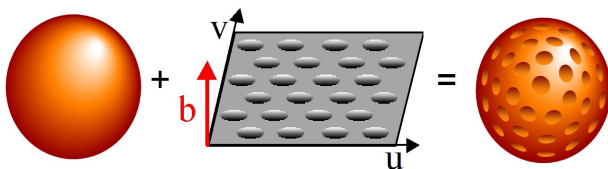
In addition to 2-dimensional form, a texture can also be given as a 3D volume. This means that a color is defined for each spatial point in a 3-dimensional parameter space, which is then retrieved from a surface located at this spatial point. With this method, the texture can either be given as a mathematical function or by volume data, the only important thing is that the values can be queried for each spatial point. The great advantage of solid texturing is that the patterns continue coherently over all edges, so that there can be no merging problems between any polygons. Furthermore, mapping the texture to the object is of course much easier to handle.



Bump Mapping

Many surfaces have a detailed geometric structure: bark, coins, plaster, leather, fabric, vegetables, planets, gravel paths, lasagna, tiles, chocolate bars, earthworms, etc. Modeling such objects completely is very tedious and generates huge amounts of data. Bump mapping can significantly reduce this effort.

If you look at the gray bar on the right, you get the impression that it has six bumps and one indentation. But if you touch it, it is completely flat! Why do we see the unevenness? Because the shading alone is enough to create a spatial impression! This trick can be used to create the impression of surface irregularities (bumps) with little effort.



The basic idea is to leave the surface unchanged but to change the normal vector according to the bumps. As a result, the shading corresponds to the bumps, but geometrically nothing needs to be changed.

Bump-Mapping Algorithm

Let the bump texture be given in the form of an array of height values $b(u, v)$, which means that the position of the point $p(u, v)$ of the surface generated by the parameter pair (u, v) should appear shifted by $b(u, v)$ in the direction of the normal vector n at this point. n is obtained by normalizing the cross product of two tangent vectors to the length 1:

$$n^* = p_u \times p_v, \quad n = n^* / |n^*|$$

The displaced point $p'(u, v)$ then results in: $p'(u, v) = p(u, v) + b(u, v) \cdot n$. However, we need n' , i.e. the normal to the shifted point:

$$n' = p'_u \times p'_v$$

Now the following applies:

$$p'_u = \partial(p + b_n) / \partial u = p_u + b_u n + b n_u$$

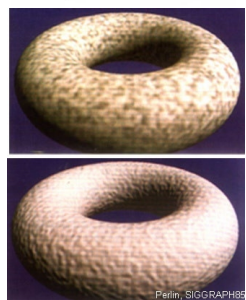
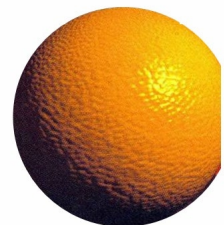
and because b is very small: $p'_u \approx p_u + b_u n$ analogously, of course, $p'_v \approx p_v + b_v n$, so that n' results in:

$$n' = p'_u \times p'_v = p_u \times p_v + b_v(p_u \times n) + b_u(n \times p_v) + b_u b_v(n \times n)$$

and from $n \times n = 0$ finally follows: $n' = n + b_v(p_u \times n) + b_u(n \times p_v)$.

You therefore do not need $b(u, v)$ for the calculation, but the derivatives of u and v . Since in practice the parameterization of the surface and the bump texture are often the same, you can easily precalculate these derivatives and save them instead of $b(u, v)$.

Note the difference between a texture map (top) and a bump map (bottom) in the donut illustration on the right. The spatial impression of the surface is only created when the shading is given a directional dependency.

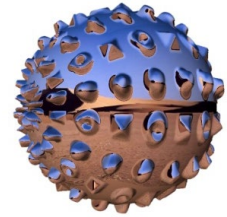


Of course, bump mapping is just a big scam where the shading is changed without correcting the geometry. Accordingly, visible errors remain, which become all the more obvious the higher the bumps are:

1. at flat angles the structure is strongly distorted
2. the silhouette remains as smooth as the original geometry is (i.e. smooth = wrong)
3. therefore the object also casts shadows with smooth (= false) edges
4. the bumps do not cast shadows on each other
5. Surface normals on the side of an object facing away from the light can nevertheless be turned towards the light and then falsely receive light!

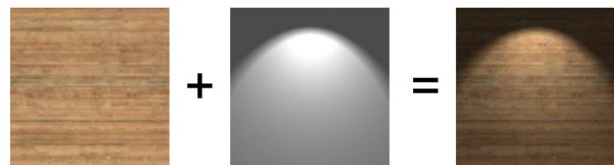
Displacement Mapping

There are more or less complex workarounds for each of these errors, but the correct method is to actually change the surface by the bump height. This method is called *displacement mapping*. The surface points are actually shifted and a correct silhouette is naturally created. However, this is much more complex to implement but is supported by newer graphics cards. This is done by means of an additional programmable hardware stage in the rendering pipeline between the vertex and pixel stage ("tessellation stage"), which divides triangles directly on the hardware into many smaller triangles and can move them according to a displacement map.

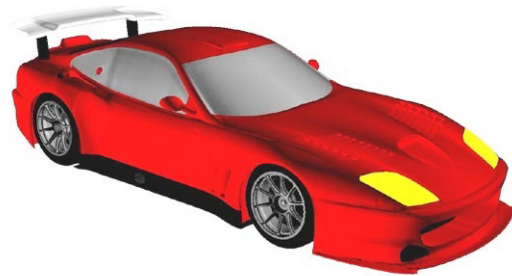
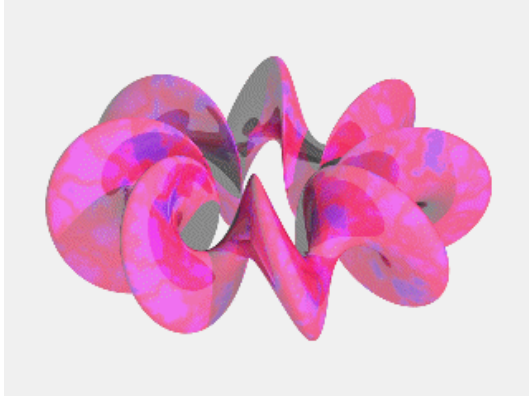


Combination of several Mappings

A very powerful method is to combine several mappings. In the case of textures, this is also called *multi-texturing*. Examples of combinable textures are basic patterns, lighting, dirt, and unevenness, but also, for example, photos plus annotations.



In addition to the simple elements, many graphical data processing applications require the ability to model and display arbitrary surfaces (free-form surfaces). The principles are first explained using free-form curves, as their mathematics is simpler. The methods developed there can easily be extended to surfaces.



Quadratic Surfaces

Frequently used surfaces can be defined implicitly, explicitly, or parametrically by formulas (analytical representation).

- In the *implicit* representation of a quadratic surface, all points (x, y, z) , that fulfill its formula, simply lie on the shape's surface (example sphere: $x^2 + y^2 + z^2 = r^2$).
- *Explicit* representations depict a coordinate value as a function of others (example sphere: $z = \sqrt{r^2 - x^2 - y^2}$).
- In a *parametric* representation (example sphere: $x = r \cdot \cos\varphi \cdot \cos\theta$, $y = r \cdot \cos\varphi \cdot \sin\theta$, $z = r \cdot \sin\varphi$, $-\pi/2 \leq \varphi \leq \pi/2$, $-\pi \leq \theta \leq \pi$), a point on the shape's surface is generated for each combination of parameter values (in our sphere example (φ, θ)). Other commonly used quadratic surfaces include ellipsoids, toruses/tori, and quadrics.

Curves

General curves can be defined either by a *formula* (analytical representation) or by specifying a set of *support points* or *control points* that determine the course of the curve. Since an analytical definition is much more difficult for the user to understand, the second method is normally used.

The following properties characterize different curve types:

- *interpolation* (curve runs through its support points) versus *approximation* (control points lie next to the curve)
- *degree of continuity* at connections
- *global* influence (all points influence every curve point) versus *local* influence (points only influence nearby curve parts)
- *axis-dependent* representations (rotation of the coordinate system changes the curve) versus *axis-independent* representations (rotation of the coordinate system does not change the curve)
- Tendency to *damping* at corners versus tendency to *overshoot*
- Possible curve shapes, restrictions, double points, closed curves, etc.

Curves that are defined by support or control points are also called *splines*. Some simple common spline methods are described below.

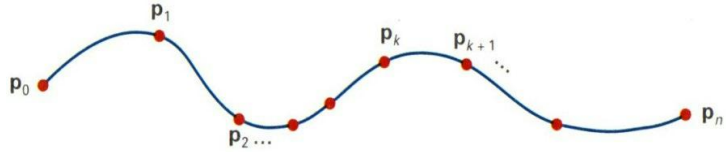
Cubic Spline Interpolation

Given $n+1$ interpolation points $\mathbf{p}_i = (x_i, y_i, z_i)$, $i = 0, \dots, n$, an interpolation curve that consists of 2 interpolation points each (placed on a cubic polynomial) is called a cubic spline. The curve between interpolation points $\mathbf{p}_k$ and $\mathbf{p}_{k+1}$ is described by a parameter u :

$$\mathbf{p}_k(u) = \mathbf{a}_k u^3 + \mathbf{b}_k u^2 + \mathbf{c}_k u + \mathbf{d}_k$$

where $k = 0, 1, 2, \dots, n-1$ and $0 \leq u \leq 1$

(Note: $\mathbf{a}_k, \mathbf{b}_k, \mathbf{c}_k, \mathbf{d}_k$ are vectors). To calculate a curve segment between 2 interpolation points, you need 4 segments. If the definition at the interpolation points is chosen such that the cubic polynomials there are connected both C^1 -continuously (differentiable) and C^2 -continuously (2x differentiable, i.e. same curvature), we speak of natural cubic splines. These are obtained by solving a system of equations with $4n$ variables and specifying 2 constraints at the edges, e.g. curvature at the beginning and at the end is zero. Cubic splines have the disadvantage that each support point has an influence on the entire curve, i.e. it has a global influence.



Hermite Interpolation

Hermite interpolation is a special form of cubic spline, for which, in addition to the interpolation points $\mathbf{p}_k$, the derivatives $\mathbf{Dp}_k$ are also specified at the interpolation points. The cubic interpolation polynomial $\mathbf{p}_k(u)$, $0 \leq u \leq 1$, between points $\mathbf{p}_k$ and $\mathbf{p}_{k+1}$ can then be calculated uniquely from the 4 determinants:

$$\mathbf{p}_k(0) = \mathbf{p}_k, \quad \mathbf{p}_k(1) = \mathbf{p}_{k+1}, \quad \mathbf{p}'_k(0) = \mathbf{Dp}_k, \quad \mathbf{p}'_k(1) = \mathbf{Dp}_{k+1} \quad \text{für } k = 0, \dots, n-1$$

$\mathbf{p}_k(u) = \mathbf{a}_k u^3 + \mathbf{b}_k u^2 + \mathbf{c}_k u + \mathbf{d}_k$ can also be written in matrix notation, as well as the first derivative of this curve $\mathbf{p}'_k(u) = 3\mathbf{a}_k u^2 + 2\mathbf{b}_k u + \mathbf{c}_k$. From this, the 4 determinants $\mathbf{p}_k, \mathbf{p}_{k+1}, \mathbf{Dp}_k, \mathbf{Dp}_{k+1}$ can be formulated:

$$\mathbf{p}_k(u) = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{a}_k \\ \mathbf{b}_k \\ \mathbf{c}_k \\ \mathbf{d}_k \end{bmatrix} \quad \mathbf{p}'_k(u) = \begin{bmatrix} 3u^2 & 2u & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{a}_k \\ \mathbf{b}_k \\ \mathbf{c}_k \\ \mathbf{d}_k \end{bmatrix}$$

$$\begin{bmatrix} \mathbf{p}_k \\ \mathbf{p}_{k+1} \\ \mathbf{Dp}_k \\ \mathbf{Dp}_{k+1} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{a}_k \\ \mathbf{b}_k \\ \mathbf{c}_k \\ \mathbf{d}_k \end{bmatrix}$$

To calculate the coefficient vectors $\mathbf{a}_k, \mathbf{b}_k, \mathbf{c}_k, \mathbf{d}_k$ of $\mathbf{a}_k u^3 + \mathbf{b}_k u^2 + \mathbf{c}_k u + \mathbf{d}_k$, this matrix is inverted. The resulting matrix is called the Hermite matrix $\mathbf{M}_H$:

$$\begin{bmatrix} \mathbf{a}_k \\ \mathbf{b}_k \\ \mathbf{c}_k \\ \mathbf{d}_k \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{bmatrix}^{-1} \cdot \begin{bmatrix} \mathbf{p}_k \\ \mathbf{p}_{k+1} \\ \mathbf{Dp}_k \\ \mathbf{Dp}_{k+1} \end{bmatrix} = \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{p}_k \\ \mathbf{p}_{k+1} \\ \mathbf{Dp}_k \\ \mathbf{Dp}_{k+1} \end{bmatrix}$$

$$\mathbf{p}_k(u) = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} \cdot \mathbf{M}_H \cdot \begin{bmatrix} \mathbf{p}_k \\ \mathbf{p}_{k+1} \\ \mathbf{Dp}_k \\ \mathbf{Dp}_{k+1} \end{bmatrix}$$

Bézier Curves

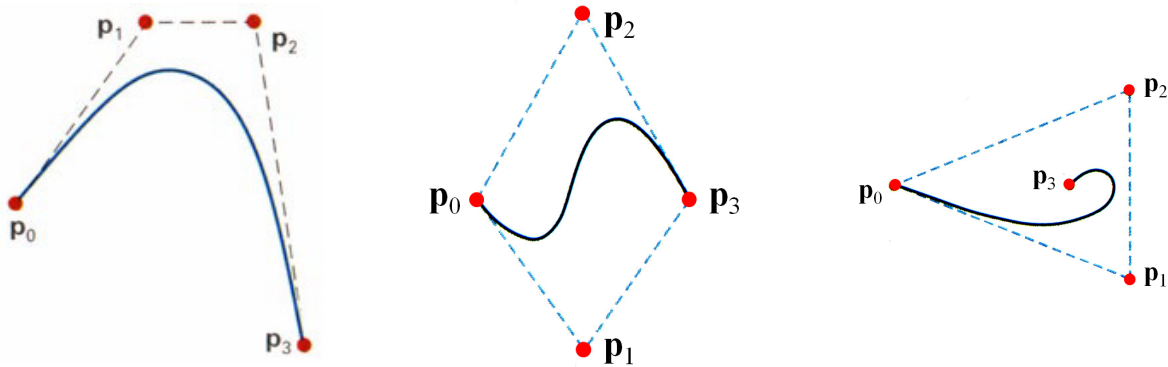
Around 1960, Pierre Bézier described an *approximating* curve for the description of car bodies at Renault, in which the so-called *Bernstein polynomials* $b_{k,n}$ are used as weight functions for the control points. Each curve point is the weighted average of all control points:

$$\mathbf{p}(u) = \sum_{k=0}^n \mathbf{p}_k b_{k,n}(u) \quad 0 \leq u \leq 1 \quad \text{where} \quad b_{k,n}(u) = \binom{n}{k} u^k (1-u)^{n-k} \quad (1)$$

As you can see, the weight functions $b_{k,n}(u)$ are defined over a parameter range u in the range from 0 to 1, and depend on 2 values: n is the number of support points of the curve (strictly speaking, there are $n+1$ support points) and k specifies the support point for which the weight function is used. Each of these values (except at

the edges $u = 0$ and $u = 1$) is greater than zero, i.e. not equal to zero. Since the sum of $b_{k,n}(u)$ over all k is 1 everywhere, each point of the curve $\mathbf{p}(u)$ is therefore a weighted mean value over *all* interpolation points.

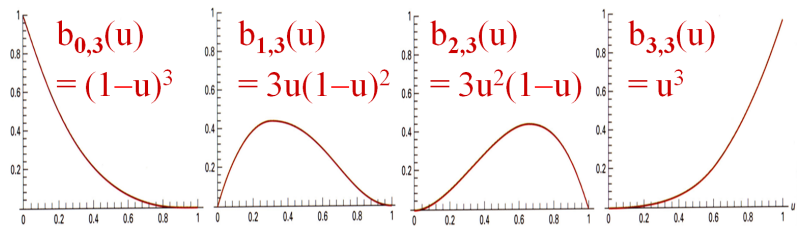
For 4 control points (i.e. $n = 3$), for example: $\mathbf{p}(u) = (1-u)^3 \cdot \mathbf{p}_0 + 3u(1-u)^2 \cdot \mathbf{p}_1 + 3u^2(1-u) \cdot \mathbf{p}_2 + u^3 \cdot \mathbf{p}_3$



Properties of Bézier curves:

- for $n + 1$ control points is $\mathbf{p}(u)$ of degree n
- each control point tightens the curve like a rubber band
- global influence (weight function almost everywhere > 0)
- $\mathbf{p}_0$ and $\mathbf{p}_n$ lie on the curve
- the tangents in $\mathbf{p}_0$ and $\mathbf{p}_n$ are the connection to the next points $\mathbf{p}_1$ and $\mathbf{p}_{n-1}$
- the curve lies entirely in the convex hull of the control points (the convex hull is the smallest convex polygon that contains all control points)

Some of these properties can be recognized from the form of the Bernstein polynomials $b_{k,n}$:

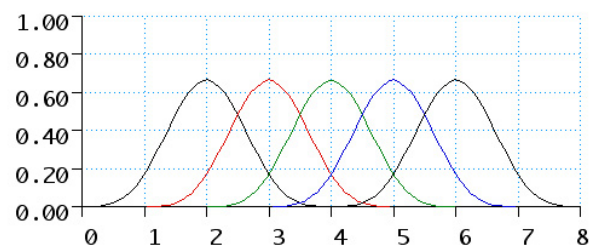


B-Spline Curves

The main disadvantage of Bézier curves is the global influence of the control points on the entire curve. This has two main disadvantages:

1. any change to the control points (insertion, removal, moving) changes the appearance of the curve at all points, and
2. the calculation time for large control point sets is higher.

The reason lies in the form of the weight functions. The so-called *B-splines* approximate curves just like the Bézier splines, but the Bernstein polynomials are replaced by B-spline polynomials $B_{k,d}$. These limit the number of control points that influence a curve point to d . The calculation of $B_{k,d}$ is somewhat more complex and is done recursively, but it is sufficient to see the form of the B-spline polynomials to understand it. It can be seen that each weight curve is only non-zero in a limited range, i.e. that each point has no influence on the curve over wide ranges (the figure shows the weight functions $B_{2,4}$ to $B_{6,4}$ for $n = 8$).

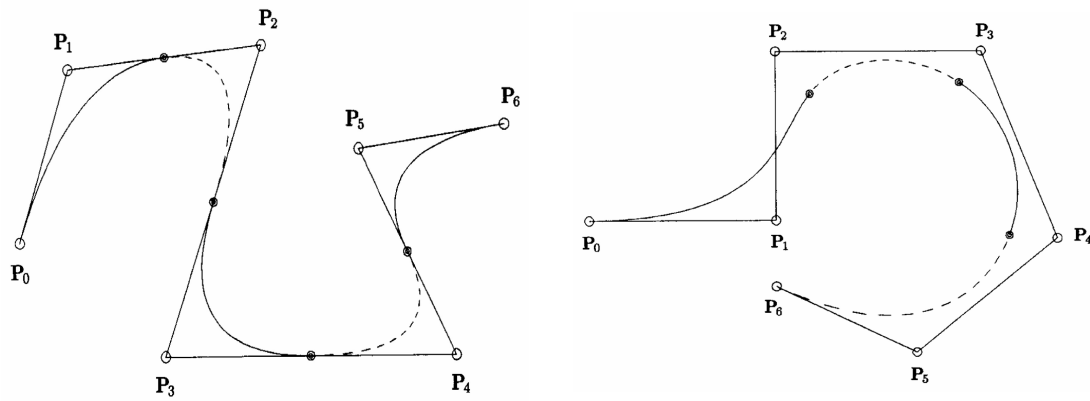


It can be seen that each weight curve is only non-zero in a limited range, i.e. that each point has no influence on the curve over wide ranges (the figure shows the weight functions $B_{2,4}$ to $B_{6,4}$ for $n = 8$).

An important property of the B-spline weight functions is the fact that (as with Bernstein polynomials) their sum is exactly 1 for each curve point. Each B-spline curve point is therefore a *weighted average* of the control points.

$$\sum_{k=0}^n B_{k,d}(u) = 1$$

Examples of B-spline curves with $d=3$ (left) and $d=4$ (right):



If you choose $d = n + 1$, you get Bézier curves, which are a special case of B-splines.

The main differences to Bézier curves are

- *local* influence of the control points
- effort linear to the number of control points (instead of quadratic for Bézier curves)

The most important extensions of these so-called uniform B-splines lead to *Non-Uniform Rational B-Splines*, better known as *NURBS*. These can also be used to consistently represent regular geometric shapes.

Please note that all the methods described apply equally to points in two and three dimensions. In principle, therefore, all these splines describe spatial curves in three-dimensional space.

Free-Form Surfaces

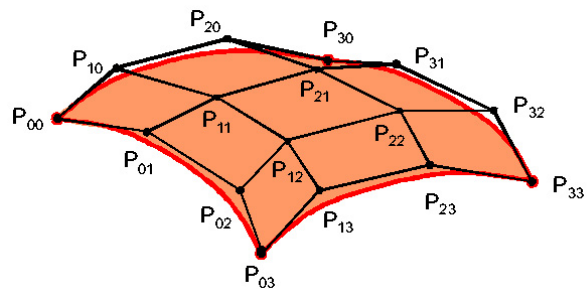
Bézier- and B-Spline Surfaces

If the Cartesian product of two sets of curves is formed over a two-dimensional point grid, free-form surfaces are obtained in a natural way. Depending on the underlying curve type, different surfaces are obtained, e.g. Bézier surfaces from Bézier curves, B-spline surfaces from B-spline curves, etc.

$$\mathbf{p}(u, v) = \sum_{j=0}^m \sum_{k=0}^n \mathbf{p}_{j,k} b_{j,m}(v) b_{k,n}(u)$$

Each pair of parameters (u, v) leads to a point on the resulting surface. In this Bézier example, the boundary curves of the surface are again Bézier curves. The other properties of the Bézier curves are also transferred to the surfaces.

Analogous to these Bézier surfaces, B-spline surfaces are obtained by inserting the B-spline weight functions into the formula, or NURBS surfaces for NURBS curves.



To draw freeform surfaces, triangle meshes can be generated from the surfaces, which are rendered like B-reps. Alternatively, ray-casting methods can be used: to do this, a procedure must be implemented that calculates the intersection point of a straight line with the surface as accurately as possible and also returns the surface normal at said point.

Transformations

An affine transformation, consisting of both a translation and rotation, can be expressed by a homogeneous 4x4 matrix (see ??):

$$\begin{bmatrix} \mathbf{R} & \mathbf{x} \\ 0 & 1 \end{bmatrix}, \text{ with } \mathbf{R} \in \mathbb{R}^{3 \times 3} \text{ being the rotational component and } \mathbf{x} \in \mathbb{R}^3 \text{ representing the translation.}$$

Translation

If we assume no rotation, an object with an initial position $\mathbf{p}^{(t_0)} \in \mathbb{R}^3$ at time t_0 can be moved to a target position $\mathbf{p}^{(t_1)}$ at time t_1 via:

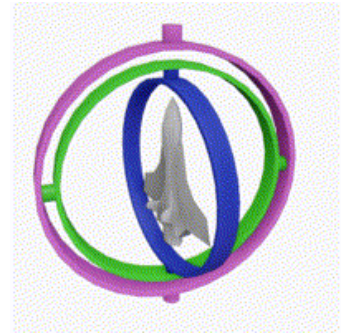
$$\mathbf{p}^{(t_1)} = \begin{bmatrix} 1 & \mathbf{x} \\ 0 & 1 \end{bmatrix} \mathbf{p}^{(t_0)}$$

We can obtain the objects position at an arbitrary time $t \in \mathbb{R}$, with $t_0 < t < t_1$ using *linear interpolation* between the start and end positions:

$$\mathbf{p}^{(t)} = \mathbf{p}^{(t_0)} + (\mathbf{p}^{(t_1)} - \mathbf{p}^{(t_0)}) \frac{t - t_0}{t_1 - t_0}$$

Rotation

Matrix representations of rotations face many difficulties, such as the *Gimbal Lock*: Usually, when you rotate an object in 3D space you have three degrees of freedom. Using matrices for rotation, there are however configurations, where an object is rotated in a way such that two axes are parallel. In the image to the right, for example, the pink and green circle “lock” and it does not make a difference anymore whether you rotate the inner blue ring or the outer pink ring. We thus have lost one degree of freedom.



One solution is to use *Quaternions* to represent rotations and *spherical interpolation*. Quaternions are an extension of complex numbers and consist of a scalar component $s \in \mathbb{R}$ and a vector component $\mathbf{v} \in \mathbb{R}^3$. To rotate around an arbitrary axis $\mathbf{n} \in \mathbb{R}^3$ by an angle ϕ , you can convert this axis-angle representation of a rotation to a quaternion $\mathbf{q}$ as follows:

$$\mathbf{q} = [s; \mathbf{v}] = [s \ x \ y \ z] = [\cos \frac{\phi}{2}; \sin \frac{\phi}{2} \mathbf{n}]$$

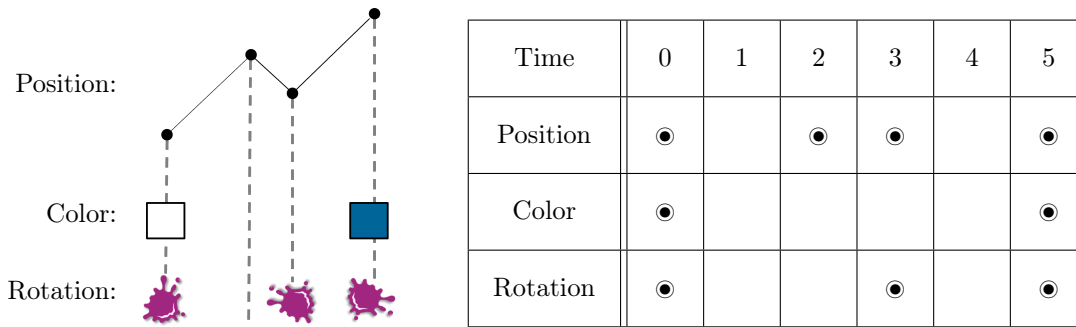
$$\mathbf{q}^{(t)} = \text{slerp}(\mathbf{q}^{(t_0)}, \mathbf{q}^{(t_1)}, t) = \mathbf{q}^{(t_0)} (\mathbf{q}^{(t_0)^{-1}} \mathbf{q}^{(t_1)})^t$$

To finally compute a rotated position $\mathbf{p}^{(t)}$ at an arbitrary time t , we apply the interpolated rotation $\mathbf{q}^{(t)}$ to the initial position $\mathbf{p}^{(t_0)}$. To achieve this, we first form a new quaternion by taking $\mathbf{p}^{(t_0)}$ as the vector component and zero as the scalar component. The result of the product with the computed rotation $\mathbf{q}^{(t)}$ is again a quaternion with zero as the scalar component and the vector component is our desired, rotated position $\mathbf{p}^{(t)}$:

$$[0; \mathbf{p}^{(t)}] = \mathbf{q}^{(t)} \cdot [0; \mathbf{p}^{(t_0)}] \cdot \mathbf{q}^{(t)^{-1}}$$

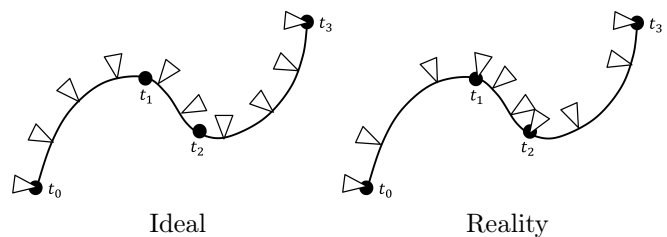
Keyframing

Keyframing achieves complex animations with a balance between automation and manual specification. Scene parameters are specified only at specific points in time and otherwise interpolated. We can encode parameter changes over time in a table as seen below. In particular, we refer to the time steps as frames and we call each combination of a time t and set of scene parameters $\mathbf{f}$ at that point in time a *keyframe* k . In our case this set of scene parameters $\mathbf{f}$ consists of position $\mathbf{p}$, color $\mathbf{c}$ and a rotation represented by a quaternion $\mathbf{q}$, thus yielding keyframe $k : (t_k, \mathbf{f}^{(t_k)}) = (t_k, (\mathbf{p}^{(t_k)}, \mathbf{c}^{(t_k)}, \mathbf{q}^{(t_k)}))$. Note, however, that you may also only specify a subset of the scene parameters as is the case for frames 2 and 3 in our example.



Instead of linearly changing an object's position, fitting an interpolating curve through all keyframe positions results in smoother movement. Depending on the curve construction method, there can be sudden jumps around the keyframe positions and the speed or velocity the object has along the curve may vary non-uniformly (which is bad for smooth camera movement for example).

Ideally, when we sample the time uniformly, we would expect our spatial sampling points also to be uniformly spread along the curve. In reality, however, most curve fitting techniques cluster the spatial samples in some way. One solution to this problem would be, to not uniformly sample the time. Instead, we could approximate the length of the curve called arc length, and then uniformly sample this arc length, meaning we divide the curve into segments of equal length.



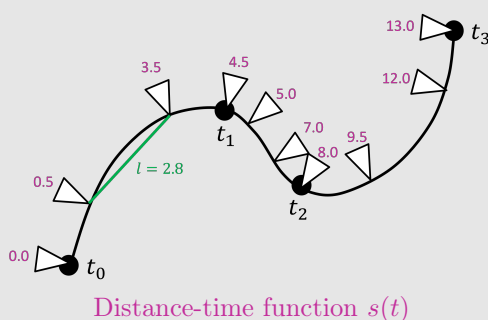
To achieve this, we introduce the *distance-time function* $s(t) : \mathbb{R} \rightarrow \mathbb{R}$. It basically tells us how far we have already traveled along a curve, meaning it assigns each time the distance up until this point. While $s(t)$ is originally continuous, we can discretize it and approximate the distance between two-time samples by using their Euclidian distance, thus approximating the shape of the curve by a number of linear line segments:

$$s(t_i) \approx s(t_{i-1}) + \|\mathbf{p}_i - \mathbf{p}_{i-1}\|$$

We can store these approximated values of our distance-time function s in a look-up table. When we then uniformly sample $s(t)$, we recompute the values t , which we need to evaluate our curve using this look-up table and linear interpolation. By non-uniformly sampling the time, we have thus achieved an approximately uniform space sampling.

Example:

In the following we assume we have some way of computing the exact distance-time function, leading to the **pink values** in the figure below. You can see an example of computing an approximation of $s(t)$ at time $t = 0.\bar{6}$ on the right using the **Euclidean distance** between $\mathbf{p}_{0.\bar{3}}$ and $\mathbf{p}_{0.\bar{6}}$ (also assumed to be given).



t	$s(t)$
0	0.0
$\frac{1}{3}$	0.5
$\frac{2}{3}$	3.5
1	4.5
$1\frac{1}{3}$	5.0
$1\frac{2}{3}$	7.0
...	...

Look-up table

Approximate $s(t)$:

$$s(t_{0.\bar{6}}) \approx s(t_{0.\bar{3}}) + l = 0.5 + 2.8 = 3.3$$

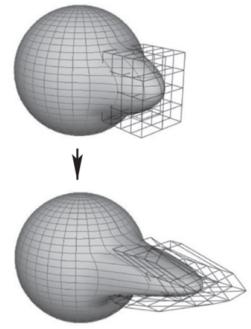
By comparing our approximation of 3.3 to the real value of 3.5 stated in the figure on the right, we notice we made an error of 0.2.

Next, we uniformly sample $s(t)$ at e.g. 0.0, 1.0, 2.0, ... and recompute the values t at which we want to evaluate our curve using linear interpolation. For example, when we sample $s(t)$ at $s = 6.0$:

$$t = \frac{1.\bar{3} + 1.\bar{6}}{2} = 1.5, \quad \text{as } 6.0 \text{ is in the middle of } 5.0 \text{ and } 7.0 \text{ (see table).}$$

Deformations

Deformations can be formalized as applying a function f to each vertex position $\mathbf{p}$ to compute the new, deformed position $\mathbf{p}' = f(\mathbf{p}, \gamma)$, based on some additional parameters γ . Some simple deformations can be applied in the form of non-constant matrices – for example the so-called *tapering*, *bending*, and *twisting* operations. Alternatively, if you want to model a not-as-nicely-behaved deformation, one solution might be to use a *deformation cage* to reduce the number of vertex positions you manually have to specify. The vertex positions of the object are “bound” to the cage and whenever the cage is moved, their position changes as well.



Physically-based Simulation

An object is represented by multiple points connected via constraints and simulated based on physical laws.

Point:

- Position $\mathbf{p} \in \mathbb{R}^3$, $[m]$
- Velocity $\mathbf{v} = \frac{d\mathbf{p}}{dt} \in \mathbb{R}^3$, $[m \cdot s^{-1}]$
- Acceleration $\mathbf{a} = \frac{d\mathbf{v}}{dt} = \frac{d^2\mathbf{p}}{dt^2} \in \mathbb{R}^3$, $[m \cdot s^{-2}]$
- Mass $m \in \mathbb{R}$, $[kg]$

Constraint:

- *Rigid*: Distance/Volume stays the same, Transformation only (wooden cubes, steel sword)
- *Soft*: Distance/Volume can change, Transformations & Deformations (cloth, soft tissue)

Our main interest lies in simulating the positions $\mathbf{p}$. To achieve this, we also consider the change of positions over time, which is called *velocity*. If the velocity $\mathbf{v}$ is constant (meaning it is not time-dependent), we can compute the future position $\mathbf{p}^{(t+\Delta t)}$ of a point after taking an arbitrary timestep Δt based on the current position $\mathbf{p}^{(t)}$ at time t as follows:

$$\mathbf{p}^{(t+\Delta t)} = \mathbf{p}^{(t)} + \Delta t \cdot \mathbf{v}$$

This way, the motion of the point would be at constant speed along a straight line (Newton's first law). We can change the velocity by exerting a force $\mathbf{F} \in \mathbb{R}^3$, $[kg \cdot m \cdot s^{-2}]$, such as gravity, upon our body. The change of velocity is called *acceleration* $\mathbf{a}$, which can be computed using Newton's second law $\mathbf{F} = m \cdot \mathbf{a} \Rightarrow \mathbf{a} = \frac{\mathbf{F}}{m}$. If we assume both a varying velocity and a varying force, computing $\mathbf{p}^{(t+\Delta t)}$ involves solving multiple integrals.

Numerical Integration

Explicit Euler:

$$\begin{aligned}\mathbf{p}^{(t+\Delta t)} &= \mathbf{p}^{(t)} + \Delta t \cdot \mathbf{v}^{(t)} \\ \mathbf{v}^{(t+\Delta t)} &= \mathbf{v}^{(t)} + \Delta t \cdot \mathbf{a}^{(t)} \\ \mathbf{a}^{(t)} &= \frac{\mathbf{F}^{(t)}}{m}\end{aligned}$$

unstable at large timesteps
(high deviation/oscillations),
easy to compute,
real-time

Implicit Euler:

$$\begin{aligned}\mathbf{p}^{(t+\Delta t)} &= \mathbf{p}^{(t)} + \Delta t \cdot \mathbf{v}^{(t+\Delta t)} \\ \mathbf{v}^{(t+\Delta t)} &= \mathbf{v}^{(t)} + \Delta t \cdot \mathbf{a}^{(t+\Delta t)} \\ \mathbf{a}^{(t+\Delta t)} &= \frac{\mathbf{F}^{(t+\Delta t)}}{m}\end{aligned}$$

unconditionally stable,
computationally evolved
(requires solving equation system),
offline

Semi-Implicit Euler:

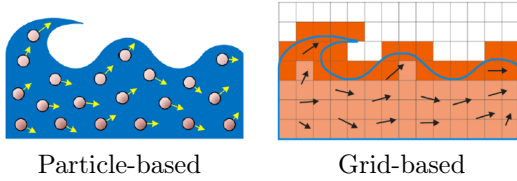
$$\begin{aligned}\mathbf{p}^{(t+\Delta t)} &= \mathbf{p}^{(t)} + \Delta t \cdot \mathbf{v}^{(t+\Delta t)} \\ \mathbf{v}^{(t+\Delta t)} &= \mathbf{v}^{(t)} + \Delta t \cdot \mathbf{a}^{(t)} \\ \mathbf{a}^{(t)} &= \frac{\mathbf{F}^{(t)}}{m}\end{aligned}$$

energy conserving,
easy to compute,
real-time

```
1 //Explicit Euler
2 Vec3 F = computeForce();
3 Vec3 acceleration = F * object.inverseMass;
4 object.position += object.velocity * deltaTime;
5 object.velocity += acceleration * deltaTime;
```

```
1 //Semi-Implicit Euler
2 Vec3 F = computeForce();
3 Vec3 acceleration = F * object.inverseMass;
4 object.velocity += acceleration * deltaTime;
5 object.position += object.velocity * deltaTime;
```

Differential Equations



Particle-based

Grid-based

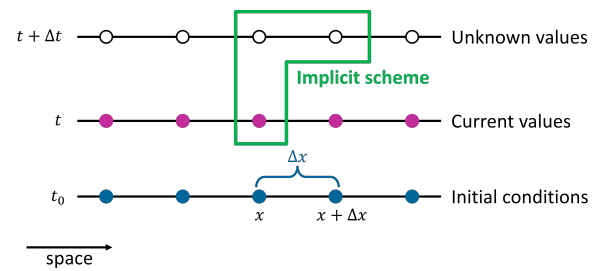
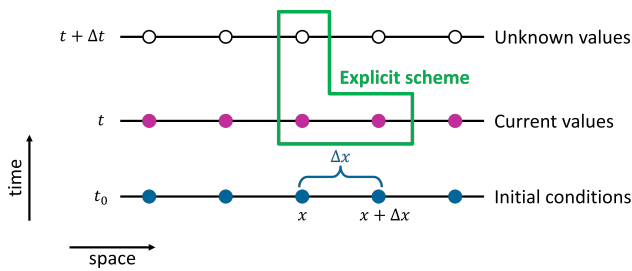
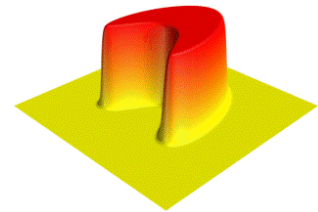
So far, we have looked at so-called particle-based simulations, where we approximate a shape by multiple points. This is also referred to as the *Lagrangian view* on simulation. Another approach is to divide the space using an often uniform grid, which is then called the *Eulerian view*. Our ultimate goal in physics-based simulation is to solve differential equations.

Differential equations are simply equations, which contain a function and its derivatives. So far we have looked at *ordinary differential equations* (ODEs), which only contain full derivatives (written as $\frac{d\mathbf{p}}{dt}$, $\mathbf{p}'(x)$ or sometimes $\dot{\mathbf{p}}$), such as Newton's second law $\mathbf{F}(t, \mathbf{p}, \mathbf{v}) = m \frac{d^2\mathbf{p}^{(t)}}{dt^2}$.

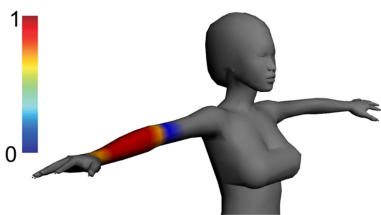
A bit more complex are *partial differential equations* (PDEs), which contain a multi-dimensional function with multiple parameters and its partial derivatives (written as $\frac{\partial \mathbf{p}}{\partial x}$ or $\partial_x \mathbf{p}$) with respect to only one of these parameters. An example is the 2D heat equation (2), which describes the way heat diffuses along a 2D surface. In the image below you can see a heightfield visualization of an exemplary scalar heat field. Other use cases are for example acoustic waves or fluid simulations (here the underlying PDE is the Navier-Stokes Equation). One technique to numerically compute derivatives (most PDEs do not have an analytical solution) are so called *Finite Difference Methods* (see for example 3). The unknown values are then computed using again explicit or implicit schemes.

$$\frac{\partial h}{\partial t} = \alpha \left(\frac{\partial^2 h}{\partial x^2} + \frac{\partial^2 h}{\partial y^2} \right), \quad \text{with } h : \Omega \subset \mathbb{R}^2 \rightarrow \mathbb{R} \quad (2)$$

$$\frac{\partial u(x, t)}{\partial x} \approx \frac{u(x + \Delta x, t) - u(x, t)}{\Delta x} \quad (3)$$



Character Animation



Linear blend skinning (LBS) is a simple method of computing updated skin vertex positions based on the deformation of an underlying skeleton resulting from *rigging* a character. Each joint j is assigned a transform $\mathbf{T}_j$ (which can also depend on a hierarchy of joints and for example be represented as a matrix $\mathbf{T}_j \in \mathbb{R}^{4 \times 4}$). Each joint j has an individual influence on each vertex i , captured by the weight $w_{i,j}$ (see image on the right). To now move a vertex i from its original position $\mathbf{p}_i$ to a deformed position $\mathbf{p}'_i$, we sum over the weighted contributions of all joints (n is the number of joints): $\mathbf{p}'_i = \left(\sum_{j=1}^n w_{i,j} \mathbf{T}_j \right) \mathbf{p}_i$

LBS comes with multiple issues, however, such as high manual effort or the “candy-wrapper” artifact you can observe in the left image. Alternatively, we could use *ragdolls* based on physically-based simulation or *motion capture*, where the movement of real humans is captured directly via cameras and markers on the actors, and then transferred to a virtual character.



Procedural Techniques

Similar to physically-based simulations, procedural techniques can be used to automatically create animations. They follow a certain set of rules combined with randomization to create plausible animations, however not with the goal of physical correctness. One famous example is the Game of Life, which is a cellular automata. Other procedural methods involve particle systems (rain, snow, fire) or boid/swarm behaviour.

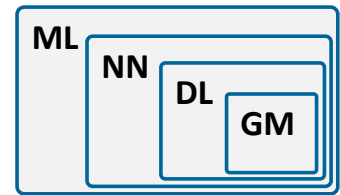
Machine Learning and Neural Networks

Machine Learning (ML) is a branch of artificial intelligence focused on developing algorithms that learn from data to make predictions or decisions. It involves optimizing parameters to model relationships within data. A basic example is linear regression, where parameters are adjusted to best fit a set of data points.

Neural Networks (NNs), fundamental to ML, consist of interconnected nodes that process inputs to produce outputs. They work by multiplying input vectors with weight matrices and applying activation functions like *ReLU* or *Sigmoid*. Despite their simplicity, these networks are powerful enough to solve a wide range of tasks.

Deep Learning (DL) is a subset of ML that uses neural networks with many parameters to tackle complex problems. DL excels in tasks like image classification, where images are sorted into categories, and segmentation, where different parts of an image are identified. It can also handle style transfer, which modifies the style of images while keeping their content intact, and novel view synthesis, which creates new perspectives of 3D objects from limited data.

Classifiers are a machine learning application where high-dimensional input data, such as images, are transformed into lower-dimensional output, typically a class label. Class labels can, for example, be represented by categories like “human”, “dog”, or “bird”. The trapezoid symbol in classifiers represents this dimensional reduction, indicating how raw data is condensed into specific categories through the classification process.

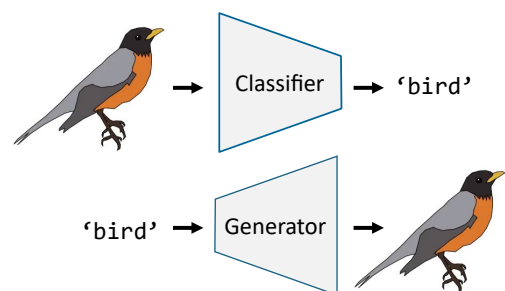


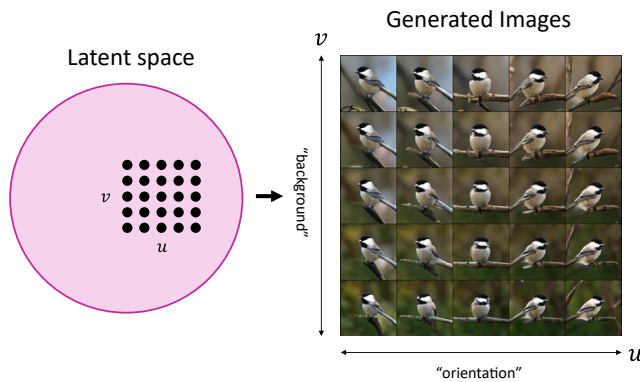
Generative Models

Generative Models (GM), a subset of DL, are a class of statistical models that are used for generating new data instances. These models learn to capture the probability distributions of the data they are trained on, enabling the generation of new data points with similar characteristics. Key types of generative models include:

- **Auto-Encoders** which learn to compress input data into a smaller representation and then reconstruct the output from this representation.
- **Generative Adversarial Networks (GANs)**, which consist of a generator that creates samples and a discriminator that evaluates them.
- **Diffusion Models** use the idea of reverse diffusion (i.e. noise removal) to iteratively generate output from random noise.

Conceptually, a generative model can be thought of as running a classifier backward. In a classifier, a high-dimensional input such as an RGB image is assigned a class in a lower dimensional space consisting of a number of class labels (this reduction of dimensionality is represented by the symbol for a classifier ∇). In a generative model, on the other hand, we take input with low dimensionality such as a text prompt, and want to output an RGB image (hence the symbol for a generator is a mirror image of the classifier symbol: ∇). However, we do not want to just reconstruct one instance of a bird, for example, but we want different instances of birds and also a way to control which instances are to be created. This necessitates additional inputs into the generator. To achieve this variability, random noise is introduced. This noise serves to map from a distribution of random inputs to a desired distribution of outputs – specifically, various representations of birds.





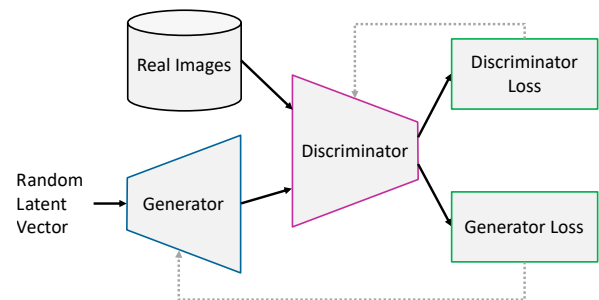
Inside the model, the data are contained within a high-dimensional latent space. This space comprises points defined by latent variables, which encode characteristics of our bird such as color and size. Initially, these variables are randomized. However, to exert control over specific attributes of the generated output, they are deliberately manipulated. For example, in a two-dimensional representation of this space, the “ u ” coordinate might dictate the orientation of the bird (left, front, right), while the “ v ” coordinate could determine the background context (blue sky, green leaves). However, the latent variables are not always perfectly distinct; changes in one can inadvertently affect others, such as a right-facing bird causing the background to appear brown—a phenomenon known as variable entanglement.

Using Generative Models to Create Images

So far, we have seen a high-level overview of the input and output of a generative model. Now we take a look at what is necessary to actually build a generative model for creating images. Here, our first step is selecting an *architecture*, such as Generative Adversarial Networks (GANs) or Diffusion Models. The *training* process involves using a large dataset of images, sometimes with labels, to teach the model to generate new images accurately. During *inference*, the model applies what it has learned to produce new images based on new inputs. This allows for the controlled generation of specific features in the output, like color and size.

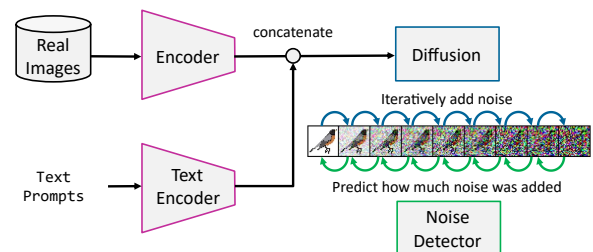
Training a Generative Adversarial Network (GAN)

In Generative Adversarial Networks (GANs), two neural networks, the Generator and the Discriminator, engage in a competitive game. The Generator’s role is to produce images from random latent vectors that are indistinguishable from real images, effectively trying to “fool” the Discriminator. Conversely, the Discriminator’s job is to accurately distinguish between the generated images and actual real images. This process involves two loss functions which are simultaneously optimized to improve both the Generator’s ability to deceive and the Discriminator’s ability to detect fakes. It is crucial that the loss functions are differentiable to support back-propagation during training, enhancing the learning and adaptation capabilities of both networks.



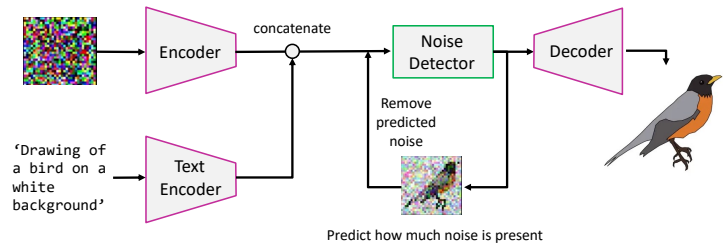
Training a Diffusion Model

Diffusion models are trained by encoding both images and textual prompts into a shared latent space, where noise – typically Gaussian for its probabilistic properties – is iteratively added to the images. This added noise allows the model to learn from the ‘noisy’ data using a noise detector that predicts the level of noise at each step. The purpose here is for the denoiser to learn to predict and subsequently subtract the noise based on the noisy image and associated text input. This learning process ensures that each iteration moves the image closer to a less noisy and more accurate representation.



Inference in a Diffusion Model

During the inference phase of a Diffusion Model, the generation process begins with a text prompt and a patch of random Gaussian noise. Both are encoded into the latent space and continually refined through a series of noise detection and subtraction steps. At each iteration, the noise detector estimates the amount of noise present, and this noise is then subtracted to gradually clarify the image. This sequence continues until the image is sufficiently denoised, at which point the latent representation is decoded back into a visual image by a decoder, typically structured as a Variational Auto-Encoder. The resulting images, which start from generalized noise and text inputs, demonstrate the model's capability to generate detailed and contextually appropriate visuals from a high-dimensional latent space.

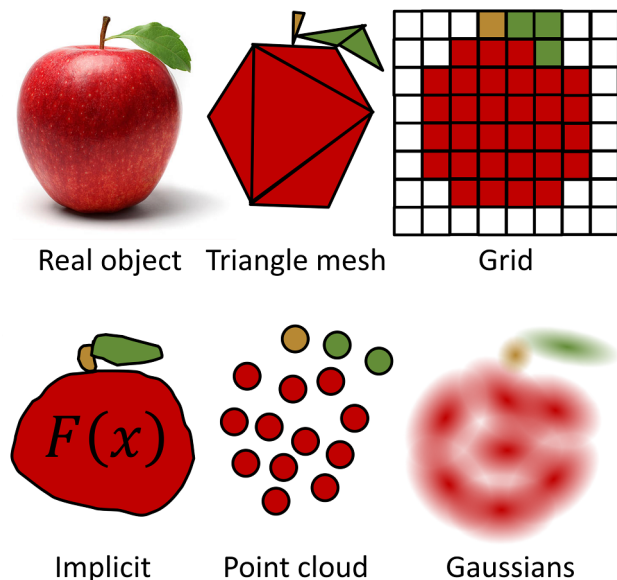


Now, we have a tool to generate 2D images given some text input; however, what we want is to generate 3D models and scenes. For that, we need a bit more.

Scene Representations for 3D Machine Learning

A real object cannot be accurately represented so that the representation captures every single detail. Therefore, there are many different representations, each with its own advantages and disadvantages. In general, they can be subdivided into two categories: implicit and explicit. Implicit representations are functions that at each point in space define a value, whether the object is there or not, its color, or how far away is the closest surface (e.g., Signed Distance Functions (SDFs)). Explicit representations are usually sets of discrete elements, e.g., a triangle mesh, grid, or point cloud. Depending on the task, some representations may be more suitable than others.

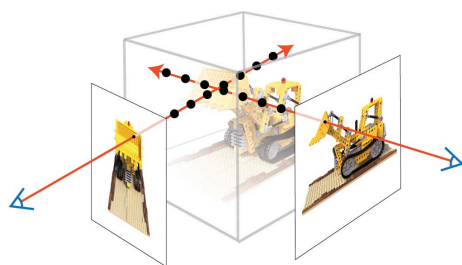
Let us take for example point clouds. If we scan a real object and get a finite set of points, there is no straightforward way to ensure that if we render the points, we will have no holes through which we can see what is behind the object. We could try to “close” them and construct a mesh. However, when training with the mesh, it is important to ensure it does not become deformed and invalid, which can occur if vertices move freely and faces intersect. Therefore, it is always important to consider which representation is suitable.



Let us take for example point clouds. If we scan a real object and get a finite set of points, there is no straightforward way to ensure that if we render the points, we will have no holes through which we can see what is behind the object. We could try to “close” them and construct a mesh. However, when training with the mesh, it is important to ensure it does not become deformed and invalid, which can occur if vertices move freely and faces intersect. Therefore, it is always important to consider which representation is suitable.

Neural Radiance Fields (NeRFs)

NeRFs offer an implicit representation of spatial data, utilizing the weights within a neural network to model 3D environments. This process is classified under deep learning within the machine learning hierarchy, requiring substantial computational resources. The core mechanism of a NeRF involves sampling along a ray in 3D space, a method driven by a multi-layer perceptron (MLP). As the ray traverses the scene, the model evaluates the color and density at various points along the path, ultimately projecting and blending these values to produce the final image. This projection condenses the volumetric data into a singular color value per sampled ray, representing a highly detailed rendering of the scene.



If we wish to reconstruct a real scene, we need ground truth images. We can use the Structure-from-Motion algorithm to obtain their corresponding camera poses, from which the scene gets rendered. Then, we can compare the rendered image with its corresponding ground truth image and optimize the network to match them. NeRFs rely on neural networks, which are black boxes. This means that directly changing one part of a scene, e.g., removing an object, is very difficult. Therefore, before editing it may be necessary to first convert it into an explicit representation, e.g., a mesh using the marching cubes algorithm. Nevertheless, NeRFs have proven to be very powerful and are widely used nowadays.

Gaussian Splatting

In contrast, Gaussian Splatting is an explicit scene representation technique that leverages the principles of volume rendering, but employs Gaussian functions as primitive shapes. Unlike NeRFs, Gaussian Splatting does not require deep neural networks and can be thought of as a soft point cloud, where each Gaussian is defined by its mean (position), covariance matrix (shape), opacity, and color. Instead of a NN, it utilizes a straightforward machine learning approach, such as stochastic gradient descent, to learn the spatial features of the scene. The rendering process in Gaussian Splatting involves projecting the 3D Gaussians onto a 2D plane, and then rasterizing the projected data to form the image. Each Gaussian's contribution to the final image is sorted and sampled in the 2D space, allowing for efficient rendering without the intense computational demand of deep learning architectures.

The training for reconstruction starts with an initial set of Gaussians. These Gaussians can be obtained using the Structure-from-Motion algorithm, which outputs (together with the camera poses) a set of 3D points that match the ground truth images. Then, the points can be directly transformed into Gaussians by setting their covariance matrices, so that they and their close neighbors create surfaces without holes, but potentially with a considerable overlap. As the initial set of Gaussians may not be enough to represent all the details in the scene, they need to be either cloned or split, which can be determined by the accumulated magnitude of their positional gradients flowing into individual Gaussians. This magnitude indicates how badly placed the Gaussians are. So, if the gradient is large, it either indicates that it needs to move somewhere rapidly or that it needs to be at multiple different locations at once—so it gets cloned or split. This way we can obtain a high-quality 3D representation with more Gaussians and thus more details where needed.

Generative Models for 3D Objects

At this point, we know how to generate 2D images and how to reconstruct a real-life scene. But what about combining these two? We can utilize the process of iteratively changing/denoising images to generate new ones together with iteratively changing a 3D object representation. Here we describe how to combine a diffusion model and 3D Gaussian Splatting to synthesize new 3D objects.

Once we know what we want to create, i.e., we have a text prompt, we need to get an initial set of Gaussians that will be further refined. This can either be done manually or by using a pre-trained network that can generate (nowadays usually rough and not particularly good-looking) point clouds based on a text prompt. These points can be converted into Gaussians using the same process as during scene reconstruction. They then get rendered from random viewpoints

(“Splatting Renderer”). These renderings are 2D images and we can use a diffusion model to inform how they should be changed to match the prompt. The only thing missing is that diffusion models work with noisy images and thus we need to add noise artificially. Based on how the diffusion model wants to change the rendered images, we can change the underlying 3D object representation. Furthermore, we need to split and clone Gaussians to add more detail, which, again, can be done by considering the magnitude of their gradient. After several hundreds or thousands of iterations, we have a new 3D object created automatically.

